Spring Knowledge Boost

Category	Count
Spring Conceptual Questions	130
Rest API Questions	30
Spring Scenario Questions	56
Testing Questions	25
Indirect Questions	366
Functional / Behavioral Questions	31
Experience wise Questions	
Spring Cheat Sheet	

Interviewers usually focus on five major areas, and this document is designed to cover each one of them

1. Theory interview questions and Indirect questions.

2. Algorithm based interview questions.

3. Output based interview questions.

4. Scenario based interview questions.

5. Machine coding round.

Interview Tips (Technical):

1. Understand Core Java Concepts

Know the basics of Java well, including Object-Oriented Programming (inheritance, polymorphism, encapsulation, abstraction), data structures, algorithms, exception handling, multithreading, and the Collections Framework. Be ready to explain how these are used in your backend projects.

2. Learn Spring Framework Well

Be familiar with Spring and Spring Boot. Understand dependency injection, AOP, Spring Data JPA, Spring Security, and transaction management. Be able to explain how you used these features in your applications. Know the common Spring annotations.

3. Know RESTful API Design

Understand REST principles like statelessness, resource-based design, and proper use of HTTP methods (GET, POST, PUT, DELETE). Be ready to explain how to create secure, fast, and well-documented REST APIs using Spring.

4. Show Problem-Solving and System Design Skills

Be prepared to talk about tough technical problems you solved. Show that you can think

about system design, performance, scalability, and fault tolerance. If you have experience with microservices or other architectures, explain it.

5. Be Good at Testing and Debugging

Explain how you test backend applications using unit tests, integration tests, or end-to-end tests. Mention tools like JUnit and Mockito. Show that you have written clean, maintainable, and well-tested code.

Interview Tips (General):

1. Discuss past projects using the STAR method

- Situation: Describe the context or challenge you faced.
- Task: State your specific responsibility in your project.
- Action: Explain the steps you took, decisions made, and collaboration involved to complete the task
- Result: Share the outcome and impact, preferably with metrics, e.g., reduced report generation time by 50%.

2. Communicate Clearly: Explain the concepts with examples or real time examples where you have used in your project

3. Highlight Collaboration Skills: Backend development involves working with frontend, DevOps, or QA teams. Share examples of teamwork and communication to show you fit in a collaborative environment.

4. Ask Smart Questions: Prepare 2–3 thoughtful questions about the team, architecture, or development practices. This shows interest and that you think beyond coding.

Click on any question to go straight to the answer.

Table of Contents

Spring

58 Spring 1: What is Spring boot?

Spring Boot is a framework built on top of Spring that makes it faster and easier to develop Java applications. Normally with Spring, we have to do a lot of manual setup, configuration, and XML files. Spring Boot removes that pain by giving auto-configuration, embedded servers, and prebuilt dependencies. So, we can just focus on writing business logic instead of wiring everything.

Indirect Question:

1. What's new in Spring boot 3.x?

Spring Boot 3.x makes some big changes. First, it switches from the old javax.* packages to the new jakarta.* ones because it now uses Jakarta EE 10. It also requires Java 17 or higher, so we get access to modern Java features. Another big addition is support for GraalVM native images, which lets us build apps that start super-fast and use less memory , perfect for cloud and container environments.

2. Can we create a non-web application in Spring boot?

Yes, Spring Boot can be used to build non-web applications. We can exclude the web-starter dependency and use CommandLineRunner or ApplicationRunner to run code at startup. It is often used for batch jobs, schedulers, or standalone services.

59 Spring 2: What are the advantages of Spring boot?

Spring Boot is a framework that creates stand-alone, production grade Spring based applications. So, this framework has so many advantages.

- **Easy to use:** The majority of the boilerplate code required to create a Spring application is reduced by Spring Boot.

- **Rapid Development:** Spring Boot's opinionated approach and auto-configuration enable developers to quickly develop apps without the need for time-consuming setup, cutting down on development time.
- **Scalable:** Spring Boot apps are intended to be scalable. This implies they may be simply scaled up or down to match the application's needs.
- **Production-ready:** Metrics, health checks, and externalized configuration are just a few of the features that Spring Boot includes and are designed for use in production environments.
- **Hot reloading** allows developers to make changes to their code, resources, or configuration files while an application is running, and see those changes immediately reflected without needing to **restart the application.**

60 Spring 3: What are embedded containers / servers in Spring and what is the default one? how to add Jetty for example?

In normal Java web apps, we used to deploy the WAR file into an external server like Tomcat or Jetty. That meant installing and managing the server separately.

Spring Boot ships with an embedded server inside the application.

- The app is packaged as a JAR (not WAR).
- When we run it, the server starts along with the code.
- We don't need to deploy to any external container.

What's the default server?

By default, Spring Boot uses Apache Tomcat as the embedded server.

So, if we don't configure anything, we are already running on Tomcat without even realizing it.

Suppose we prefer Jetty instead of Tomcat. Here is what we do in

Maven: Exclude Tomcat (since it comes by default) and Add Jetty

dependency:

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
        <exclusions>
                <exclusion>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-tomcat</artifactId>
                </exclusion>
        </exclusions>
</dependency>
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-jetty</artifactId>
```

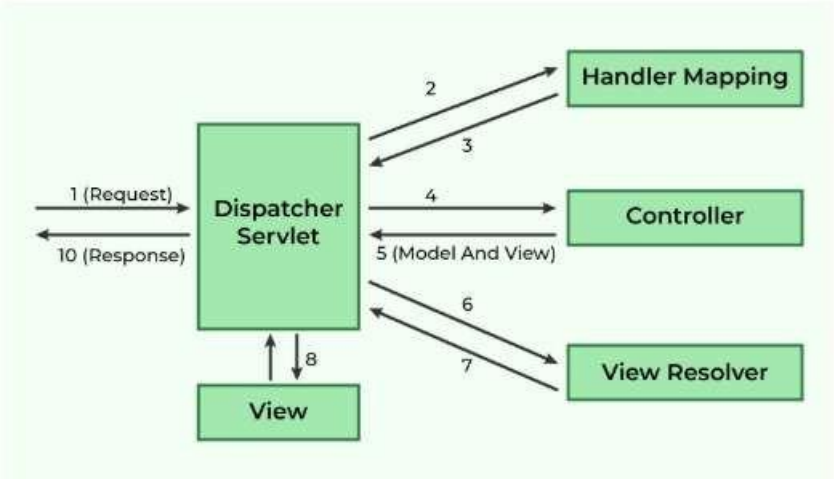
</dependency>

61 Spring 4: Difference between Spring and Spring Boot?

- **Spring**: It is a framework that gives us tools for dependency injection, data access, and building applications. But it requires a lot of setup, XML configurations, and deployment steps.
- **Spring Boot**: It is built on top of Spring and provides auto-configuration, embedded servers, and starter dependencies. It cuts down setup time and allows us to build production-ready apps with less effort.

  In short: Spring is the foundation; Spring Boot is the fast and modern way to use Spring.

62 Spring 5: Explain Spring MVC architecture?



The Spring MVC architecture is a Model-View-Controller (MVC) framework used for building web applications in Java. It promotes a clear separation of concerns, making applications more modular, maintainable, and easier to develop.

Key Components of Spring MVC Architecture:

- **DispatcherServlet (Front Controller):**
  - Acts as the central entry point for all incoming HTTP requests.
  - Initializes the Spring application context and loads configurations.
  - Delegates requests to appropriate controllers based on configured mappings.
- **HandlerMapping:**
  - Maps incoming requests to specific controller methods.
  - Determines which controller should handle a particular request based on URL patterns, HTTP methods, and other criteria.

- **Controller:**
    - Handles user input and business logic.
    - Processes requests, interacts with the Model (business logic and data), and prepares data for the View.
    - Returns a ModelAndView object, which contains the view name and the model data.
- **ModelAndView:**
    - A container object that holds both the model data and the name of the view to be rendered.
- **ViewResolver:**
    - Resolves the logical view name returned by the controller into a concrete View implementation (e.g., a JSP page, Thymeleaf template).
- **View:**
    - Renders the user interface, displaying the data from the Model.
    - Responsible for generating the final output (e.g., HTML, JSON, XML) to be sent back to the client.

63 Spring 6: What is @SpringBootApplication?

It is a combination of three important Spring annotations:

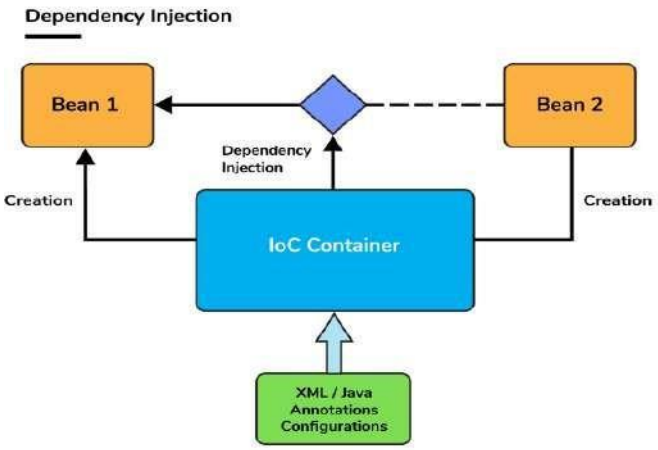
- **@EnableAutoConfiguration**: Automatically configures beans depending on what's on the classpath. For example, when we illustrate the spring-boot-starter-web dependency in the classpath, Spring boot auto-configures Tomcat , and Spring MVC .
- **@ComponentScan** : This annotation scans the components (@Component, @Service, etc.) in the package of annotated class and its sub-packages.
- **@Configuration:** This annotation configures the beans and packages in the class path.

So, using @SpringBootApplication at the main class makes the application ready with all these features at once.

64 Spring 7: What is Dependency Injection?

Dependency injection is a design pattern where objects don't create their own dependencies. Instead, the container (Spring IOC) provides the required objects.

For example, if a Car class needs an Engine, instead of writing new Engine(),Spring inject the engine for us. This reduces coupling, makes testing easier (we can inject mock objects), and improves maintainability.




```
@Component
class Engine {}

@Component
  class Car {
  @Autowired
    private Engine engine; // Spring will inject Engine object here
}
```

Interview Tip:

- Dependency Injection is about providing an object with its dependencies from the outside rather than creating them internally. This makes the code loosely coupled, easier to maintain, and easier to unit test.
- In Spring, we can do this using constructor injection, setter injection, or field injection with annotations like @Autowired and @Qualifier.
- Mention about how you implemented a repository and service layers in your project. Example: In a service class which needs repository, we inject the repository instead of creating it, which makes testing and maintenance straightforward.

Indirect Questions:

1.How does Spring achieve Inversion of Control (IoC)?

IoC is achieved in Spring through Dependency Injection (DI), where the Spring container manages the lifecycle and dependencies of the objects, freeing the developer from manually instantiating dependencies.

2. How does Spring boot makes Dependency injection easier compared to traditional Spring MVC?

Spring boot makes dependency injection easier compared to traditional Spring by auto-configuring the beans and reducing the need for explicit configuration. In traditional Spring, we had to define beans and their dependencies in XML files or with annotations, which can be complex for large applications.

But in Spring boot, we use Auto-configuration and component scanning to automatically discover and register beans on the applicaiton's context and classpath. This means now we don't have manually wire up beans.

65 Spring 8: What is IOC Container?

IOC (Inversion of Control) container is the core part of Spring. It is responsible for:

- Creating objects (beans).
- Injecting dependencies.
- Managing the life cycle of those objects.

Instead of we controlling how objects are created and connected, the control is inverted to the container. Spring will take care of creating the beans and managing them. When ever we are ask for some object, Spring will provide it. There are two main types: **BeanFactory** (basic) and **ApplicationContext** (advanced, most used).

```
                car.drive();
        }
}
```

Here, the **ApplicationContext** (IOC container) manages all beans (Car, Engine, etc.). We just ask the container for the bean instead of creating it ourselves.

Reference: Understanding IoC Container in Spring Boot with a Real-Time Project Example | by A cup of JAVA coffee with NeeSri | Medium

```
@SpringBootApplication
public class DemoApplication {
            public static void main(String[]
        args) { ApplicationContext context =
                        SpringApplication.run(DemoApplication.class, args);

                Car car = context.getBean(Car.class);
```

Indirect Questions:

1. What are the types of IOC containers in Spring?

a. BeanFactory: BeanFactory is like a factory class that contains a collection of beans. It instantiates the bean whenever asked for by clients.

b. ApplicationContext: The ApplicationContext interface is built on top of the

BeanFactory interface. It provides some extra functionality on top BeanFactory.

2. What is the default container?

ApplicationContext is the default and preferred container, adding enterprise features like events, internationalization, AOP, and annotation support.

3. Difference between BeanFactory and ApplicationContext.

Aspect	BeanFactory	ApplicationContext
Definition	Basic IoC container, provides only DI	Advanced IoC container, built on top of BeanFactory
Bean Initialization	Lazy (beans created only when requested)	Eager by default (all singleton beans created at startup)
Features	Only manages beans	Adds event handling, internationalization, AOP integration, annotation scanning, BeanPostProcessor support
Performance	Lightweight, good for memory-constrained apps	Slightly heavier (loads more at startup) but faster access later
Configuration	XmlBeanFactory (deprecated after Spring 3.x)	ClassPathXmlApplicationContext, AnnotationConfigApplicationContext, WebApplicationContext

Aspect	BeanFactory	ApplicationContext
Use Case	Rarely used today, legacy/simple apps	Default choice for modern Spring & Spring Boot apps
Support	No support for @Autowired,	Full support for annotations, enterprise beans, and Boot autoconfiguration

66 Spring 9: What is the difference between @Component, @Service, and @Repository?

- @Component: The most generic annotation. It marks a class as a Spring bean.
- @Service: A specialization of @Component, used for classes that hold business logic. This makes code easier to understand since we know it belongs to the service layer.
- @Repository: Another specialization, used for persistence layer (DAO classes).
  Point to remember: It also provides automatic translation of database-related exceptions into Spring's DataAccessException.

So basically, they all register beans with the container, but @Service and @Repository give extra meaning and behaviour for their specific layers.

```
@Component // generic bean
class UtilityHelper {}

@Service        // business logic
class PaymentService {
        public void processPayment() {
      System.out.println("Payment processed");
      }
}

@Repository // data access layer
class UserRepository {
        public void saveUser(String name) {
      System.out.println("User " + name + " saved to DB");
      }
```

Indirect Questions:

1. Why do we need to separate controller, business and DAO layers?

Think of it like building a house: We wouldn't ask an electrician to lay the foundation, and we wouldn't ask a plumber to build the roof. Each expert should focus on their own work. In the same, In Spring,
each layer has its job, and separating them keeps things clear, safe, and easier to maintain.

- **Controller layer**: Handles HTTP requests and responses. It is the entry point where the outside world talks to our app.
- **Business (Service) layer**: It contains the business logic
- **DAO (Data Access Object) layer**: Talks to the database. It only cares about saving, reading, updating, and deleting data.

Why separate?

1. **Clean code**: Each part has one clear responsibility.
2. **Easier to change**: We can swap a database or change business rules without touching controllers.
3. **Testability**: We can test each layer in isolation.
4. **Reusability**: Services can be reused across different controllers or APIs.

2. Is it required to write @Repository annotations in spring data JPA?

@Repository annotation is not strictly required when we use Spring Data JPA.

Why? Because when we create a repository interface that extends JpaRepository, CrudRepository, or PagingAndSortingRepository, Spring automatically finds it and creates an implementation at runtime. That implementation is registered as a Spring bean, so we can use it with @Autowired even without @Repository.

```
public interface UserRepository extends JpaRepository<User, Long> {

}
```

So, what is the point of @Repository then?

It is mainly about **exception translation**. If something goes wrong in the database (like a SQLException), @Repository tells Spring to wrap it into a generic DataAccessException. This way, the error handling stays consistent

67 Spring 10: What is application.properties file?

It is the file used to store application configuration in Spring Boot. Instead of hardcoding values, we put them here, so they can be easily changed without touching the code. Example:

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/test


spring.datasource.username=root
spring.datasource.password=1234
```

This makes the application flexible and easier to configure across different environments (dev, test, prod).

Additional Question:

1. How will you configure different Databases in your App for different environments?

We can do using application.properties and Profile. Spring gives us profiles like dev, test, and prod. Each profile can have its own configuration. When the app runs, we simply tell Spring which profile to use, and it picks the right settings.

The most common way is to create different property files for each environment like application- dev.properties, application-prod.properties. When we switch the profile, Spring automatically loads the right file.

Sometimes it is not just the properties that change, the beans themselves need to be different. In that case, we can use @Profile.

```
@Configuration
public class DataSourceConfig {

        @Bean
    @Profile("dev")
        public DataSource devDataSource() {

        }

        @Bean
    @Profile("prod")
        public DataSource prodDataSource() {

        }
`
```

2. What is relaxed binding in spring boot?

Relaxed binding in Spring Boot means we can define configuration properties in different styles (kebab- case, snake_case, camelCase, or UPPER_CASE) and Spring will still map them correctly to the Java fields.

For example, server.port, server_port, or SERVER_PORT all bind to the serverPort property. This makes configuration flexible and developer-friendly, especially when working with environment variables, YAML, or properties files across different operating systems and deployment setups.

```
# application.properties
myapp.server-url=http://localhost:8080
myapp.serverUrl=http://localhost:8080
myapp.server_url=http://localhost:8080
MYAPP_SERVER_URL=http://localhost:8080 # from environment variable
```

All of these map correctly to serverUrl in MyAppProperties.

```
@ConfigurationProperties(prefix = "myapp")
public class MyAppProperties {
        private String serverUrl;
        // getter & setter
}
```

68 Spring 11: What are Spring boot starters?

Starters are preconfigured dependency bundles that make it easy to add features to the application.

For example, if we want to create a web app, just add **spring-boot-starter-web** and it will bring everything we need: Spring MVC, Tomcat, JSON libraries, etc. Without starters, we would have to add each dependency manually.

- Data JPA starter
- Web starter
- Security starter
- Test Starter
- Thymeleaf starter

69 Spring 12: How many ways dependency injection can be done or Types of Dependency Injection?

Spring supports three main ways:

- Constructor Injection
- Setter Injection
- Field Injection

Constructor Injection: Here, we pass dependencies using the class constructor.

```
@Component
class Car {
        private final Engine engine;

        @Autowired
        public Car(Engine engine) {
                this.engine = engine;
        }
}
```

Setter Injection: We provide Dependencies are passed through setter methods. It is good when dependencies are optional

```
@Component

        @Autowired
        public void setEngine(Engine engine) {
                this.engine = engine;
        }
}
```

Field Injection: Dependency is directly injected into the field using @Autowired. It is very easy but not recommended for large projects since it makes testing and refactoring harder.

```
@Component
class Car {
        @Autowired
        private Engine engine;
}
```

Interview tip: Always say constructor injection is best practice because it makes the class immutable and easier to test**.**

Indirect Questions:

1. Why Setter Injection is not recommended?

Setter injection is okay for optional things, but not for important ones. If we forget to call the setter, the object may not work and can throw errors later. If we forget to give @Autowired to setter we will
get Null pointer. We can also alter the objects using setters.

2. When to choose field injection over constructor injection?

Choose field injection only for very simple, non-critical beans and optional. For example, a logger or a utility that's not central to the object's main job. It makes the code shorter but it hides what class
actually needs, so it's not recommended for important dependencies.

3. Why constructor injection is recommended?

Constructor injection is recommended because it forces all required dependencies to be provided when the object is created. The object is complete and stable from the beginning, and its dependencies can't be changed later. This makes the design clear, the code easier to test, and the application more reliable.

4. If a class needs too many dependencies for example 7-10, then how do you design it?

If a class needs 7 different dependencies, you still use **constructor injection**, but instead of making a huge constructor hard to read, you can:

- **Group related dependencies** into a separate class (a configuration or service aggregator).
- **Use interfaces** to reduce the number of direct dependencies.

Reference: Spring Core: Why Constructor Based Dependency Injection is Recommended?

70 Spring 13: What do you mean by Annotation-based container configuration?

Before Spring annotations, people used **XML files** to declare beans. That was lengthy and hard to maintain.

Now Instead of XML files, we can configure Spring beans using annotations. This is called annotation-based configuration.

Some common annotations we use:

- @Component: Marks a class as a Spring bean.
- @Service : Marks a class as service layer bean.
- @Repository : Marks a class as DAO bean.
- @Controller / @RestController : For web controllers.

- @Configuration : Defines configuration class.
- @Bean : Defines a bean inside @Configuration.
- @Autowired : Injects dependencies.
- @Value : Injects values from properties file.

71 Spring 14: Explain about @Autowired?

The @Autowired annotation in Spring is a mechanism for automatic dependency injection. It allows the Spring IoC (Inversion of Control) container to automatically resolve and inject dependencies into the Spring-managed components (beans) without requiring explicit configuration in XML or Java.

```
// Service class
@Service
class GreetingService {
        public String greet() {
                return "Hello, Spring!";
        }
}

// Component that uses the service via constructor injection
@Component
class MyApp {
        private final GreetingService greetingService;

        // Constructor injection
        @Autowired // Optional since Spring 4.3 if there's only one constructor
        public MyApp(GreetingService greetingService) {
                this.greetingService = greetingService;
        }



            public void run() {
        System.out.println(greetingService.greet());
        }
}
```

Point to remember: If there are multiple beans of the same type, Spring throws an error unless we use @Qualifier or @Primary.

Indirect Question:

1. How would you achieve the same thing that the auto-wired annotation does without using the annotation?

Instead of @Autowired, we declare beans in a @Configuration class:

```
@Configuration
    class AppConfig {
    @Bean
        public Service service() {
                return new ServiceImpl();
        }

        @Bean
        public Client client() {
                return new Client(service()); // manual injection
        }
}
```

Here, Client gets Service injected, same as @Autowired.
@Autowired is just a shortcut for dependency injection. Without it, we do the wiring explicitly in XML or @Bean methods.

2. Is Autowired is mandatory when there is only one constructor?

Autowired is `Optional since Spring 4.3 if there's only one constructor`

72 Spring 15: Explain about @Qualifier and @Primary?

In Spring, both @Qualifier and @Primary are used to resolve ambiguity when multiple beans of the same type are available for dependency injection.

@Qualifier

Why we need it: When there are multiple beans of the same type, Spring gets confused. @Qualifier tells Spring to pick the exact one using the name.

How it works: We need to give the bean a name and then point to it when injecting.

```
@Component("creditCard")
class CreditCardPayment implements Payment {}

@Component("debitCard")
class DebitCardPayment implements Payment {}
```

```
@Component
    class PaymentService {
    @Autowired
        @Qualifier("creditCard") // pick credit card specifically
        private Payment payment;
}
```

@Primary

Why we need it: If multiple beans exist but one is used most often, then we can mark it as Primary.

How it works: Mark one bean as @Primary, and Spring will pick it by default when we use @Autowired without qualifiers.

```
@Component
@Primary
class CreditCardPayment implements Payment {} // default

@Component
class DebitCardPayment implements Payment {}

@Component
    class PaymentService {
    @Autowired
        private Payment payment; // gets CreditCardPayment by default
}
```

Indirect Questions:

1. How do you handle multiple beans of the same type in Spring??

Use @Qualifier with @Autowired to specify which bean to inject. This ensures the correct bean is selected when duplicates exist.

2. When you use both @Primary and @Qualifier, which one will take precedence?

When both @Qualifier and @Primary are used in, @Qualifier takes precedence. @Primary indicates a default bean to use when multiple beans exist, while @Qualifier allows for very specific bean selection. When both are present, Spring prioritizes the bean identified by @Qualifier

3. Two beans of same type are found and Spring throw NoUniqueBeanDefinitionException. How would you resolve it?

Use @Qualifier with @Autowired to specify which bean to inject. This ensures the correct bean is selected when duplicates exist.

4. There is only one bean but you annotated with @Qualifier and @Primary, how does it affect? If there is only one bean of that type:

- @Primary has no effect, because there is nothing to choose between.
- @Qualifier also doesn't matter, the single bean will be injected

regardless. They only come into play when multiple beans of the same type exist.

73 Spring 16: Difference between application.properties vs application.yml?

Both are used for external configuration in Spring Boot, but the **syntax style** is different.

application.properties (key-value pairs):

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/test
spring.datasource.username=root
```

application.yml (YAML format):

```
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/test
```

Indirect Question:

1. When do you prefer .properties and

For simplicity and small projects: application.poperties might be preferred due to its directness.

For complex, hierarchical configurations and improved readability: application.yml is generally considered superior due to its structured nature and support for various data types.

Interview Tip: These file helps us to keep configs outside the code, so we can change them per environment without touching Java classes.

74 Spring 17: What is the use of @Value / How to load config values dynamically?

@Value is used to inject values from application.properties or application.yml into our code.

```
app.name=MySpringApp

@Component
    class Config {
    @Value("${app.name}")
    private String appName;

            public void printName() {
        System.out.println(appName); // MySpringApp



        }
}
```

Indirect Question:

1. How will you read values mentioned in properties file? Using @value

2. If you want to inject dynamic port from a different config file, how will you use value?

```
@Value("${server.port}")
private int port;
```

75 Spring 18: What is the use of @ConfigurationProperties / How to load config values dynamically?

@ConfigurationProperties is a Spring annotation used to bind external configuration (like

```
app:
    email:
        host:
    smtp.example.com port:
    587


@Component
@ConfigurationProperties(prefix = "app.email")
    public class
    EmailProperties { private
    String host; private int
    port;
        private String username;

        // getters and setters
```

application.properties or application.yml) to a Java class. Instead of reading each property with @Value, we can map a whole group of related properties into a strongly-typed class.

Spring automatically reads app.email.host, app.email.port, app.email.username from the configuration file.

Values are injected into the EmailProperties bean.

76 Spring 19: How do profiles work in Spring boot, Explain about @Profile?

In the real time projects, we might need different configurations for different environments like Prod, QA and Dev.

This is where profiles come into picture. Spring boot allows us to manage application configurations and

components based on the environment in which the application is running.

This can be done through various methods:

- application-{profile}.properties or application-{profile}.yml: Separate configuration files are created for each profile (e.g., application-dev.properties, application-prod.yml).
- @Profile annotation: This annotation is used on classes (e.g., @Configuration, @Component, @Service, @Repository) or methods to indicate that they should only be loaded or activated when a specific profile is active

Key points:

Profiles are activated at run time using JVM system properties: -

Dspring.profiles.active=prod. Or Using spring.profiles.active property, Set in

application.properties, application.yml **@Profile**

The @Profile annotation used to conditionally register beans or load configuration based on the active profiles.

It is applied to:

@Configuration classes: To load specific configuration classes only when a particular profile is active.

```
@Configuration
@Profile("production")
public class ProductionConfiguration {
        // ... Beans specific to production environment
}
```

@Component, @Service, @Repository: To register specific implementations of interfaces or classes based on the active profile.

```
@Service
@Profile("dev")
public class DevUserServiceImpl implements UserService {
        // ... Development-specific user service implementation
}

@Service
@Profile("!dev") // Active when 'dev' profile is NOT active
public class ProdUserServiceImpl implements UserService {
        // ... Production-specific user service implementation
}
```

@Bean methods: To conditionally create beans within a configuration class.

```
@Configuration

        @Bean
    @Profile("prod")
        public DataSource prodDataSource() {
                // ... Production data source
        }
}
```

77 Spring 20: How spring auto configuration works internally?

Spring Boot Auto Configuration is like having a smart assistant that automatically sets up the Spring application based on what it detects in the project.

When we add a dependency (like Spring Data JPA or Spring MVC), Spring Boot notices this and auto configure the Data source

How it works:

Spring Boot looks at the classpath (all the jars in the project) and the beans we have already

defined. It checks what's missing and tries to auto-configure.

Point to remember: Auto Configuration is triggered by the @EnableAutoConfiguration annotation (or just @SpringBootApplication which includes it).

Example: JPA Auto Configuration

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
</dependency>
```

Let's say when we have dependencies.

Now, we don't need to define a DataSource or EntityManagerFactory. Spring Boot auto-configures, means we don't need to write the below code, Sprint does it for us.

```
@Bean
public DataSource dataSource() {
    return new org.h2.jdbcx.JdbcDataSource();
}
```

Indirect Questions:

1. How Spring Boot knows what to configure?

Spring Uses conditional annotations like:

- @ConditionalOnClass: configures bean only if a certain class is on the classpath.
- @ConditionalOnMissingBean: configures bean only if we haven't already defined one.
- @ConditionalOnProperty: configures bean only if a certain property is set.

2. What happens if I define my own DataSource bean?

Auto Configuration will skip creating a DataSource because of @ConditionalOnMissingBean.

3. How do conditional annotations work in Auto Configuration?

They check the environment, classpath, and existing beans to decide whether to create a bean or skip it

4. Can Auto Configuration be disabled?

Yes, using @SpringBootApplication(exclude = DataSourceAutoConfiguration.class) or via properties

5. How do you override default auto configurations?

- Define our own bean: Spring Boot will skip its default because most auto configs use @ConditionalOnMissingBean.Use properties in application.properties (e.g., spring.datasource.url, spring.jpa.show-sql) to change the defaults.
- Exclude specific auto configs with

@SpringBootApplication(exclude =
DataSourceAutoConfiguration.class).

6. What if two auto-configurations conflict?

If two auto-configurations provide the same bean or configure the same feature differently, we can end up with a conflict. This might cause:

1. **BeanDefinitionOverrideException** : when two beans with the same name get created.
2. **Unexpected behavior** : e.g., two DataSource beans when Spring expected only

one. Solutions:

- Use @ConditionalOnMissingBean, @ConditionalOnClass, etc., to make configs load only when needed.
- Exclude one of the auto-configs via spring.autoconfigure.exclude in application.properties.

7. How to tell auto-configuration not to create bean when already bean exists? Use @ConditionalOnMissingBean

78 Spring 21: Difference between @Bean vs @Configuration **@Configuration**

It is a class-level annotation. When we annotate a class with @Configuration, Spring treats it as a source of bean definitions and manages it as a full Spring configuration.

Point to remember: @Configuration classes are **enhanced with CGLIB proxies**. This means if one method calls another bean method inside the same class, Spring ensures the bean is **singleton** and doesn't create a new instance each time.

```
@Configuration
public class AppConfig {

        @Bean
        public Service service() {
                return new Service(repository());
        }

    @Bean
    public Repository repository() {
        return new Repository();
    }
}
```

Here, service() depends on repository(). Even though we call repository() directly, Spring ensures the same singleton Repository is used.

@Bean:

@Bean is a method-level annotation. It tells Spring: *"The object returned by this method should be registered as a bean in the Spring context."*

We can declare @Bean methods in both @Configuration and @Component classes, but there's an important difference**,** if one @Bean method calls another in a @Component class, we might
accidentally create multiple instances instead of getting the singleton behaviour.

```
@Component
public class AppConfig {

        @Bean
        public Service service() {
                return new Service(repository());
        }

        @Bean
        public Repository repository() {
                return new Repository();
        }
}
```

When we call repository() directly, Spring may create multiple beans.

Indirect Question:

1. Why do you sometimes get multiple instances of a bean if you don't use @Configuration but just @Bean in a class?

Without @Configuration, Spring doesn't proxy the class, so calling one @Bean method from another creates a new instance instead of reusing the singleton.

2. What happens if a @Bean method calls another @Bean method inside the same class?

In a non-@Configuration class, calling a @Bean method directly bypasses Spring, so it creates a fresh object instead of using the Spring-managed bean.

3. Can you use @Bean without @Configuration? What are the implications?

Yes, it works, but we lose singleton guarantees between methods, and Spring simply registers whatever object the method returns.

79 Spring 22: Difference between @Bean vs @Component @Component

It is a class-level annotation. If we put this annotation on top of a class and that class is inside a package that Spring scans, Spring will automatically create an object of it and manage it as a bean.

@Component works when we write our own code. We put the annotation right on the class.

```
@Component
public class EmailService {
            public void sendMail(String to) {
        System.out.println("Mail sent to " + to);
        }
}
```

Now, as long as the @SpringBootApplication (or @ComponentScan) is scanning the package, EmailService is picked up with no extra configuration. It is fast, simple, and we usually use it for our application-level services, repositories, and controllers.

@Bean:

@Bean is a method-level annotation. @Bean is more like the "manual way." We use it inside a class annotated with @Configuration. We explicitly tell Spring *how* to build it

For Example, If we use a class from an outside library because we can't add @Component to it as we don't have control to its code. Instead, we use @Bean in our own configuration class. We write a method that creates the object for that library class and returns it.

```
@Bean
public DataSource dataSource(Environment env) {
        return DataSourceBuilder.create()
                .url(env.getProperty("db.url"))
                .username(env.getProperty("db.user"))
                .build();
}
```

For Example. DataSource is a class from a third-party JDBC library, not our own code, so we can't mark it with @Component. Instead, we create it as a Spring bean by writing a @Bean method that returns a
DataSource object with the right configuration.

80 Spring 23: What is Spring Bean

- A java class which is managed by the IOC container is called as Spring Bean. The life cycle of the spring bean is taken care by the IOC container.
- Spring beans are instantiated, configured, wired, and managed by IoC container.
- Beans are created with the configuration metadata that the users supply to the container (by means of XML or java annotations configurations.)

Additional questions:

1. What is Pojo and DTO?

A POJO (Plain Old Java Object) is a simple Java class that holds data and can also include business logic. A DTO (Data Transfer Object) is a lightweight object used only to carry data between layers or services.

Pojo:

DTO:

How they differ

- **POJO** has everything: id, password, timestamps, business rules.
- **DTO** is trimmed down: only what the client needs (name +

email). So, when we return user data in an API, we send UserDTO, not

the raw User.

2. How is POJO and DTO are different from Spring bean?

A **Spring Bean** is any object managed by the Spring IoC container (with lifecycle, scope, and dependency injection).
POJOs/DTOs are just plain data holders, not managed by Spring unless explicitly registered as beans.

3. When will you get NoSuchBeanException even though there is bean?

- If package scanning is wrong, Spring won't even see our class, so it never creates the bean.

- If we use a wrong qualifier name, Spring looks for a bean with that exact name, can't find it, and throws the exception.
- If the bean is marked with a Profile or Condition, but that profile isn't active or the condition fails, Spring will skip creating the bean.

81 Spring 24: What are bean scopes?

1. Singleton:

Spring creates only one instance of the bean for the entire application context. Spring manages full lifecycle (init and destroy). It is the Default scope.

When to use: For stateless beans or shared services/repositories where all components can use the same instance.

2. **Prototype:**

Spring creates a new instance every time the bean is requested. Spring calls init methods but does not call destroy methods.

When to use: For stateful beans or temporary objects where each client/request needs a fresh copy, like form objects or calculations.

3. **Request:**

A new bean instance is created for each HTTP request. Only relevant in web applications. Each request gets a fresh bean.

When to use: For request-specific beans, like handling data for a single web request or form submission.

4. **Session:**

A new bean instance is created per HTTP session. All requests in the same session share the same bean.

When to use: For user-specific session data, like shopping carts, logged-in user info.

5. **Application:**

A single bean instance per ServletContext. It is shared across the whole web application.

When to use: For application-wide shared resources, like configuration objects, caches, or global services.

Indirect Questions:

1. If multiple components autowire the same service, do they share the same object? Yes, with singleton scope, they all share the same instance

2. If I autowire a prototype bean in a singleton bean, will it be a new object every time?

No, the singleton will get one instance at initialization unless we use @Lookup or ObjectFactory.

3. With @Scope("request"), Will two HTTP requests get the same bean instance?

No, each request gets a new instance.

4. If you autowire Prototype bean in singleton bean how many instances of prototype bean would be created?

- If we autowire a prototype bean into a singleton bean, only one instance of the prototype bean gets created at the time of injection.
- That same instance will be reused by the singleton throughout its lifetime.
- Spring doesn't create a new prototype object every time we call it from the singleton.
- To get a fresh prototype on each use, we need **ObjectFactory** or **Provider**.

Whenever we call usePrototype(), both objectFactory.getObject() and provider.get() will give a new PrototypeBean instance instead of reusing the old one.

5. Can we inject singleton bean into request-scoped bean?

```
@Component
@Scope("prototype")
class PrototypeBean {
        public PrototypeBean() {
                System.out.println("New PrototypeBean created");
        }
}

@Component
    class SingletonBean {
    @Autowired
        private ObjectFactory<PrototypeBean> objectFactory;

        @Autowired
        private Provider<PrototypeBean> provider;

        public void usePrototype() {
                PrototypeBean bean1 = objectFactory.getObject(); // fresh
        instance PrototypeBean bean2 = provider.get();     // fresh
        instance
        }
```

Yes, the request-scoped bean has a shorter lifetime (one HTTP request) than the singleton bean (the entire application). The singleton bean is fully initialized and available before any request comes in. The request-scoped bean can simply have a reference to the singleton bean injected into it, just like any other dependency.

6. Can we inject request-scoped bean into singleton bean?

The singleton bean is created once at application startup. At that time, there is no active HTTP request, so the request-scoped bean does not exist yet. If Spring tried to inject the request-scoped bean directly into the singleton during construction, it would fail because the context for the required scope (HTTP request) isn't active.

The Solution: Scoped Proxy

Spring's solution is to not inject the actual request-scoped bean. Instead, it injects a proxy object that implements the same interface (or extends the class) as the request-scoped bean.

- This proxy is created at startup and injected into the singleton.
- Whenever a method is called on this proxy during an HTTP request, the proxy delegates the call to the real request-scoped bean instance that was created for that specific request.

How to do it: We must specify proxyMode = ScopedProxyMode.TARGET_CLASS (for class-based proxies) or ScopedProxyMode.INTERFACES (for interface-based proxies) in the @Scope annotation of the request-scoped bean.

Refer below question to understand it more.

How proxyMode works in Spring and how does it help? (Refer Question No. 379, Spring 128)

7. How does scope behave in multithread rest controller?

Singleton (default): Only one instance exists for the entire application. Multiple threads (HTTP requests) share the same instance, so instance variables are shared across threads. we must avoid mutable state unless properly synchronized.

Prototype: A new instance is created each time it is requested from the Spring context. In a REST

controller, if we inject a prototype bean, Spring resolves it once at startup for the singleton controller. To get a fresh prototype per request, we need a proxy (@Scope("prototype", proxyMode =
ScopedProxyMode.TARGET_CLASS)).

Request / Session / Web Scopes: These scopes create a new bean per HTTP request or session, so each thread (request) gets its own instance. Using proxies ensures the singleton controller can safely access these shorter-lived beans.

Reference: The Scope of Beans in Spring Boot: A Comprehensive Guide | by Dev Cookies | Medium

82 Spring 25: What is the bean life cycle? Bean Lifecycle in Spring (Step by Step)

Think of a Spring Bean like a person's life: it is born, it learns things, it works, and finally it is retired (destroyed). The IOC container manages all of this**.**

Note: There are many life cycle methods but here I am mentioning only the important. We can read from the link below and check all the methods in the image

1. Instantiation (Bean is created)

The Spring container creates an object of the class. Usually, it calls the no-arg constructor or a factory method.

```
public class MyBean {
    public MyBean() {
                System.out.println("Bean is being instantiated");
        }
}
```

2. Populating Properties (Dependency Injection)

After creating the bean instances, Spring injects all the dependencies (other beans or values) into the bean, which can be done through constructor injection, setter injection, or field injection.

```
@Component
public class MyBean {
        private final MyDependency
    myDependency; @Autowired
        public MyBean(MyDependency myDependency) {
                this.myDependency = myDependency;
        }
}
```

3. **Initialization (@PostConstruct)**

After dependencies are injected, Spring calls methods marked with

@PostConstruct. This is where we put setup logic (like opening a DB connection,

loading configs).

It is the safest place to initialize things because all dependencies are already injected.

```
@Component
public class DatabaseInitializer {

        @PostConstruct
        public void init() {
                System.out.println("DB connection established!");
                // Load initial data, cache, etc.
        }
}
```

4. Destruction (@PreDestroy)

When the Spring container shuts down, methods marked with @PreDestroy

run. Used for cleanup (closing DB connections, releasing resources).

Prevents memory leaks by properly releasing resources.

```
@Component
public class CacheManager {

        @PreDestroy
        public void cleanup() {
                System.out.println("Clearing cache before shutdown!");
                // Close cache, release memory
        }
}
```

Reference: Spring Bean Life cycle. Content | by Himani Prasad | Medium

83 Spring 26: Can we create beans conditionally, if yes how?

Spring has always been famous for Dependency Injection, but sometimes we don't want a bean to always be created. For example, we may need one type of bean in dev environment and another type in prod. That is where @Conditional comes in.

Spring checks the condition **at runtime** before putting the bean in the application context.

Where can we use it?

- **Method level:** On a @Bean method > conditionally register that single bean.
- **Class level:** On a @Configuration class > all beans inside follow the condition.
- **Component level:** On @Component, @Service, @Repository > conditional scanning.

There are 4 types of @Conditional

The **@ConditionalOnProperty** annotation creates the bean only if the specified property is present in the environment and optionally has a specific value. It is commonly used to enable or disable beans based on application configuration properties

```
@Bean
@ConditionalOnProperty(name = "feature.email.enabled", havingValue = "true")
public EmailService emailService() {
    return new EmailService();
}
```

The **@ConditionalOnClass** annotation creates the bean only if the specified class is present on the classpath. It is useful for creating beans that depend on the presence of a particular library or class.

```
@Bean
@ConditionalOnClass(name = "com.mongodb.MongoClient")
public MongoTemplate mongoTemplate() {
    return new MongoTemplate();
}
```

The **@ConditionalOnMissingBean** annotation creates the bean only if the specified bean type or name is not already defined in the application context.

```
@Bean
@ConditionalOnMissingBean
public MyBean defaultBean() {
        return new MyBean();
}
```

The **@ConditionalOnBean** annotation creates the bean only if the specified bean type or name is already defined in the application context. This annotation is useful for defining beans that depend on the presence of other beans.

```
@Bean
@ConditionalOnBean(DataSource.class)
public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
}
```

Real Time Example: Say we want our Spring Boot app to work with Firestore in dev and MongoDB in prod.

```
application.properties: db.type=firestore    # or mongo
```

```
@Configuration
public class DatabaseConfig {

        @Bean
        @ConditionalOnProperty(name = "db.type", havingValue = "firestore")
        public FirestoreService firestoreService() {
                return new FirestoreService();
        }

        @Bean
        @ConditionalOnProperty(name = "db.type", havingValue = "mongo")
        public MongoService mongoService() {
```

```
                return new MongoService();
        }
}
```

This way, we don't have to change the code when moving environments, just changing the property will work.

Indirect Questions:

1.How would you enable or disable a bean without touching the code, just by configuration? Use @ConditionalOnProperty.

2. If I want to create a bean only if MongoDB driver is on the classpath, how do I handle it? Use @ConditionalOnClass.

3. Suppose you already have a custom bean defined. How do you stop Spring Boot from creating its default one?
Use @ConditionalOnMissingBean.

84 Spring 27: How does @Conditional works internally?

Under the hood, Spring's @Conditional annotation works by leveraging the **Condition** interface. This interface must be implemented by a class that contains the conditional logic. When Spring evaluates this logic, it decides whether to register the annotated bean or not.

```
    public class MyCondition implements
    Condition { @Override
            public boolean matches(ConditionContext context, AnnotatedTypeMetadata
        metadata) { String env = context.getEnvironment().getProperty("app.env");
                return "prod".equalsIgnoreCase(env);
        }
}


@Configuration
@Conditional(MyCondition.class)
public class ProdConfig {
        @Bean
        public DataSource prodDataSource() {
                return new HikariDataSource();
        }
}
```

Here, the bean will only load if app.env=prod.

85 Spring 28: Difference between CommandLineRunner and ApplicationRunner?

These two interfaces in Spring Boot are used to run some code right after the application starts up (After the Spring application context is fully initialized. They are super useful for initialization tasks like:

- Loading test data
- Running startup checks
- Starting background processes

CommandLineRunner

- It is an interface in Spring Boot.
- If the class implements it, the run(String... args) method will execute right after the Spring ApplicationContext is loaded and before the application fully starts serving requests.
- The method gives the access to the raw command-line arguments as a String[].

ApplicationRunner

```
@Component
public class MyStartupRunner implements CommandLineRunner {

        @Override
            public void run(String... args) throws Exception {
        System.out.println("App started with args: " + Arrays.toString(args));

                // Example: Load test data if "--load-test-data" is passed
                    if (args.length > 0 && args[0].equals("--load-test-data")) {
            System.out.println("Loading test data...");
                }
        }
}
```

ApplicationRunner is another interface in Spring Boot that serves a similar purpose to CommandLineRunner, but it provides a more structured way to access and parse command-line arguments. The main difference is: instead of giving a raw String[], it provides an ApplicationArguments object.

Indirect Questions:

1. How do you run some logic right after Spring Boot starts? Using CommandLineRunner or ApplicationRunner.

2. If you just need to check if a simple flag like --enable-feature was passed, which runner would you use? CommandLineRunner is enough since we are just checking raw args.

3. If your Spring Boot app needs to read --db-url=localhost:3306 from command line, which runner is better?

ApplicationRunner because it easily handles --key=value arguments.

4. If you had both CommandLineRunner and ApplicationRunner in your project, which one runs first? Both run, order depends on **@Order** annotation or Ordered interface.

```
@Component
@Order(1) // Runs first
public class FirstRunner implements CommandLineRunner { ... }

@Component
@Order(2) // Runs second
public class SecondRunner implements ApplicationRunner { ... }
```

86 Spring 29: How to handle Circular dependencies?

A circular dependency happens when two (or more) beans depend on each other to get

created. For example:

```
@Component
class A {
        private final B b;
        public A(B b) { this.b = b; }
}

@Component
class B {
        private final A a;
        public B(A a) { this.a = a; }
}
```

Here, A needs B and B needs A.

When Spring tries to create them, it gets stuck in a loop and fails with a **BeanCurrentlyInCreationExceptio**

n. There are a few ways to handle

this:

1. Use @Lazy injection

Lazy tells Spring: “don’t create this bean immediately, create it only when needed.”

This breaks the cycle because Spring can first create one bean and keep a proxy for the other.

```
@Component
class A {
        private final B b;
        public A(@Lazy B b) { this.b = b; }
}
```

Now Spring can create A first, and only when we call a.getB(), it will initialize B.

2. Use Setter or Field injection instead of Constructor

Constructor injection creates problems in circular dependency because both beans are required immediately.
But if we use a setter, Spring can first create the object and later “inject” the dependency.

```
@Component
class A {
        private B b;
    @Autowired
        public void setB(B b) {
                this.b = b;
        }
}
```

This way, Spring can create A without needing B instantly.

3. Refactor the design (Best Solution)

Sometimes, circular dependency is a **code smell**. It means the classes are too tightly coupled.

Example: Instead of having A depend on B and B depend on A, write a third-class ‘C’ and move shared logic , and both A & B classes depend on C.

```
@Component
class C {
        // shared logic
}
@Component
class A {
        private final C c;
        public A(C c) { this.c = c; }
}
@Component
class B {
        private final C c;
        public B(C c) { this.c = c; }
}
```

87 Spring 30: What is eager loading and lazy loading of beans??

By default, Spring creates all singleton beans eagerly at the time the ApplicationContext starts.

```
@Component
class EagerBean {
            public EagerBean() {
        System.out.println("Eager bean created!");
        }
}
```

When we run the app, we will see "Eager bean created!" printed immediately, even if we never used it. Lazy loading means: Create the bean only when it is first requested (injected or accessed).

We mark it with @Lazy.

```
@Component
@Lazy
class LazyBean {
        public LazyBean() {
                System.out.println("Lazy bean created!");
        }
}
```

Now, the bean won't be created when the app starts.
It will only be created when some other bean asks for it, like:

```
@Component
    class Service {
    @Autowired
        private LazyBean lazyBean; // LazyBean created only when Service is called
}
```

88 Spring 31: What is Dispatcher Servlet?

The DispatcherServlet a core component of the Spring MVC framework, acting as the front controller for handling all incoming HTTP requests in a web application. Spring Boot automatically configures the DispatcherServlet. It is Role:

Front Controller:

It is the central entry point for all web requests. When a client sends an HTTP request, it first arrives at the DispatcherServlet.

Request Routing:

The DispatcherServlet is responsible for analyzing the incoming request and determining which specific controller and handler method should process it. This is often done based on URL patterns and annotations like @RequestMapping.

Request Delegation:

After identifying the appropriate handler, it delegates the request to that controller method for processing.

View Resolution:

Once the controller processes the request and returns a logical view name (or a direct response), the DispatcherServlet works with ViewResolvers to resolve this logical view name into an actual view (e.g., a JSP page, a Thymeleaf template).

Response Handling: It then renders the selected view and sends the final response back to the client.

Request Handling flow in Spring Boot:

Client Request: Browser sends an HTTP request : DispatcherServlet catches it.

HandlerMapping: DispatcherServlet asks: *Which controller method should handle this URL?* That is decided by HandlerMapping.

```
@GetMapping("/hello")
public String sayHello() {
  return "Hello World!";
   }
```

Here, /hello is mapped to sayHello().

HandlerAdapter: Once the handler (controller method) is found, HandlerAdapter helps in executing it.

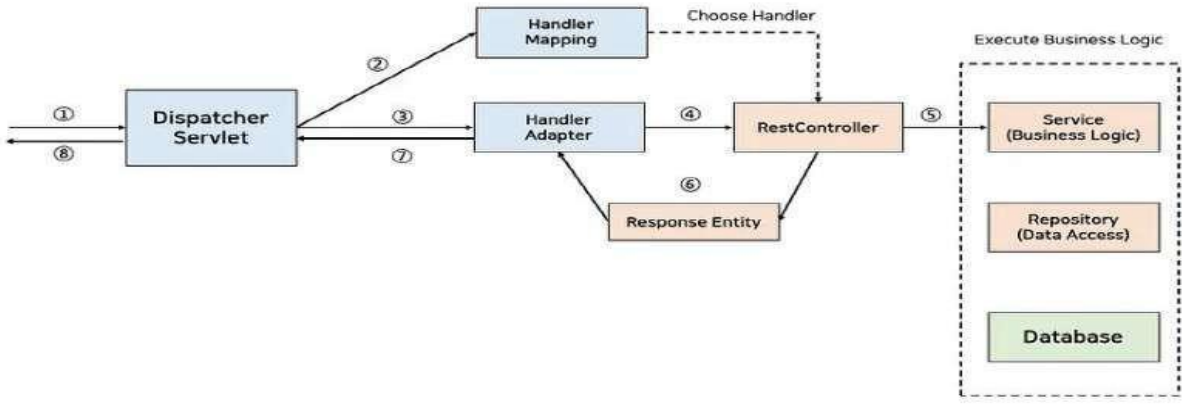
Controller: The controller contains the business logic and usually returns data or view name.

ViewResolver: If the controller returns a view name (like "home"), ViewResolver decides which actual page (like home.html or home.jsp) should be shown.

View: Finally, the view (JSP, Thymeleaf, etc.) is rendered with the model data, and the response goes back to the client.

Full Lifecycle in One Line:

Request > DispatcherServlet > HandlerMapping > HandlerAdapter > Controller > ViewResolver > View > Response



Reference: The DispatcherServlet: The Engine of Request Handling in Spring Boot | by Lakshya Agarwal | Medium

Indirect Question: What happens internally when you hit a REST API in Spring Boot?

89 Spring 32: How embedded server works in Spring boot?

In traditional Java web applications (before Spring Boot), we had to deploy the .war file into an external server like Tomcat, Jetty, or WebSphere. That meant setting up the server separately, copying the WAR into its webapps folder, and then starting it.

Spring Boot makes life easier by giving we an embedded server. This means the server (like Tomcat/Jetty/Undertow) is packaged inside the application as a dependency, so when we run the app,

the server also starts automatically.

In short: the app carries its own server. We don't deploy it anywhere ; we just run the JAR, and it runs like a normal Java program.

Step-by-Step Flow

1. We add dependency in pom.xml or build.gradle Example (pom.xml):

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

By default, this pulls Tomcat as the embedded server.

2. Spring Boot auto-configures the server

When the application starts, Spring Boot sees spring-boot-starter-web and auto-configures an embedded Tomcat (or Jetty/Undertow if we choose those).

3. DispatcherServlet is registered inside that server

Check the DispatcherServlet we discussed above. It is automatically mapped to "/", so it can handle all incoming requests.

4. Server starts with the application If we run: mvn spring-boot:run

We will see logs like: Tomcat started on port(s): 8080

That means the app + server are bundled and started together.

5. Requests are served directly

Now we just hit http://localhost:8080/hello and Tomcat (running inside the JAR) handles the request.

90 Spring 33: What is ORM?

ORM stands for Object Relational

Mapping. Here is the idea:

- In Java, we work with objects (classes, objects, methods).
- In databases, we work with tables, rows, and columns.
- But they don't directly match. Example: a User object in Java is not the same as a user table in

SQL. ORM is the bridge between objects and database tables. It automatically maps:

- A class to a table
- An object to a row
- A field/variable to a column

So instead of writing raw SQL like this:

```
INSERT INTO user (id, name) VALUES (1, CodingLyf);
```

We can just do

```
User u = new User(1, "CodingLyf");
session.save(u);
```

The ORM framework will convert that into SQL internally and talk to the database.

Hibernate is the popular ORM Framework.

91 Spring 34: What is Hibernate and how it works?

Hibernate is the **most popular ORM framework** in Java. It implements ORM and makes database interaction much simpler.

Key features:

1. **Automatic Mapping**: Maps Java classes to DB tables using annotations.
2. **No need for boilerplate SQL**: We use methods like save(), update(), delete().
3. **HQL (Hibernate Query Language)**: We can query using object names instead of table names.
4. **Caching**: Improves performance by storing frequently used data in memory.
5. **Database independence**: Same code works for MySQL, PostgreSQL, Oracle, etc. We need to just change the config.

Exampl

e:

Entity

Class

```
@Entity
@Table(name="users")
public class User {
        @Id
        private int id;

        private String name;

        // getters, setters
}
```

Saving the data:

```
User user = new User();
user.setId(1);
user.setName("CodingLyf");

Session session =
sessionFactory.openSession();
session.beginTransaction();
session.save(user);// Hibernate will generate INSERT query
session.getTransaction().commit();
```

Here we just call save(). Hibernate internally does: INSERT INTO users (id, name) VALUES (1, 'CodingLyf');

92 Spring 35: What is JPA and how does it different from Hibernate?

Originally, Java developers used **JDBC** to talk to databases. But JDBC code was repetitive and messy:

```
Connection con = DriverManager.getConnection(...);
PreparedStatement ps = con.prepareStatement("INSERT INTO user VALUES (?, ?)");
ps.setInt(1, 1);
ps.setString(2, "CodingLyf");
ps.executeUpdate();
```

We had to write SQL everywhere, handle connections, manage transactions. Too much boilerplate.

To fix this, **Hibernate** came along. Hibernate is an **ORM (Object Relational Mapping) framework** that maps Java classes to DB tables. So instead of SQL, we work with objects:

```
User u = new User(1, "CodingLyf");
session.save(u); // Hibernate converts to SQL behind the scenes
```

Problem with Hibernate

Hibernate was powerful, but it was **vendor-specific**. Everyone using Hibernate was tied to Hibernate APIs.

Example:

```
Session session = sessionFactory.openSession();
session.save(user);
```

If tomorrow we wanted to switch to another ORM framework (like EclipseLink or OpenJPA), we have to rewrite a lot of code, because Hibernate APIs are not standard.

To solve this, JPA (Java Persistence API) was introduced by Sun/Oracle as a standard specification.

- JPA defines a set of common annotations (@Entity, @Id, @Table, etc.) and methods (persist(), remove(), find()).
- Different frameworks like Hibernate, EclipseLink, OpenJPA can implement

JPA. So, if the code uses JPA annotations and APIs, you can easily switch

providers.

Spring Data JPA:

Even with JPA, we still need to write repository/DAO classes, manage transactions, etc. That is still boilerplate.

Spring Data JPA makes life easier:

- We just create a repository interface, extend JpaRepository, and Spring generates all the CRUD methods.
- Under the hood, Spring uses JPA (and Hibernate as provider by default). It many manages all the transactions.

Example with Spring Data JPA:

```
public interface UserRepository extends JpaRepository<User, Integer> {
}
```

```
@Autowired
UserRepository repo;

repo.save(new User(1, "CodingLyf")); // INSERT automatically
```

```
repo.findById(1);  // SELECT
repo.findByName("CodingLyf");    // Custom query by
```

In Short:

JPA (Java Persistence API) is a specification (or interface) for ORM (Object-Relational Mapping) in Java. Hibernate is a concrete implementation of JPA.

Indirect Questions?

1. What will happen under the hood when you run repo.save()? (Refer Question No. 307 Spring 55)

2. How to configure Database in Spring

Using the application.properties file: This is the most common way to configure a database connection. We can specify the connection properties in the application.properties file, and Spring Boot will automatically configure the database connection.

Using the @Configuration class: This is a more advanced way to configure a database connection. We can create a @Configuration class and specify the connection properties in the class. Spring Boot will then use the @Configuration class to configure the database connection.

3. How do you make sure existing database schema matches the entity mappings.

- We need to set spring.jpa.hibernate.ddl-auto=validate, which makes Hibernate check if the entities match the database schema at startup.

- Better to use a migration tool like **Flyway** or **Liquibase** to version and align schema changes with entity updates.

4. What are derived query methods?

In Spring Data JPA, a derived query method is a method in a repository interface where Spring automatically generates the query based on the method name. We don't have to write @Query or JPQL. Spring parses the method name and creates the SQL behind the scenes.

```
public interface UserRepository extends JpaRepository<User,
Long> { List<User> findByLastName(String lastName); // Derived
query
```

Here, findByLastName will automatically generate a query like:

```
SELECT * FROM user WHERE last_name = ?;
```

5. Can we use Spring JPA without Spring boot?

Yes. Spring Boot is not required. Spring Boot just makes configuration easier with auto-configuration. We can use plain Spring:
With plain Spring, we need to configure DataSource, EntityManagerFactory, TransactionManager manually in XML or Java config.

6. How does Spring boot simplify the data access layer implementation?

- It auto-configures essential settings like data source and JPA/Hibernate based on the libraries present in the class-path. it reduces the manual set up.
- It also provides and Repository support like JPARepository, enabling easy CRUD operations without the need for boilerplate code.
- Additionally, Spring boot can automatically, initialize database schemas and seeding process using scripts.
- Spring boot can automatically initialize database schemas and seed data using Scripts. It integrates smoothly with various databases and ORM technologies and translates SQL exceptions into Spring's data access exceptions, providing a consistent and simplified error handling.

93 Spring 36: JDBC vs Hibernate vs JPA? JDBC (Java Database Connectivity)

- This is the lowest level API to interact with the database.
- We write SQL queries manually, set parameters, execute them, and handle ResultSet.
- Very flexible, but a lot of boilerplate code like opening connections, closing them, handling exceptions.

```
Connection con = DriverManager.getConnection(url, user, pass);

PreparedStatement ps = con.prepareStatement("SELECT * FROM student WHERE id=?");
ps.setInt(1, 1);
ResultSet rs = ps.executeQuery();
    while (rs.next()) {
    System.out.println(rs.getString("name"));
}
```

Hibernate

- Hibernate is an ORM (Object Relational Mapping) framework built on top of JDBC.
- Instead of writing SQL manually, we work with objects, and Hibernate translates them into SQL behind the scenes.
- It handles connection management, caching, lazy loading, and relationships (@OneToMany, @ManyToOne).
- Provides HQL (Hibernate Query Language) and Criteria API for queries.

JPA (Java Persistence API)

- JPA is not a framework but a specification (a set of rules/interfaces).
- It defines how Java objects should map to database tables, but doesn't provide implementation.
- Frameworks like Hibernate, EclipseLink, OpenJPA implement JPA.

Hibernate vs JDBC

Hibernate converts Checked Exceptions to Runtime exceptions, In JDBC we need to handle SQLException

Hibernate supports caching, where JDBC doesn't.

Hibernate has inbuilt transaction support, where In JBDC we have manage explicitly

94 Spring 37: How do you use @Entity, @Table, @Column in JPA? @Entity

- Marks a Java class as a JPA entity (i.e., a table in the database).
- Without @Entity, JPA won't know that this class should be mapped to a database table.

```
import jakarta.persistence.*;

@Entity      // This class will map to a table
public class User {
        @Id  // Primary
    key private int id;
    private String name;
}
```

Here User class is nothing but User table.

@Table

- In JPA By default, the class name will the table name (as we seen above)
- But if we want to customize the table name, we will have to use @Table.

```
@Entity
@Table(name = "users")    // Map to 'users' table instead of 'User'
    public class User
    { @Id
        private int id;
        private String name;
}
```

Now this class maps to the users table in the DB.

@Column

- By default, column name is nothing but the field name in the class.
- If we want to customize column name, length, nullability, uniqueness, etc., use @Column.

```
@Entity
@Table(name = "users")
    public class User
    { @Id
        @Column(name = "user_id")      // custom column name
        private int id;

        @Column(name = "full_name", length = 50, nullable = false, unique = true)
    private String name;
}
```

This means:

- id maps to column user_id
- name maps to column full_name, max length = 50, cannot be null, must be unique

Indirect Questions:

1.How do you mark a column as not nullable or unique?

To make sure JPA saves only unique values, we define a unique constraint on the entity field. This can be done with @Column(unique = true).

Use @Column(nullable = false, unique = true)

2. How does JPA decide table/column names if you don't give any?

JPA uses the class name as table name and field names as column names.

3. What happens if you don't give @entity to class?

If we don't give @Entity on a class, JPA will not treat it as a database entity, it will just behave

like a normal Java class means no table will be created.

4. Can we make @Entity as final class?

No, we should not make an @Entity class final

- JPA providers (like Hibernate) often use proxy classes at runtime to enable features like lazy loading.
    - To create these proxies, they extend the entity class.
- If the entity is final, it cannot be subclassed, so Hibernate can't create the proxy which breaks lazy loading and other internal mechanisms.

5. Can we make @Entity as Abstract class?

Yes, it can be abstract, and other entities can extend it

95 Spring 38: What are best practices or rules while creating the Entity? Use a Primary Key (@Id)

Every entity must have an identifier. Always define a primary key with @Id.

Avoid Final Classes

Entities should not be final because JPA providers like Hibernate use proxies (subclassing) for lazy loading.

Provide a No-Arg Constructor

Entities must have a default (public or protected) no-arg constructor for JPA to instantiate them.

Make Fields Private with Getters/Setters

Keep fields private and expose them via getters/setters to maintain encapsulation.

Don't Use Static or Transient Fields for Persistent State

JPA ignores them, so use @Transient explicitly for non-persistent fields.

Prefer Wrapper Types for Nullable Columns

Example: use Integer instead of int if the column can be null.

Use Meaningful Table and Column Names

Override defaults using @Table(name="...") and @Column(name="...") for clarity.

Override Equals and HashCode carefully

Override equals and hashCode based on primary keys. Avoid including lazy-loaded relationships to prevent performance issues.

Reference: Entity Class best practices and rules | by Sumit sharma | Medium

96 Spring 39: What are the different types of relationships in JPA and explain @OneToOne with example

Database relationships are fundamental concepts in relational database design, allowing data to be stored efficiently and retrieved accurately.

Different types are: OneToOne, OnetoMany, ManytoOne, and ManytoMany.

Reference: Types of Relationship in DBMS: Explained with Examples

@OneToOne Relationship

This is when one entity is directly related to exactly one other entity. Like a Person and their Passport means each person has one passport, and each passport belongs to one person.

```
@Entity
public class Person {
```

```
        @Id
        private Long id;

        @OneToOne
        @JoinColumn(name = "passport_id")
        private Passport passport;
        // ... other fields
}

@Entity
    public class
    Passport { @Id
        private Long id;

        @OneToOne(mappedBy = "passport")
        private Person person;
        // ... other fields
}
```

The **@JoinColumn** annotation is placed on the owning side of the relationship. It explicitly defines the foreign key column in the database table that links to the primary key of the associated entity.

mappedBy declares that the current entity is not responsible for managing the foreign key and that another entity (the one specified in mappedBy) handles the relationship's persistence.

From the Example:

```
@OneToOne
@JoinColumn(name = "passport_id")
private Passport passport;
```

This means:

- In the person table, there will be a column passport_id.
- That column acts as a foreign key pointing to the passport table's id.
- So, the relationship is stored in the person

table. In Passsport class we wrote

```
@OneToOne(mappedBy = "passport")
private Person person;
```

mappedBy = "passport" tells Hibernate: "Don't create a new join column here. Just look at the field passport in the Person entity, because that is where the relationship is actually managed."

So passport table does not get an extra column like

person_id. This is how table looks like.

Person Table

id	name	passport_id
1	John	101
2	Alex	102

Passport Table

id	passport_number	issue_date
101	A12345	2020-01-01
102	B67890	2021-02-01

Here:

- passport_id in Person points to id in Passport.
- Only Person has the foreign key column, because it is the owning

side. Jave code:

```
Passport passport = new Passport();
passport.setId(101L);
passport.setPassportNumber("A12345");

Person person = new Person();
person.setId(1L);
person.setName("John");
person.setPassport(passport);

// save person : will also link passport id
```

Interview Tip: Always explain these relations with examples and dummy tables. So that it will be easy for the explanation.

97 Spring 40: Explain @OneToMany or @ManytoOne with example

@OneToMany Relationship

This represents a one-to-many connection. For example, a department can have many Employees,but each Employee belongs to only one Department.

```
@Entity
    public class
    Employee { @Id
        private Long id;

        @ManyToOne
        @JoinColumn(name = "department_id")
        private Department department;
```

```
        // ... other fields
}

@Entity
    public class
    Department { @Id
        private Long id;

        @OneToMany(mappedBy = "department")
        private List<Employee> employees = new ArrayList<>();
        // ... other fields
}
```

Employee owns the relationship because it has @JoinColumn.

Department is the inverse side because it uses mappedBy.

In the employee table, a column department_id will be created, that column is a foreign key referencing department.id.

The department table will **not** get any employee_id column or extra join table.

Department Table

id	name
10	HR
20	IT

Employee Table

id	name	department_id
101	Alice	10
102	Bob	10
103	Charlie	20

Here:

- Employees **Alice** and **Bob** belong to Department **HR (id=10)**.
- Employee **Charlie** belongs to Department **IT (id=20)**.
- The link is stored in employee.department_id.

Reference: ONE TO MANY mapping in Spring JPA | by Mohammed Youssef | Medium

@ManyToOne Relationship

This is the inverse of OneToMany. In the example above, the Employee side uses @ManyToOne because many employees can belong to one department.

Reference: Many To One mapping in Spring Boot JPA (unidirectional) | by Vinotech | Medium

98 Spring 41: Explain @ManytoMany with example

This is when multiple entities relate to multiple other entities. Like Students and Courses , means a student can take many courses, and a course can have many students.

Student is the owning side because it has @JoinTable.

Course is the inverse side because it uses mappedBy.

The **@JoinTable** annotation in Hibernate (part of JPA) is used to define the join table used in the mapping of associations, particularly for many-to-many relationships

```
ManyToMany
@JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
private Set<Course> courses;
```

This code means,

A new join table called student_course will be

created. It has two foreign key columns:

- student_id > references student.id
- course_id > references course.id

mappedBy = "courses" tells Hibernate: *"Don't create another join table here. Just use the join table defined in Student."*

Without mappedBy, Hibernate would create **two join tables**, which is wasteful.

Student Table

id	name
1	John
2	Alice

Course Table

id	title
100	Math
200	Physics

Student_Course Join Table

student_id	course_id
1	100
1	200
2	100

Reference:

Theory: Spring Boot Many To Many Relationship | by HK | Medium

Example: Java Spring Boot — Many to Many Relationship | by Zübeyr Bahadır Damar | Medium

Scenario Question:

1. Supposed One employee has multiple managers, and one manager can have multiple employees. How many tables do you use to save data?

We need **3 tables**:

1. **Employees**: stores employee details (employee_id, name, etc.)
2. **Managers**: stores manager details (manager_id, name, etc.)
3. **Employee_Manager**: a junction (bridge) table that connects employees and managers.

- It has at least two columns: employee_id and manager_id, both as foreign keys.

99 Spring 42: What is cascading in JPA and when would you use it?

In JPA, **cascading** means that an operation done on a parent entity automatically flows (or "cascades") to its child entities.

Think of it like this: if we delete a department, and we want all Employees under that Department to also get deleted automatically, we set cascade rules. Without it, we need to delete each Employee manually.

It saves us from writing extra code and keeps parent-child relationships

consistent. Different types of cascade types: ALL, PERSIST, MERGE, REMOVE,

REFRESH, DETACH Explanation with Relationships:

CascadeType.PERSIST:

Use Case: When a new parent entity is persisted, its newly created child entities should also be persisted.

Relationship Example (One-to-Many): A Customer entity has a list of Order entities. When a new Customer is saved, all new Order entities associated with that customer should also be saved.

PERSIST: Save parent also saves child.

REMOVE: Deleting parent deletes

child. MERGE: Updating parent

updates child. REFRESH: Refresh

parent refreshes child. ALL: Applies all

above.

Reference: JPA/Hibernate Cascade Types. Cascading relationships are designed to… | by Himani Prasad | Medium

Indirect Questions:

1. If I delete a department, will all its Employees also get deleted automatically? By default, deleting a parent does **not** delete children.

   If we want that, we need to explicitly add cascade = CascadeType.REMOVE or CascadeType.ALL on the relationship.

2. Why does saving a parent entity not automatically save the child entities? If cascade is missing, we must save the children separately.

   With cascading, saving the parent pushes the persist operation down to its children.

3. Can you give me an example where you should NOT use CascadeType.ALL?

   For example, deleting a customer should not delete all their Orders, since Orders might be needed for reporting or audit.

   So, we will avoid CascadeType.REMOVE in that relationship.

4. What happens if you delete parent entity when cascading type is all? Child entities will also be deleted.

100 Spring 43: How does eager loading and lazy loading works in JPA?

Whenever we fetch an entity from the database, it may have relationships (like Department has Employees).

The question is: Should JPA fetch everything immediately (Eager) or Will it fetch only when we actually need it (Lazy)?

Eager Loading: Related entities are loaded immediately along with the parent, even if we don't use them.

```
@Entity
    public class
    Department { @Id
        @GeneratedValue
        private Long id;

        private String name;

        @OneToMany(mappedBy = "department", fetch = FetchType.EAGER)
        private List<Employee> employees;
}
```

If we run: `Department dept = entityManager.find(Department.class, 1L);`

JPA will fetch the Department and also all its Employees in the same query or with extra queries. So even if we don't need Employees, they are still loaded.

Lazy Loading: Related entities are loaded only when accessed. JPA uses a proxy (like a placeholder object).

```
@Entity
    public class
    Department { @Id
        @GeneratedValue
        private Long id;

        private String name;

        @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
        private List<Employee> employees;
}
```

If we run `Department dept = entityManager.find(Department.class, 1L);` only the Department is fetched. Employees are not.

But when we do `dept.getEmployees();` now JPA fires another query to load Employees.

Point to remember: If we try to access Employees after the Hibernate session is closed (like outside a transaction), we will get LazyInitializationException.

Indirect questions

1. Why am I getting LazyInitializationException when accessing a collection after transaction is closed? Because it was marked LAZY, and the data wasn't loaded before the session closed.
2. Which is the default fetch type for @OneToMany and @ManyToOne?
   - @OneToMany : LAZY (default)
   - @ManyToOne : EAGER (default)
3. How would you optimize performance if you always need parent + child data together? Use fetch = FetchType.EAGER or better, use **JPQL JOIN FETCH** to control it at query level.

Interview Tip: We often prefer Lazy and control loading explicitly with JOIN FETCH in queries.

4. How does lazy loading breaks in rest Api, how do you solve it properly?

In JPA/Hibernate, associations are often marked as LAZY to avoid unnecessary data fetching. For example:

```
@Entity
class User {
        @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
        private List<Order> orders;
}
```

How it breaks:

- In a Spring REST controller, we often return entities directly as JSON.
- When Jackson tries to serialize User, it accesses orders.
- If the Hibernate session is already closed (because the transaction ended after service layer), we get: org.hibernate.LazyInitializationException

Fix:

Use DTOs to Map the correct data before

returning. Use join fetch

101 Spring 44: Explain Repositories, like CrudRepository, JpaRepository, and PagingAndSortingRepository?

CrudRepository, JpaRepository, and PagingAndSortingRepository are interfaces in Spring Data JPA that offer different levels of functionality for data access. They form an inheritance hierarchy, with each extending the previous one to add more features:

CrudRepository:

- This is the most basic interface.
- It provides fundamental Create, Read, Update, and Delete (CRUD) operations. It includes methods like save(), findById(), findAll(), and delete().
- If only basic data manipulation is required, CrudRepository is sufficient.

PagingAndSortingRepository:

- This interface extends CrudRepository and adds capabilities for pagination and sorting.
- It introduces methods like findAll(Pageable pageable) for retrieving data in pages and findAll(Sort sort) for sorting results.
- Use this interface when we need to display large datasets in a paginated or sorted manner.

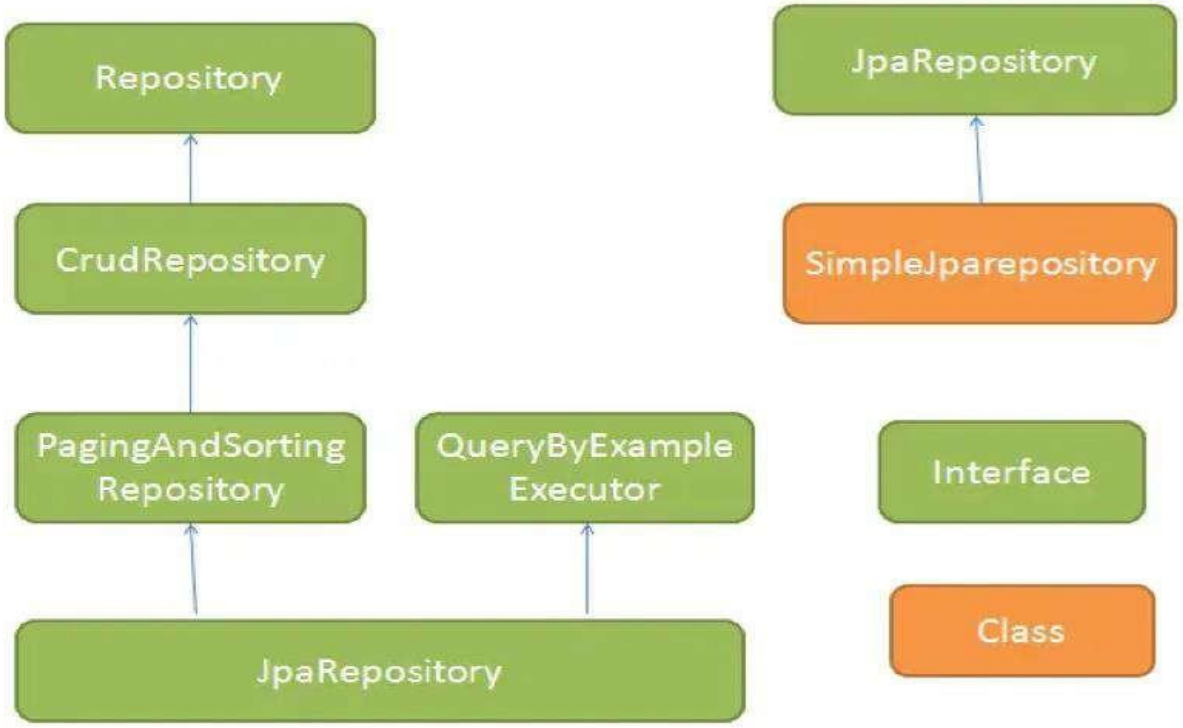
JpaRepository:

This is the most comprehensive interface, extending PagingAndSortingRepository and thus inheriting all its functionalities, as well as those of CrudRepository.

JpaRepository adds JPA-specific features such as:

- **Batch operations:** Methods for deleting entities in bulk.
- **Flushing the persistence context:** Methods like flush() to synchronize the in-memory changes with the database.
- **Support for custom queries:** Facilitates the definition of custom queries using annotations or method names.

Repository hierarchy:



Indirect Questions:

If you only need simple save and find operations, which repository will you choose? Use CrudRepository.

1. How would you fetch 20 students at a time from the database? Use PagingAndSortingRepository with PageRequest.
2. Which repository allows batch deletion or flushing? Use JpaRepository.
3. If you extend JpaRepository, do you still get CrudRepository methods? Yes, because it is a hierarchy.

102 Spring 45: What is JPQL and how does it different from HQL?

JPQL(Java Persistence Query Language) queries are written using the entity classes and their attributes instead of raw SQL queries.

```
//Entity: Employee (mapped to employees table)
String jpql = "SELECT e FROM Employee e WHERE e.department.name = :deptName";
List<Employee> employees = entityManager.createQuery(jpql, Employee.class)
                                                                    .setParameter(

                                                .getResultList();
```

Here Employee is the entity class, and department.name is a field in the object, not the DB column.

HQL (Hibernate Query Language) is a powerful object-oriented query language used in Hibernate ORM. HQL is a superset of the JPQL. A JPQL query is a valid HQL query, but not all HQL queries are valid JPQL queries.

1.If I switch from Hibernate to EclipseLink, will my queries still work?

Only JPQL queries are guaranteed to work. HQL has Hibernate-specific stuff.

2. What's the difference between using EntityManager.createQuery() and Session.createQuery()?

The first is JPQL (JPA standard), the second is HQL (Hibernate-specific).

3. Can JPQL query database tables directly?

No, it queries entities and their mappings, not raw tables. For raw SQL, we will use @NamedNativeQuery or createNativeQuery().

103 Spring 46: What is @Query (Custom Queries) and why should we use them??

By default, Spring Data JPA creates queries automatically just by looking at the method names.

Example: `List<Employee> findBySalaryGreaterThan(double salary);`

Spring generates the query behind the scenes.

But sometimes we need more control, maybe a complex join, or a query that can't be expressed with method naming. That is when we use **@Query**.

@Query allow us write the own queries directly inside the repository method.

Two ways to use it

1. JPQL with @Query (works on entities and fields)

```
    public interface EmployeeRepository extends JpaRepository<Employee,
    Long> { @Query("SELECT e FROM Employee e WHERE e.salary > :salary")
        List<Employee> findEmployeesWithHighSalary(@Param("salary") double salary);
}
```

Here, notice we wrote the query using entity class (Employee) and its field (salary), not database table/column.

2. Native SQL with @Query

Native queries are queries written in **plain**

SQL Why should we use them?

Complex queries

Sometimes JPQL can't express what we want, like database-specific functions, window functions, or advanced joins.

Example:

```
SELECT e.*, ROW_NUMBER() OVER (ORDER BY e.salary DESC) as rank
FROM employees e
```

This is not possible with JPQL, but we can do it with a native query.

Performance tuning

Native queries useful when we need to write highly optimized SQL, sometimes better than what JPA would generate for us.

Limitations of native queries:

- Native queries are tied to the database (Oracle, MySQL, PostgreSQL, etc.). But JPQL is database- independent.
- SQL is harder to maintain if the DB schema changes.

Indirect Questions:

1. How Native queries create problems in Spring?

Native queries are specific to a database; switching DBs may break the query.

Changes in table structure require updating raw SQL manually

104 Spring 47: What is @NamedQuery and @NamedNativeQuery?

Imagine we want to always get a list of users who are “admins” from a database.
Without @NamedQuery, we need to write the same query in several places in the code, which is repetitive.

@NamedQuery is an annotation in JPA (Java Persistence API) that allows to define a reusable query. This query is named (we give it a name) and defined once in the entity class. Later, we can reuse that query by referencing the name in the code.

- Named Queries are static, meaning once they are defined, they cannot be modified at runtime.
- The @NamedQuery annotation is placed at the top of the relevant Entity Class and consists of two parameters:
- name: A unique identifier used to reference the query.
- query: The JPQL statement (JPQL query) that will be executed at

runtime. Example:

```
@Entity
@NamedQuery(
        name = "User.findByRole", // Name of the query
        query = "SELECT u FROM User u WHERE u.role = :role" // The actual JPQL query
)
    public class User
    { @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;
    private String name;
    private String role;

        // Getters and setters...
}
```

- name = "User.findByRole" is the **name** of the query. We will reference this name later to execute the query.
- query = "SELECT u FROM User u WHERE u.role = :role" is the actual query. It is written in JPQL (Java Persistence Query Language), which is similar to SQL

Reference: Spring Data JPA @NamedQuery and @NamedQueries Example

We can also use @NamedNativeQueries Read here : Spring Data JPA @NamedNativeQuery and @NamedNativeQueries Example

105 Spring 48: What is the use of @Transactional?

In Spring (and JPA), @Transactional is used to manage database transactions automatically.

A transaction means a group of operations that should either:

- all succeed together (commit), or
- all fail together (rollback).

Without transactions, if something fails halfway, we might end up with corrupted data in the database. Think of a banking example:

- We transfer money from Account A to Account B.

- Step 1: Deduct from Account A.
- Step 2: Add to Account B.

If step 1 succeeds but step 2 fails, out money is gone!
That is why both steps should be inside one transaction; either both happen, or both fail.

The **@Transactional** annotation in Spring is used to manage database transactions in a declarative manner. It simplifies transaction management by automatically handling the beginning, committing, or rolling back of transactions

```
@Service
public class BankService {

		@Autowired
		private AccountRepository accountRepo;

		@Transactional
			public void transferMoney(Long fromId, Long toId, double
		amount) { Account from = accountRepo.findById(fromId).get();
				Account to = accountRepo.findById(toId).get();

				from.setBalance(from.getBalance() -
		amount); to.setBalance(to.getBalance() + amount);



				accountRepo.save(fro
		m); accountRepo.save(to);

				// if something fails here, both updates rollback
		automatically
		}
```

With @Transactional, if anything inside fails, **both account updates are rolled back**. Points to remember:

1. Use @Transactional on service layer methods, where business logic happens (not usually on repository methods).

2. In Spring, transactions only work when a method is called from outside the class itself, because Spring uses proxies to manage transactions.

3. By default, rollback only happens for **runtime exceptions**. If we want rollback for checked exceptions too, we must configure it:
@Transactional(rollbackFor = Exception.class)

Interview Tip:
- Mention real world scenario while explaining about the transitional.
- Use @Transactional to ensure a group of DB operations succeed or fail together, maintaining consistency.
- Mention Spring automatically handles commit and rollback, so you don't manage transactions manually.
- It Can be applied at method or class level depending on the scope of the transaction.
- Real-world example: In an e-commerce service, deducting stock and creating an order are in the same transactional method; if order creation fails, stock deduction is rolled back automatically.

Indirect Question:

1. What happens if a Spring bean is @Transactional, but the method is called from inside the same class?

Answer: Self-invocation bypasses the proxy, so the transaction is ignored

Reference: Discover how to use the @Transactional annotation in Spring Boot | by Tharindu Dulshan | Medium

2. What is the best place to keep @Transactional Annotation and why?

In a Spring application, @Transactional is used to make sure that a group of database operations happen as one complete unit of work. If something goes wrong, all the changes are rolled back, so the data stays consistent.

Now the question is: where should you place it?

- **Controller layer**
  Controllers should only handle requests and responses. They aren't supposed to know how transactions work, so putting @Transactional here mixes responsibilities.
- **Repository (DAO) layer**
  Repositories are focused on single database operations like saving, finding, or deleting one entity. If we mark them transactional, we only control one small piece of the puzzle, but not the bigger business process.
- **Service layer (Best place)**
  Services contain the actual business logic. They often call multiple repositories, maybe even across different entities. By putting @Transactional at the service method level, we ensure that all those operations run inside a single transaction. If anything fails, the whole set of operations rolls back.

Example

Imagine an **e-commerce app**:

1. When a customer places an order, the system saves the order details.
2. Then it deducts money from the customer's balance.
3. Finally, it updates the product stock.

All three must succeed together. If the payment fails but the stock is already reduced, you end up with inconsistent data.
That is why the service method should be transactional:

```
@Service
public class OrderService {

        @Transactional
            public void placeOrder(Order order) {
        orderRepository.save(order);
        paymentRepository.debit(order.getPayment());
        productRepository.updateStock(order.getProduct());
        }
}
```

Here, if payment fails, the saved order and stock update are rolled back automatically.

3. What type of proxy @Transactional creates?

When we put @Transactional on a method or class, Spring needs a way to wrap that code so it can start, commit, or roll back a transaction. It does this with proxies. Now, which proxy type gets used depends on the class.

1. JDK Dynamic Proxy

If the class implements an interface, Spring will by default create a JDK dynamic proxy. This proxy implements the same interface and intercepts calls to add transactional behavior.

Here, UserServiceImpl implements UserService. Spring creates a JDK proxy that also implements UserService. All calls go through that proxy.

```
public interface UserService {
        void saveUser(User user);
}

@Service
@Transactional


public class UserServiceImpl implements UserService {
        public void saveUser(User user) { ... }
}
```

2. CGLIB Proxy

If the class does not implement any interface, Spring falls back to CGLIB, which creates a subclass at runtime. That subclass overrides the methods to add transaction logic.

```
@Service
@Transactional
public class OrderService {
        public void placeOrder(Order order) { ... }
}
```

Since OrderService has no interface, Spring uses CGLIB to generate something like OrderService$$EnhancerBySpringCGLIB, which wraps the real logic.

106 Spring 49: What is propagation in Transaction?

When we use @Transactional in Spring, we are telling Spring how the transaction should behave. But here is the thing, sometimes one method with @Transactional calls another method that also has @Transactional.

Now the big question is:
Should both methods share the same transaction, or should the second method start a new transaction?

That is what **Propagation** decides. It is basically the rule for how transactions should behave when one transactional method calls another.

Let's say we have two services:

```
@Service
public class OrderService {

        @Transactional
        public void placeOrder() {
                // some DB operations for order
        paymentService.processPayment();
        }
}


@Service
public class PaymentService {

        @Transactional
        public void processPayment() {
                // DB operations for payment
        }
```

}

Now, the question is:

- When placeOrder() calls processPayment(), do both run in one single transaction?
- Or should processPayment() start its own separate

transaction? This is controlled by Propagation.

Propagation Types:

REQUIRED (default)

- If there's already a transaction, join it.
- If not, create a new one.
- Example: Both order and payment will run in **one transaction**. If payment fails, the whole order fails.

  @Transactional(propagation = Propagation.REQUIRED)

REQUIRES_NEW

- Always start a new transaction, even if one already exists.
- Example: Payment starts in a **new transaction**. So even if order fails later, payment can still be committed.

  @Transactional(propagation = Propagation.REQUIRES_NEW)

NESTED

- Runs inside the parent transaction but can roll back independently.
- Example: Payment failure will roll back payment only, but not necessarily the whole order. @Transactional(propagation = Propagation.NESTED)

SUPPORTS

- If a transaction exists, join it. If not, just run without a transaction.

NOT_SUPPORTED

- Runs the method non-transactionally and suspends any existing transaction.

MANDATORY

- Must run inside an existing transaction. If none exists throw exception.

NEVER

- Must run without a transaction. If one exists throw exception.

Let's take one simple example and apply all the 4 cases.
Imagine a **BankService** class with two methods: transferMoney() and sendEmailNotification().

```
@Service
public class BankService {

	@Transactional
	public void transferMoney() {
		// deduct from account A
		// add to account B
	sendEmailNotification();
	}

	@Transactional
	public void sendEmailNotification() {
		// send email to user
	}
}
```

Q1. If two methods are annotated with @Transactional

- When transferMoney() calls sendEmailNotification(), Spring reuses the same transaction (by default Propagation.REQUIRED).
- Both DB updates and email logs (if stored in DB) are part of the same transaction.
- If email fails then the whole transaction rolls back.

2. If transferMoney ()method is not annotated with @Transactional

- transferMoney() runs without any transaction.
- If sendEmailNotification() is annotated with @Transactional, Spring starts a new transaction only for that method.

3. If sendEmailNotification() method is NOT supported

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void sendEmailNotification() {
	// send email without any transaction
}
```

If this method is called from a transactional method, Spring suspends the existing transaction and runs this code without transaction.

No error. It simply says "I don't want a transaction here."

4. If transferMoney() method is in transaction but you don't want sendEmailNotification() method in transaction

If transferMoney() is transactional but we want sendEmailNotification() to run outside the transaction, then:

- With **NOT_SUPPORTED**, Spring will suspend A's transaction, run B without any transaction, then resume A's transaction. Everything works fine.
- With **NEVER**, Spring will throw an exception because A already has an active transaction. B refuses to run at all.

So, in this case, NOT_SUPPORTED is the right choice, because we are saying: *"I want this method to run, but I don't want it wrapped in a transaction."*

NEVER is useful only when we want to enforce a hard rule means "this method must never run if there's an active transaction, otherwise fail."

107 Spring 50: How transaction works under the hood?

When Spring sees a method annotated with @Transactional, it doesn't directly execute the method. Instead, it creates a **proxy** (a wrapper around the class).

This proxy is responsible for:

- Opening a transaction (start)
- Calling the actual method
- Deciding to **commit** or **rollback** the transaction based on the result

```
@Service
    public class PaymentService {
    @Transactional
        public void makePayment() {
                // 1. deduct money
                // 2. update account balance
                // 3. save transaction log
        }
}
```

What happens when makePayment() is called?

1. **Proxy Intercepts Call**

   When we call paymentService.makePayment().

   The proxy catches this call before our method

   runs.

2. **TransactionManager Starts Transaction**

   Proxy asks PlatformTransactionManager (usually DataSourceTransactionManager) to start a transaction.

   It sets autoCommit=false on the JDBC connection.

3. **Our Method Executes**

   Now the actual method runs (deduct money, update balance, etc.).

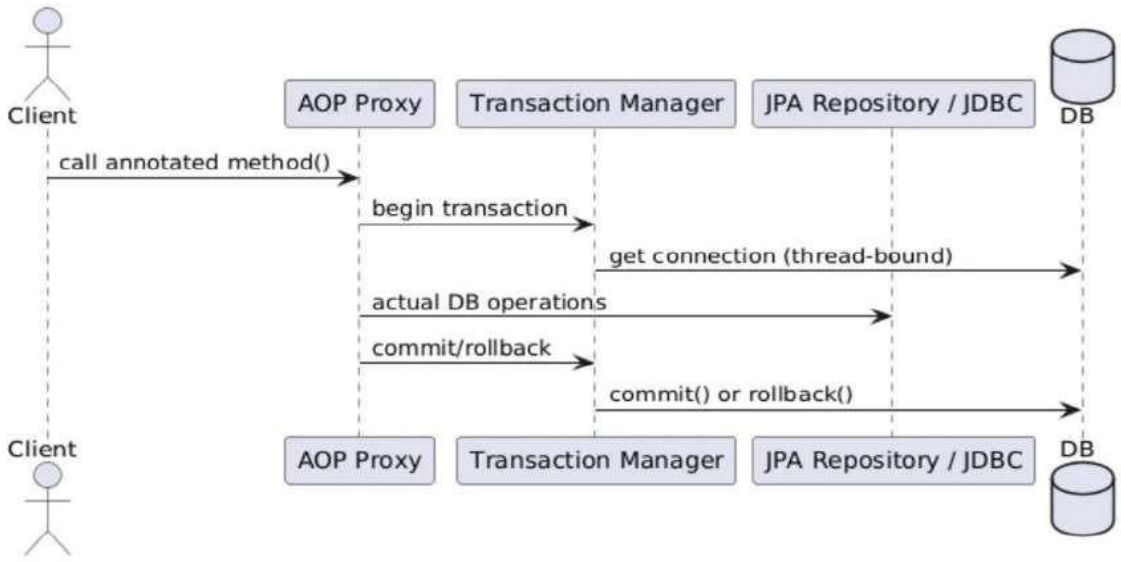
   All DB operations use the same connection and are part of one transaction.

4. **Commit or Rollback**

   If our method finishes without exception > Proxy tells TransactionManager to

   commit. If out method throws a RuntimeException > Proxy tells

   TransactionManager to rollback.



Reference: Spring Transaction Internals — What Really Happens Under the Hood | by Arvind Kumar |Medium

303 Spring 51: What is N+1 problem how do you fix it?

Imagine we want to fetch all students and their courses from the

- First, we run 1 query to get all students.
- Then, for each student (say there are N students), we run 1 query per student to fetch their courses.

That means: 1 (main query) + N (extra queries) = N+1 queries.

So instead of just 2 queries (students + their courses), we end up making dozens or hundreds of queries, which slows everything down badly.

```
List<Student> students = entityManager
.createQuery("SELECT s FROM Student s", Student.class)
.getResultList();

for (Student s : students) {
// This triggers another query for each student (Lazy loading)
System.out.println(s.getCourses().size());
}
```

Example of N+1 problem in JPA/Hibernate
If there are 100 students, this will make:

- 1 query to fetch students
- 10000 extra queries for their

courses Total = 10001 queries (instead

of just 2). **How to fix N+1 problem?**

Use Join Fetch: It tells Hibernate to fetch related data in a single query. Instead of fetching students only, fetch students and their courses in a single query.

```
public interface StudentRepository extends JpaRepository<Student, Long> {

        @Query("SELECT s FROM Student s JOIN FETCH s.courses")
    List<Student> findAllStudentsWithCourses();
}
```

Use Entity Graphs (alternative way)

If we don't want to hardcode JOIN FETCH in queries, we can use entity graphs.

Reference: JPA EntityGraphs: A Solution to N+1 Query Problem | by Thameena S | Geek Culture | Medium

304 Spring 52: Difference Between JOIN and JOIN FETCH? JOIN:

A SQL join combines data from two or more tables using a common column. In HQL or JPQL, a JOIN adds the joined table's data to the SQL query. But this doesn't mean the related objects in the code are loaded right away. If they are set to load lazily, they will only load when we actually use them, which can cause extra queries (the N+1 problem) if we access them many times.

JOIN FETCH:

JOIN FETCH in JPA/Hibernate is like a normal join, but it also forces Hibernate to load the related entities immediately in the same query, avoiding lazy loading and the N+1 problem.

Use JOIN when we only need the association for filtering, but don't actually need the associated entities in memory. Example: get students who have at least one course.

Use JOIN FETCH when we know we will access the associated entities and want to avoid the **N+1 problem** by loading everything in one query. Example: get students and load their courses right away.

Reference: Spring JPA: when to use "Join Fetch" | by Dev INTJ Code | Javarevisited | Medium

305 Spring 53: How does locking works in Spring JPA?

When multiple users/transactions try to read and update the same data at the same time, there is a risk of conflicts. To handle JPA provides the locking mechanisms. Transaction locks are crucial for managing concurrent access to the data in the database. It ensures data integrity and consistency when multiple transactions interact with the same data.

In Spring Data JPA, two primary types of locks are used: Pessimistic Locking and Optimistic Locking

Optimistic Locking assumes that conflicts are rare and allows the concurrent transactions to proceed without locking the data. It uses the @version field to detect the conflicts and ensure that updates do not overwrite the each other.

```
@Entity
    public class Product
    { @Id
        private Long id;
        private int stock;

    @Version

        private int version; // JPA uses this to detect conflicts
}
```

How it works:

1. Transaction A reads a row (version = 1).

2. Transaction B updates the same row then version becomes 2.
3. Transaction A tries to update with version = 1 so Hibernate detects mismatch and throws OptimisticLockException.

Best when conflicts are rare because it avoids unnecessary database locks and scales well.

Pessimistic Locking assumes that conflicts will occur and locks the data to prevent the other transactions from accessing it concurrently.

How it works?

1. Acquire Lock:

- When the transaction reads a record with the pessimistic lock, the database acquires the lock on that record.
- The lock can be of the different types, such as PESSIMISTIC_READ (prevents other transactions from writing) or PESSIMISTIC_WRITE (prevents other transactions from reading or writing).

2. Block Other Transactions

- Other transactions trying to access the same record will be blocked until the lock can be released. This prevents data inconsistency caused by the concurrent modifications.

3. Release Lock

- The lock is held for the duration of the transaction and it can be automatically released when the transaction commits or rolls back.

```
public interface ProductRepository extends JpaRepository<Product, Long> {

        @Lock(LockModeType.OPTIMISTIC)
    Optional<Product> findById(Long id);



        @Lock(LockModeType.PESSIMISTIC_WRITE)
    Optional<Product> findByName(String name);
}
```

Examples:

Optimistic Locking: Imagine an online shopping website. Thousands of people are viewing products, but only a few actually place orders at the same time. If two users try to buy the last piece of a product, both can proceed without blocking each other. But at the moment of update, Hibernate checks a
@Version column. If the version in the database has changed, it throws an OptimisticLockException.

Pessimistic locking: Think about a bank account. Two people try to withdraw money from the same account at the same time. If both see ₹1000 and withdraw ₹800 each, the account would go negative. That is dangerous.

With pessimistic locking, the first transaction locks that row in the database. The second transaction has to wait until the first one commits or rolls back. That way, we never get inconsistent balances.

Reference: Pessimistic and Optimistic Locking in JPA, Spring Boot | by Berat Yesbek | Medium

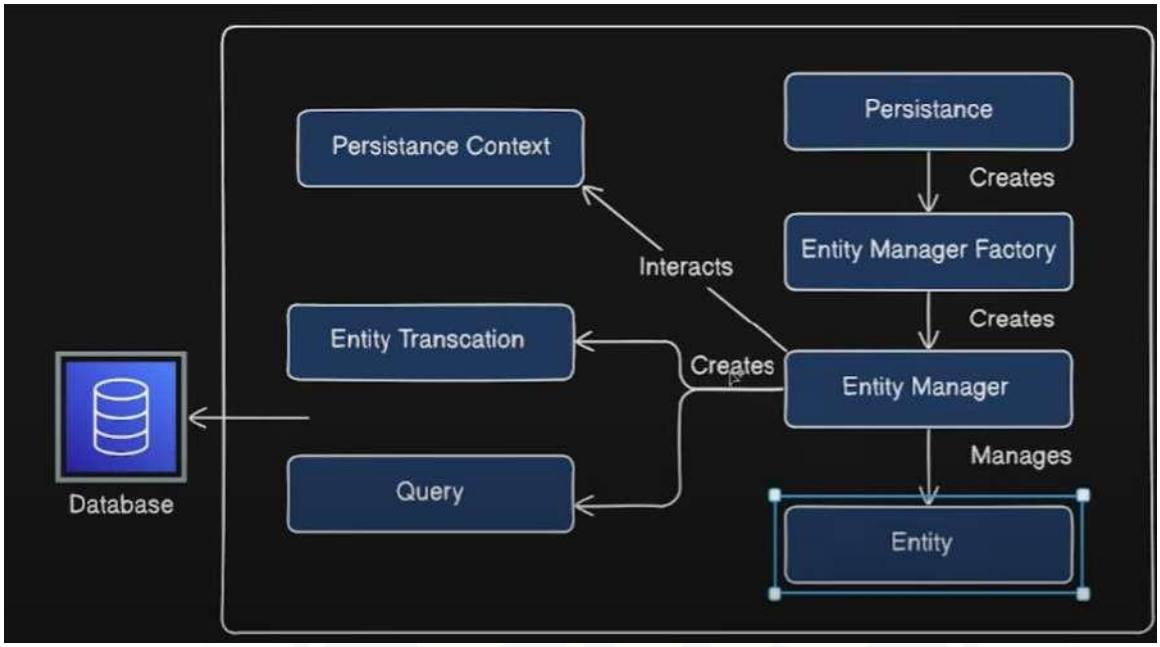
1. Database

This is the actual relational database (like MySQL, PostgreSQL, Oracle) where the tables and data live. JPA's job is to talk to this database but using object, so we don't directly write a lot of SQL.

2. Entity

- An **Entity** is just a normal Java class mapped to a table in the database using @Entity.

306 Spring 54: Explain JPA Architecture?



- Each object of that class represents a row in the table.

```
@Entity
    public class Student
    { @Id
        private Long id;
    private String name;
    private String email;
}
```

Here, the Student class is an entity, mapped to a table Student.

3. Persistence

- The Persistence class is a bootstrap helper provided by JPA.
- Its job is to create an EntityManagerFactory.

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("my-persistence-unit");
```

4. EntityManagerFactory

- It is heavyweight object created only once for the whole application.
- It is Responsible for creating multiple EntityManagers.
- Think of it like a "factory" that handles database managers.

5. EntityManager

- This is the core interface we use in JPA.
- Manages all the entity operations like insert, update, delete, query)

- It interacts with the Persistence Context and Entity Transactions.

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

Student s = new Student();
s.setId(1L);
s.setName("Ramesh");
s.setEmail("ramesh@test.com");

em.persist(s); // inserts into DB
em.getTransaction().commit();
```

6. Persistence Context

- A cache (in-memory storage) where EntityManager keeps track of all entities.
- If we fetch an entity, it is stored here.
    - It is like a temporary storage area between the application and the database.
- Whenever we make changes to an entity during a transaction, the persistence context keeps track of those changes. Once the transaction is done, it saves these changes to the database.

```
Student s1 = em.find(Student.class, 1L); // comes from DB
Student s2 = em.find(Student.class, 1L); // comes from cache (same object)
```

Here, no second DB query is executed for s2 because JPA fetches it from the persistence context.

7. EntityTransaction

- Every operation in JPA happens inside a transaction.
- begin() starts it, commit() saves changes, rollback() undoes them.

```
EntityTransaction tx = em.getTransaction();
tx.begin();

Student s = em.find(Student.class, 1L);
s.setEmail("newmail@test.com");

tx.commit(); // update query fires here
```

8. Query

- We use JPQL (Java Persistence Query Language) or native SQL through EntityManager to fetch data.

```
List<Student> students = em
.createQuery("SELECT s FROM Student s", Student.class)
.getResultList();
```

All the code together:

```
//1. Entity (mapped to a table)
tx.begin();

Student s = new Student();
s.setId(1L);
s.setName("Ramesh");
s.setEmail("ramesh@test.com");

em.persist(s);     // INSERT query happens
here tx.commit();

//5. Persistence Context example
Student s1 = em.find(Student.class, 1L); // from DB
Student s2 = em.find(Student.class, 1L); // from cache (same object)

//6. Query example
List<Student> students = em
  .createQuery("SELECT s FROM Student s", Student.class)
  .getResultList();
```

Reference: JPA Architecture

Here is the flow Your Code (Spring Data JPA) -> EntityManager -> Hibernate (JPA implementation) -> JDBC Driver -> Database.

1. Our Code (Spring Data JPA)

- You define repositories by extending JpaRepository.
- Spring auto-implements CRUD methods like save, findById, delete.

2. EntityManager (JPA)

- Central JPA interface that manages entities and queries.
- Spring Data JPA delegates all operations to this under the hood.

3. Hibernate (JPA Implementation)

- Hibernate implements JPA and powers the EntityManager.
- It generates SQL, manages caching, lazy loading, and ORM mapping.

4. JDBC Driver

- Vendor library like mysql-connector-java or postgresql.
- Converts JDBC calls into the database-specific protocol.

5. Database

- Stores your actual data (tables, rows, indexes).
- Executes SQL and returns results back up the stack.

Spring 55: What will happen under the hood when you call .save method of Repository? The following is code is for save. UserService is calling save() of userRepository.

```
public interface UserRepository extends CrudRepository<User, Integer> {
        // no need to write save(), findById(), etc.
```

```
        // CrudRepository already provides them
}


@Service
public class UserService {

        @Autowired
        private UserRepository userRepository;

        public User saveUser(User user) {
                return userRepository.save(user);
        }
}
```

Step 1 - Call save()

When we write the line `userRepository`.save(new User(1, "CodingLyf"));.
At this moment, the Java object is just a plain User object in memory (a POJO). It is not yet connected to the database.

Step 2 - Repository Layer (Spring Data JPA)

The UserRepository we wrote extends JpaRepository or CrudRepository.That means Spring has already created a **proxy class** for it at runtime.

So, when we call save(), we are not calling the actual code, we are calling a generated proxy that knows how to talk to JPA's EntityManager.

Step 3 - EntityManager Comes Into Play

The proxy internally delegates to **JPA's EntityManager**.
Think of EntityManager as the gatekeeper between the Java objects and the

database. When save() is called, EntityManager decides whether to:

- **Insert** (if the object is new, not in DB yet)
- **Update** (if the object already exists in DB with the same id)

```
629    @Override
630    @Transactional
631    public <S extends T> S save(S entity) {
632
633        Assert.notNull(entity, ENTITY_MUST_NOT_BE_NULL);
634
635        if (entityInformation.isNew(entity)) {
636            entityManager.persist(entity);
637            return entity;
638        } else {
639            return entityManager.merge(entity);
640        }
641    }
```

Inside SimpleJpaRepository, the save() method checks whether the entity is new or already exists.

Step 4 - Check Entity State (Transient, Managed, Detached, Removed)

- At this moment, new User(1, "CodingLyf") is in the **Transient state**.
- That means it exists in memory but is not yet known to the persistence context.
- save() will call EntityManager.persist() if it is new, or merge() if it thinks it already exists.

Step 5 - Persistence Context (a.k.a First-Level Cache)

EntityManager maintains something called a persistence context. Think of it as a cache in memory that holds all entities being tracked in the current transaction.

When we save the User:

- It first gets added to the persistence context.
- Now JPA starts tracking it. If we change the object later, JPA knows what changed.

Step 6 - SQL Generation

The actual **INSERT** or **UPDATE** statement is executed only when:

- The transaction commits, or
- EntityManager.flush() is called.

At that moment, Hibernate translates the entity into SQL. For

example: For a new user > JPA generates an INSERT INTO user ...

query.

For an existing user > JPA generates an UPDATE user ... query.

Step 7 – Transaction Commit

Spring Data JPA usually runs the save() inside a

transaction. When the transaction commits:

- JPA flushes all pending changes from persistence context to the database.
- The SQL is executed.
- The record is physically written in the DB table.

Step 8 – Return Value

Finally, save() returns the persisted entity.

Indirect Questions:

1. Difference between save() and saveAndFlush()

When we call save(), Spring Data JPA doesn't immediately fire the SQL query. Instead it makes entry in the persistent context. It saves in DB only when the transaction is committed or a flush
operation occurs

saveAndFlush () performs the same operation as save() but immediately flushes the changes to the database. This ensures that the entity's state is synchronized with the database right away

2. Difference between persist() and merge()

persist() is used to take a brand-new object (in the *transient* state) and make it managed state, scheduling it for insertion into the database.

merge() is used to take an existing object that is not managed (in the *detached* state) and bring it back into the managed state, scheduling it for update in the database.

307 Spring 56: what is Persistence context in JPA?

Persistence context in JPA as a kind of first-level cache that sits between the application and the database.

- When we fetch or save an entity using EntityManager, JPA doesn't immediately go to the database every time. Instead, it keeps those entity objects inside the persistence context (in memory).
- If we ask for the same entity again, JPA just gives the cached object from the persistence context , no extra SQL.
- Any changes we make to that entity are tracked automatically. When we call commit() or flush(), JPA writes those changes back to the database.

```
EntityManager em = ...;

em.getTransaction().begin();

Employee e1 = em.find(Employee.class, 1); // SQL fired, entity loaded
Employee e2 = em.find(Employee.class, 1); // No SQL, comes from persistence context

e1.setName("John Updated"); // Change tracked in persistence context

em.getTransaction().commit(); // SQL UPDATE happens here
```

Reference: Hibernate Persistence Context. In this article, we'll talk about... | by Emre DALCI | emlakjet | Medium
[Spring Boot] Persistence Context | by Hwangon Jang | Medium

Additional Question:

If I have a student object managed by JPA and I change the student's name multiple times (say 100 times) before committing, will all 100 changes be saved to the database, or only the final change? How does the persistence context determine what to update.

```
@Entity
public class Student {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        private String name;

        // Constructors
        public Student() {}

        public Student(String name) {
                this.name = name;
        }
        // Getters and Setters
}
```

- In JPA/Hibernate, the persistence context acts like a cache for entity objects. When we fetch or create an entity and it is managed by the persistence context
- Changes are not immediately written to the database. They are flushed either when we explicitly call entityManager.flush() or at the end of a transaction.
- Persistence context compares the current in-memory state of the entity with the snapshot taken when the entity was first loaded to decide what to update. This is called **dirty checking**.

- Even if we change the student's name 100 times within the same transaction, only the last change will be saved. The intermediate changes are not individually saved because the
persistence context only cares about the difference between the original and the final state at the time of flush/commit.

308 Spring 57: what are different states on an Entity?

1. Transient (New)

An entity is transient when it is just created using new, not linked to the database or JPA. It is not tracked, not stored, and no SQL will be triggered. Unless we explicitly persist it, it is just a regular object living in memory with no database interaction.

2. Managed (Persistent)

Once we call persist() or fetch it using find(), the entity becomes managed. JPA starts tracking it. Any changes we make are auto-saved to the database on commit. This state ensures automatic synchronization between the Java object and the underlying database table, reducing manual SQL handling.

3. Removed

If a managed entity is passed to remove(), it is marked for deletion. It stays in memory but will be deleted from the database when the transaction commits. While in this state, it is still tracked by JPA, but all changes become irrelevant as the entity is scheduled for removal.

4. Detached

A detached entity was once managed but is no longer tracked; usually after commit or closing the EntityManager. Changes to it won't reflect in the database unless reattached using merge().

309 Spring 58: How Cache works in JPA?

When we work with JPA, caching plays a huge role in performance. Without cache, every time we try to fetch an entity, JPA would hit the database, which is expensive. Caching avoids unnecessary round trips to the database and makes the application faster.

Now let's understand step by step.

1. First-Level Cache (Persistence Context Cache)

Think of the first-level cache like a short-term memory. When we start a transaction, JPA creates a persistence context. Inside this persistence context, all the entities we load or save are stored temporarily in

memory. The

important thing is:

- This cache is tied to the lifecycle of the EntityManager.
- Once we close the EntityManager, the cache is gone.

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

Student s1 = em.find(Student.class, 1L); // First time : SQL query runs
Student s2 = em.find(Student.class, 1L); // Comes from cache, no SQL

em.getTransaction().commit();
em.close();
```

Here, the second time we ask for the student, JPA doesn't bother going to the database. It

already knows the entity is inside the persistence context, so it just returns the cached object.

Key Point: we cannot turn off the first-level cache. It is always there.

2. Second-Level Cache (Shared Cache)

Now imagine your app has multiple sessions and same entity is needed by multiple sessions? Without a shared cache, each session would query the database

again. That is where the **second-level cache** comes in.

- It sits at the EntityManagerFactory level.
- It can be shared across multiple sessions.
- It is not enabled by default; we have to configure it.
- Common providers are **EhCache, Infinispan, Hazelcast**.

```
@Entity
@Cacheable
@org.hibernate.annotations.Cache(
        usage = CacheConcurrencyStrategy.READ_WRITE
)
    public class Student
    { @Id
        private Long id;
        private String name;
}
```

When this is enabled, if one session loads a Student from DB, the next session can pick it up directly from the cache instead of querying the database again. This is where the real performance boost happens in large applications.

Real time Example:

Without Second-Level Cache

- **User A** logs in > System opens a session
- They view **Product with ID=101** > DB is hit, product details are fetched.
- Later, **User B** logs in > New session (
- They also view **Product with ID=101** > Again DB is hit because each session has its own first-level cache only.

So, the same product info is fetched twice from DB, even though nothing changed.

With Second-Level Cache

- **User A** logs in > System fetches **Product 101** from DB and puts it into second-level cache.
- **User B** logs in > When they view **Product 101**, Hibernate/JPA picks it from the cache instead of hitting DB again.

That is why in real-world apps with many users requesting the same data (products, courses, profiles), second-level cache saves number of DB hits.

Point to remember: First level cache is session-scoped, whereas second level cache is application scope

Indirect Questions:

1. When will cache be updated?

The second-level cache is updated automatically whenever Hibernate changes the entity (insert, update, delete). If changes happen outside Hibernate means if we update the DB directly, we need refresh or eviction to sync it.

2. Can caching cause stale data issues?

Yes. If another transaction updates the database, the cache might still hold the old value. To avoid this, second-level caches use cache invalidation or concurrency strategies like READ_ONLY, NONSTRICT_READ_WRITE, etc.

- READ_ONLY > cache never updated (safe for reference data like countries, constants).
- READ_WRITE > updates both DB and cache when entity changes.
- NONSTRICT_READ_WRITE > DB updated immediately, cache updated lazily (may serve slightly stale data).

We must have proper configuration for cache, like TTL Time-to-Live

If the application is clustered, we must use a distributed L2 cache like Hazelcast

310 Spring 59: How to connect to multiple databases?

I would recommend to watch this video first, then go through the theory below.

Reference video: Spring Boot : How to connect with multiple databases using Spring

Data JPA When we work with multiple databases, we need three main things for *each*

database:

1. **DataSource**: the connection details (URL, username, password). We need do define multipe datasoures
2. **EntityManagerFactory**: responsible for creating and managing JPA's entity managers (basically the bridge between the entities and the DB).
3. **TransactionManager**: tells Spring how to handle transactions

(commit/rollback). And finally, we map specific **entities** and **repositories** to the right

database.

Let's say we have two databases:

- Primary DB: users (userdb)
- Secondary DB: orders (orderdb)

Step 1: First, we need to configure the data sources in properties file.

```
            # Primary DB (users)
spring.datasource.primary.url=jdbc:mysql://localhost:3306/usersdb
spring.datasource.primary.username=root
spring.datasource.primary.password=secret
spring.datasource.primary.driver-class-name=com.mysql.cj.jdbc.Driver

# Secondary DB (orders)
spring.datasource.secondary.url=jdbc:mysql://localhost:3306/ordersdb
spring.datasource.secondary.username=root
spring.datasource.secondary.password=secret
spring.datasource.secondary.driver-class-name=com.mysql.cj.jdbc.Driver
```

Step 2: Package structure

A neat way is to group everything by database:

```
com.example.app
 ├── userdb
 │     ├── entity
 │     │     └── User.java
 │     ├── repository
 │     │     └── UserRepository.java
 │     └── config
 │           └── UserDbConfig.java
 ├── orderdb
 │     ├── entity
 │     │     └── Order.java
 │     ├── repository
 │     │     └── OrderRepository.java
 │     └── config
 │           └── OrderDbConfig.java
 └── service
       └── DemoService.java
```

- Each DB has its **own package**.
- Inside it, we separate entity, repository, and config.
- Shared business logic (services, controllers) sits outside, so they can use repos from both DBs if needed.

Step 3: Define @EnableJpaRepositories

1. @EnableJpaRepositories is a Spring Data JPA annotation used to enable the detection and creation of Spring Data JPA repositories within a Spring application.

Purpose:

- It instructs Spring to scan specified packages for interfaces extending Spring Data JPA's JpaRepository (or related interfaces like CrudRepository, PagingAndSortingRepository).
- For each such interface, Spring automatically generates a concrete implementation (proxy) at runtime, allow to inject and use these repository interfaces directly in the application.

```
@EnableJpaRepositories(
                    basePackages = "com.example.users",
          entityManagerFactoryRef = "userEntityManagerFactory",
          transactionManagerRef = "userTransactionManager"
             )
```

2. basePackages = "com.example.users"

This is the folder (package) where our repository interfaces exists (Check above package info).

- Example: inside com.example.users.repository, we have UserRepository, UserProfileRepository.
- Spring will scan this package, find those interfaces, and auto-generate classes for

them. Without this, Spring won't know where the repositories are hiding.

3. entityManagerFactoryRef = "userEntityManagerFactory"

Now, repositories need to know *which database connection + mapping rules* they belong to. That is handled by the **EntityManagerFactory**.

"Repositories in com.example.users should use the

userEntityManagerFactory." Why is this important?

- In single-DB apps, there's only one entity manager, so we don't need this.
- In multi-DB apps, each DB has its own entity manager, so we must explicit set it.

4. transactionManagerRef = "userTransactionManager"

Databases don't just read and write; they also commit or roll back transactions.

It says "When these repositories do database work, handle their transactions using userTransactionManager."

So:

- If something fails in a UserRepository call, this manager decides whether to rollback or commit.
- Again, in multi-DB setups, each DB has its own transaction manager.

Code link: ashokitschool/springboot_multi_db_config: springboot_multi_db_config

Indirect Questions:

1. What happens if you don't specify basePackages in @EnableJpaRepositories?

- By default, it scans the same package where our configuration class is placed (and sub- packages).
- If our repos are elsewhere, they won't be found and we willl get No qualifying bean errors.

2. Why do we need entityManagerFactoryRef in a multi-database setup?

- Because each DB has its own entity manager.
- Without this reference, Spring wouldn't know which database connection to use for those repositories.
- It is used to mixing up entities of DB1 with DB2.

3. Can we skip transactionManagerRef? What happens?

- If we have only one database, Spring will auto-wire the default transaction manager.
- In multi-DB apps, we must specify it, otherwise the wrong transaction manager might get applied, leading to runtime errors or commits going to the wrong DB.

4. If I put all my repositories (User and Order) in the same package, will @EnableJpaRepositories still work?

- Yes, @EnableJpaRepositories will still work even if we put all the repositories (User, Order, etc.) in the same package.
- But it is good to put repositories according to the DB for better maintainability:

5. What's the difference between @EnableJpaRepositories and @Repository on an interface?

- @Repository marks an interface or class as a Spring bean.
- @EnableJpaRepositories is what actually scans for those interfaces and generates implementations.
- Without @EnableJpaRepositories, our @Repository interfaces won't be picked up automatically.

6. How does Spring Boot behave if you don't even use @EnableJpaRepositories anywhere?

- For a single database setup, Spring Boot auto-configures it.
- That is why in single DB projects, we never see @EnableJpaRepositories
- For multiple DBs, we must configure it explicitly, otherwise Spring won't know how to separate repositories by database.

7. Can a repository use multiple EntityManagerFactories at once?

- No. A repository is tied to exactly one entity manager (hence one DB).
- If we need data from multiple DBs, we
- fetch them via different repositories in the service layer.

311 Spring 60: How do you create Custom repository in Spring JPA?

Spring Data JPA gives us a lot out of the box with JpaRepository, like findAll(), save(), delete(), etc. But sometimes, we need custom queries or complex logic when we can't achieve with derived query methods. That is where a custom repository comes in.

Reference: Spring data JPA custom repository. Watch this video before reading the theory.

Steps to create a custom repository

1. Create a custom interface, this is where we declare methods that aren't in JpaRepository.

```
public interface EmployeeRepositoryCustom {
        List<Employee> findEmployeesWithHighSalary(double minSalary);
}
```

2. Create an implementation class: By convention, name it <RepositoryName>Impl. Here we directly use EntityManager.

```
import jakarta.persistence.EntityManager;
public class EmployeeRepositoryImpl implements EmployeeRepositoryCustom {

        @PersistenceContext
        private EntityManager entityManager;

        @Override
        public List<Employee> findEmployeesWithHighSalary(double minSalary) {
                return entityManager.createQuery(
                        "SELECT e FROM Employee e WHERE e.salary > :salary",
            Employee.class)
                        .setParameter("salary", minSalary)
                        .getResultList();
        }
```

3. Extend the main repository with the custom one. Now EmployeeRepository has both the standard JPA methods and the custom one.

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository
              extends JpaRepository<Employee, Long>, EmployeeRepositoryCustom {
}
```

Code: shameed1910/spring-jpa-custom-repo

312 Spring 61: What is connection pooling, how it helps in Spring?

Connection pooling is a technique used to manage and optimize database connections in applications. Instead of opening a new database connection for each request and then closing it after use, connection pooling creates a pool of pre-defined and reusable connections. When an application needs to interact with the database, it borrows a connection from this pool, uses it, and then returns it to the pool for later reuse.

Since Spring Boot 2.x, **HikariCP** has been the default connection pool due to its speed,

simplicity, and reliability. HikariCP is designed to be a lightweight and high-performance connection pooling library that performs well under various loads.

Setting up HikariCP in Spring Boot is straightforward since Spring Boot automatically configures HikariCP as the default connection pool if it is available in the classpath. HikariCP settings can be customized in the application.properties or application.yml file.

```
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
```

Reference: How to Use Connection Pooling for Faster Database Access in Spring Boot | by Alexander Obregon | Medium

313 Spring 62: What do you know about @NoRepositoryBean?

```
@NoRepositoryBean
public interface CrudRepository<T, ID> extends Repository<T, ID>  { }
```

Spring Data JPA is a powerful tool for simplifying database access in Spring Boot applications, providing repository interfaces that reduce the boilerplate code for CRUD operations.

Sometimes we want to define a **base repository** with shared methods for multiple repositories, but we don't want Spring to create a bean for that base interface itself. That is where @NoRepositoryBean comes in.

The primary function of @NoRepositoryBean is to prevent Spring Data JPA from creating a repository proxy for the annotated interface or class. It is typically used on base repository interfaces or classes that define common methods for a set of repositories.

Example: **Library System**

Let's say we are building a library system. We want a base repository with common methods for all items in the library (books, magazines, etc.).

Entities:

```
@Entity
    public class Book
    { @Id
        private Long id;
    private String title;
    private String author;
        // getters/setters
}

@Entity
    public class
    Magazine { @Id
        private Long id;
    private String title;
    private int
    issueNumber;
```

Now create a BaseRepository called `LibraryItemRepository, which contains common method to both Book and Magazine.`

```
@NoRepositoryBean
public interface LibraryItemRepository<T, ID> extends JpaRepository<T, ID> {

        // A custom method shared by all library
    items List<T> findByTitleContaining(String
    keyword);
```

Now Create Separate Respositories for Book and Magazine.

```
    public interface BookRepository extends LibraryItemRepository<Book,
    Long> { List<Book> findByAuthor(String author);
}

    public interface MagazineRepository extends LibraryItemRepository<Magazine, Long> {
    List<Magazine> findByIssueNumber(int issueNumber);
}
```

LibraryItemRepository is marked with @NoRepositoryBean, so Spring **does not create a bean** for it.

But Spring will create beans for BookRepository and MagazineRepository, and they both inherit the shared findByTitleContaining() method.

It is useful to avoid duplicate code in multiple repositories.

Reference: Simplifying Database Access with @NoRepositoryBean | by Arsen Sargsyan | Octa Labs Insights

314 Spring 63: What is Spring Security?

For Spring security along with JWT check this video: Spring Security 6 with Spring Boot 3 and JWT Tutorial

Spring Security is a framework that helps us to protect the Java applications. Think of Spring Security as a bodyguard for the application. Whenever a request comes into the system, it doesn't let anyone walk in freely.

It first checks who the user is (authentication) and then decides what the user is allowed to do (authorization).

So, in short:

- Authentication: Verifying identity (like logging in with username/password).
- Authorization: Checking permissions (like only admins can delete users).

Without Spring Security, we would have to write a lot of this logic by hand; validating users, handling login failures. Spring Security handles all of this in a structured way.

Imagine we are building an online banking app. we don't want just anyone to:

- Log in with any random credentials
- View someone else's account details
- Make unauthorized transactions

Spring Security makes sure that doesn't happen. It places a **security filter chain** in front of the application and intercepts requests before they reach the business logic.

Here is the most basic Spring Security setup in Spring Boot (using the new way with SecurityFilterChain). We will see every option in the upcoming questions.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

        @Bean
            public SecurityFilterChain filterChain(HttpSecurity http) throws
        Exception { http
                        .authorizeHttpRequests(auth -> auth
                                .requestMatchers("/admin/**").hasRole("ADMIN")
                                .requestMatchers("/user/**").hasRole("USER")
                                .anyRequest().authenticated()
                        )
                        .formLogin(Customizer.withDefaults()); // enables default login
            form
                    return http.build();
        }
```

315 Spring 64: How do you enable Security in Spring applications?

Just add the dependency spring-boot-starter-security.

Once this is on the classpath, Spring Boot automatically configures a basic security setup.

By default, it secures all endpoints with HTTP Basic authentication and generates a default password (we can see it in logs).

Spring boot does default configuration when we added starter-security to class path.

Spring Boot Defaults (with spring-boot-starter-security)

1. All endpoints are secured

   By default, every HTTP endpoint requires authentication. Even / or /home is protected unless explicitly allowed.

2. HTTP Basic authentication enabled

   Spring Boot automatically provides a login mechanism using HTTP Basic. Spring will give default login page.

3. Auto-generated user

   Spring Boot creates a default user with username user. Password is auto-generated at runtime and printed in logs. This password changes every time the app restarts.

4. Password is encoded

   The default password uses BCryptPasswordEncoder internally.

5. CSRF protection enabled

For web applications, POST, PUT, DELETE, PATCH requests are protected with CSRF by default. But if we want override the default we need to a security configuration file with **@EnableWebSecurity.**

316 Spring 65: What is @EnableWebSecurity?

The @EnableWebSecurity annotation is used to enable the web security of an application. When this annotation is added to a configuration class, it indicates that the class will provide the necessary configurations and settings for securing the web application.

In Spring boot application, using the @EnableWebSecurity annotation is optional. If we don't write one, Spring uses the default one.

When we want to override the default configuration we will use EnableWebSecurity.

Ex: when we want to make some pages public or when we want to provide custom filters

```
 @Configuration
@EnableWebSecurity // This is still used to mark this as the security config class
public class SecurityConfig {

	// Bean to define security rules (replaces configure(HttpSecurity http))
    @Bean
	public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
		return http
			.authorizeHttpRequests(auth -> auth
				.requestMatchers("/public/**").permitAll()
				.anyRequest().authenticated()
			)
			.formLogin(withDefaults())
			.httpBasic(withDefaults())
			.build();
	}

	// Bean to ignore certain paths (e.g., for static resources)
    @Bean
	public WebSecurityCustomizer webSecurityCustomizer() {
		return (web) -> web.ignoring().requestMatchers("/css/**", "/js/**");
	}
```

317 Spring 66: What are the key features of Spring Security?

1. Authentication: Authentication is the process of validating the identity of a user who is attempting to access the application. Spring Security provides multiple authentication methodologies
(Username/Password Authentication, JWT, OAUTH2, SSO).

2. Authorization (Access Control): Authorization is the process of granting permissions or rights to authenticated users. Once a user is successfully authenticated, authorization determines what actions or resources they are allowed to access within an application.

Example: Only admins can delete users.

3. Protection against Common Attacks

Spring Security automatically protects against many common vulnerabilities:

- CSRF (Cross-Site Request Forgery) > prevents fake form submissions.
- XSS (Cross-Site Scripting) > Escapes dangerous inputs.

4. Password Security

Passwords are never stored in plain text. Spring Security provides hashing algorithms like BCrypt, so even if the database is stolen, raw passwords aren't exposed**.**

Indirect Questions:

1. Difference between Authentication and Authorization

Authentication is checking who the user is. Example: logging in with the email and password.

Authorization is checking what the users are allowed to do. Example: an admin can delete users, but a normal user cannot.

318 Spring 67: What is authentication and different types of it?

Authentication is the process of verifying the identity of a user or system before granting access to resources.

Types of Authentications:

1. **Username and Password Authentication**: User provides a username and password and system verifies against the DB.

2. **Token-Based Authentication (JWT)**: User logs in once and receives a token (like JWT). Every subsequent request includes the token (usually in Authorization header).

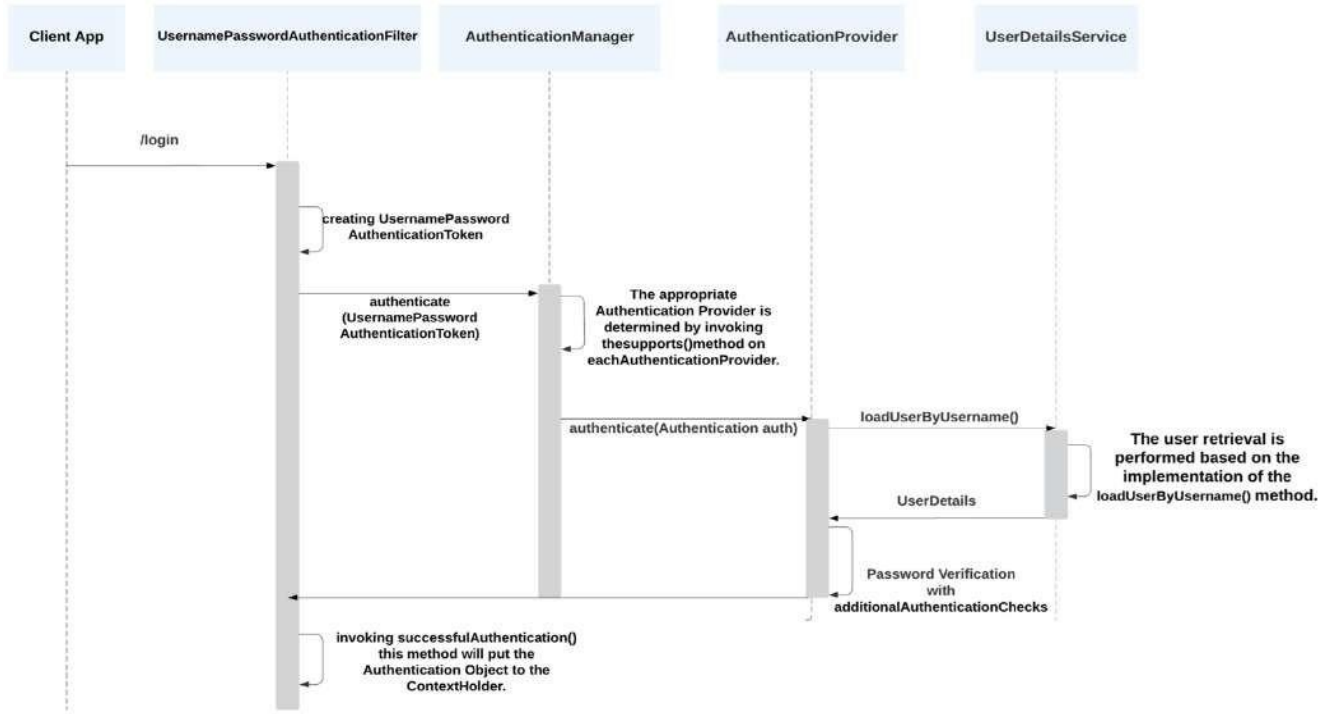
3. **HTTP Basic Authentication**: Credentials are sent in base64-encoded format in the request header. Simple, but not recommended for production unless combined with HTTPS.

4. **OAuth2 / OpenID Connect**: Delegated authentication via third-party providers like Google, Facebook, or Okta. The system relies on the external provider to authenticate the user and provide an access token. Useful for SSO (Single Sign-On).

5. **Multi-Factor Authentication (MFA)**: Combines two or more authentication factors:

    1. Something we know (password)
    2. Something we have (OTP, mobile app)
    3. Something we are (fingerprint, face)

319 Spring 68: What is the authentication flow in Spring security?



Authentication Flow in Spring Security

Request Enters the System

Every request first passes through the **SecurityFilterChain**. Here, Spring Security decides which filter should handle it.

If the request is a POST with username and password, the

UsernamePasswordAuthenticationFilter will be invoked.

- If the request has an Authorization: Basic base64(username:password) header, then the **BasicAuthenticationFilter** will be called.
- And so on, depending on the type of authentication method.

2. **Filter Extracts Credentials**
   The chosen filter pulls authentication details (like username/password or token) from the request and builds an **Authentication object**.
3. **AuthenticationManager Delegates**
   The filter then hands this Authentication object to the **AuthenticationManager**. This manager doesn't validate directly but it delegates to a chain of **AuthenticationProviders**.
4. **AuthenticationProviders Validate**
   Each AuthenticationProvider tries to handle the request:
   - If one recognizes the type of authentication and validates it successfully (say, by comparing username and password against the database), the process moves forward.

- If not, the next provider in the list gets a chance.
- AuthenticationProvider Uses **UserDetailsService (**Refer Question No. 327. Spring 75)
- If the provider is something like **DaoAuthenticationProvider**, it calls the UserDetailsService.
- UserDetailsService loads the user's details from the system (e.g., a database row with username, password, and roles).
- The provider then compares the supplied credentials (from the request) with the stored credentials (from UserDetailsService).

5. **Success : Authentication Stored**
   Once a provider authenticates successfully, a fully populated **Authentication object** is created. This object contains the user's details and their authorities (roles/permissions).
6. **SecurityContext Updated**
   Finally, this Authentication object is placed into the **SecurityContext**, which is like a session-level memory of "who is currently logged in." From this point on, Spring Security uses this context to decide whether the user has access to specific resources.
7. **Failed** AuthenticationException will be thrown

Additional Question:

1. What is SecurityContext and SecurityContextHolder

When a user log into a Spring Security application, the system needs a place to store information about who the user is e and what he can do. That is where the SecurityContext comes in.

Think of the SecurityContext as a little box that holds the authentication details:

- Who the user is (username, roles, authorities).
- Whether they are authenticated or anonymous.

Every request has its own SecurityContext, so Spring Security knows *which user* is making the request and what they're allowed to access.

SecurityContextHolder

Now, how do we access the SecurityContext? That is the job of the SecurityContextHolder. It is like the "manager" that gives the current SecurityContext for the running thread.

```
Authentication auth = SecurityContextHolder
.getContext()
.getAuthentication();

String username = auth.getName();
Collection<?> roles = auth.getAuthorities();
```

getContext() gives the SecurityContext.

getAuthentication() gives the actual login details.

320 Spring 69: What is JWT and how does it differ from Basic auth?

Both JWT (JSON Web Token) and Basic Authentication are ways to check a user's identity.

Basic Authentication

With Basic Auth, the client (like a browser or mobile app) sends the username and password with every request. These credentials are usually sent in the HTTP header, encoded in Base64 (which is not

encryption, just a form of encoding).

- **Pros**: Simple to implement, no extra setup needed.
- **Cons**: Insecure if not used over HTTPS (credentials can be stolen), and credentials are repeatedly exposed on every request.

JWT takes a different approach. Instead of sending the username and password every time, the server gives the client a **token** after a successful login. This token is a signed string containing encoded information about the user (like their username and roles).

From that point on, the client only needs to send the token with each request, no passwords involved. The server checks if the token is valid and not expired, then processes the request.

Good to know

Before JWTs became popular, web apps commonly used session IDs. Here is how that worked:

1. When user logs in, server creates a session and stores user info on the server.
2. Server gives the client a session ID, usually stored in a cookie.
3. For each request, the client sends the session ID, and the server looks up the session

data. This works fine for small apps, but it doesn't scale well:

- The server must keep session data in memory or a database. More users mean server requires more memory.
- In a load-balanced system with multiple servers, we need session to store in shared cache (Redis, Memcached). That adds complexity.
- REST APIs are supposed to be stateless, but sessions make the server stateful.

JWT solves these problems by keeping all the necessary user information inside the token itself, so the server doesn't need to store anything about the user between requests.

Reference: What is JWT? JSON Web Tokens Explained (Java Brains)

Additional Question:

1. Should you use JWT or Session-based authentication in the microservices environment?

In a microservices environment, JSON Web Tokens (JWTs) are generally preferred over session-based authentication.

In session-based authentication, the server has to store session data. Works fine for small apps, but in microservices it is hard because every service would need to share session data.

With JWT after login, we get a token. we send that token every time. Each service can check it without talking to others. Much easier for microservices.

2. How does Basic Authentication work in Rest API?

Basic Authentication in REST API works like this:

1. When a client (say, a browser or Postman) makes a request, it attaches an HTTP

   header: Authorization: Basic <Base64 encoded username:password>
2. The server receives it, decodes the Base64 string back to username:password, and checks against its user store (like DB or in-memory).
3. If it matches, the request is allowed. If not, it is rejected with 401 Unauthorized.

Problem: credentials are sent on every request, so it must be used over HTTPS to avoid the exposure of passwords

3. In Spring boot, how is session managed?

When a user logs in, the server needs to remember the user between requests. HTTP itself is stateless, meaning it forgets the user every time. To fix that, Spring Boot uses session management.

1. Session Creation

- After a successful login (say with form login or basic authentication), the server creates a session object (HttpSession).
- This session is just a unique ID (called JSESSIONID) stored on the server.

2. Session ID Sharing

- The server sends this ID back to the browser as a cookie.
- On every new request, the browser automatically sends this cookie back.
- Spring uses it to look up the stored session and retrieve the user's authentication details.

3. Where the Session Data Lives

By default, Spring Boot keeps session data in memory (in the server's JVM). But we can change this to:

- Database
- Redis (common for microservices and scaling)
- Custom storage

321 Spring 70: What is structure of the JWT?

A JWT (JSON Web Token) is just a long string, but it is actually made of three distinct parts, separated by dots (.):

xxxxx.yyyyy.zzzzz

1.**Header**

A small JSON object that describes the

token. It usually contains:

- The type of token (typ) , always "JWT".
- The signing algorithm used (alg) ike "HS256" or "RS256".

```
{
    "alg": "HS256",
    "typ": "JWT"
}
```

2. Payload

This is the main content of the token.

It holds the information about the user and some

metadata. Claims are key-value pairs, for example:

sub: Subject (the user ID)

name: Username

roles: User's roles or

authorities exp: Expiration

time of the token

```
{
    "sub": "12345",
    "name": "CodingLyf",
  "roles": ["ADMIN", "USER"],
  "exp": 1699999999
}
```

3. Signature

This is the security part of the token

It is created by `Base64UrlEncode(header) + "." + Base64UrlEncode(payload)` and then signing it with a secret key or private key using the algorithm from the header.

Reference: **What is the structure of a JWT - Java Brains**

322 Spring 71: How server validates the JWT token?

When a user logs in successfully, the server creates a JWT token and sends it back to the client (a browser or an API client). From then on, the client must store this token (usually in local storage or cookies) and attach it to every request it makes. This way, the server can check if the request comes from an authenticated user.

But here is the real question is, how does the server know the JWT is valid?

1. Browsers send Token in Authorization header. Authorization: Bearer <jwt_token>
2. Server will read this header and extracts the token.
3. From Token, we need to extract the username and now verify this username with the usernames stored in DB. If it matches token is valid , if it is invalid token and returns 401 error.

Point to remember: All these steps will be done in OncePerRequestFilter.

OncePerRequestFilter ensures the JWT validation logic runs exactly once for each request, extracts the username from the token, validates it, and sets authentication in the security context.

Reference: #38 Spring Security | Validating JWT Token

Additional Question:

If JWT Token is expired midway of the request, how would you handle this?

Handling a JWT token that expires midway through a request typically involves a combination of client-side and server-side mechanisms, often leveraging refresh tokens.

1. Server-Side Handling:

Token Validation:

The server-side API endpoint receiving the request must validate the JWT token. If the token is found to be expired during this validation, the server should respond with a 401 Unauthorized status code.

Refresh Token Endpoint:

A dedicated endpoint for refreshing access tokens is crucial. This endpoint accepts a valid refresh token and, upon successful verification, issues a new, unexpired access token.

2. Client-Side Handling:

Intercepting Responses:

The client-side application (e.g., a web or mobile app) should have a mechanism to intercept API responses, specifically looking for 401 Unauthorized errors.

Refresh Token Flow:

When a 401 Unauthorized response is received due to an expired access token, the client should:

- Attempt to use the stored refresh token to request a new access token from the refresh token endpoint.
    - If the refresh token is valid and a new access token is successfully obtained, the client should update its stored access token and then retry the original failed request with the new access token.

323 Spring 72: How to implement JWT authentication in Spring boot?

I recommend to watch this video: Easy JWT Authentication & Authorization with Spring Security | Step- by-Step Guide

Code Link: learnwithiftekhar/Spring-Security-JWT

JWT (JSON Web Token) authentication in Spring Boot is basically about replacing the default session- based authentication with stateless token-based authentication.

1. User Login Request

- The client (browser or mobile app) sends a login request with username and password.
- This hits the AuthenticationController.

2. Authentication Manager + UserDetailsService

- Spring Security passes the credentials to the AuthenticationManager.
- That, in turn, uses the custom UserDetailsService to fetch the user and check the password with a PasswordEncoder.

If it is valid > user is authenticated. If not > reject immediately.

3. JWT Token Creation

- Instead of creating a session, we now generate a JWT.
- JWT contains claims like: username, roles, issued time, expiration.
- It is signed with the secret key (so no one can tamper with it).
- It spring we use JWT Library (commonly used is jjwt) This token is then sent back to the client in the response.

4. Client Stores the JWT

- The client usually stores this JWT in localStorage or sessionStorage or cookies (in browsers), or secure storage (in mobile apps).
- From now on, every request from client : server includes this token in the Authorization header: Authorization: Bearer <jwt_token>

5. JWT Authentication Filter

- We add a custom filter by extending OncePerRequestFilter in the Spring Security filter chain (before UsernamePasswordAuthenticationFilter).
- This filter:

1. Reads the Authorization header.
2. Extracts the JWT token.
3. Validates it (check signature + expiration).
4. If valid : create an Authentication object and put it into the SecurityContext.
5. If invalid : reject request with 401.

324 Spring 73: How does SecurityFilterChain works and list down the filters?

In Question No 320, Spring 68 we have seen Architecture of the Spring security. From here we are going depth into every component of the Spring security.

In Spring Security, everything that protects the application runs through a chain of filters. Think of it like airport security , we don't just pass through one gate, we pass through a series of checks:
baggage scan, passport check, body scan, etc. Each check is a filter with a specific

job. The SecurityFilterChain is literally that chain of filters applied to incoming

requests.

- Every HTTP request enters the filter chain.
- Each filter either does some security work (like authentication, authorization, CSRF check, session check) or passes control to the next filter.
- If a request fails a check (say invalid credentials), the chain is broken, and the request is rejected immediately.

Without this chain, Spring Security can't intercept and secure requests.

Types of filters:

- **Authentication Filter**: Responsible for handling user authentication.
- **Authorization Filter:** Enforces access control policies and checks whether a user is authorized to access a resource.
- **CSRF** (Cross-Site Request Forgery) Filter: Protects against CSRF attacks. (Refer Question No)
- **UsernamePasswordAuthenticationFilter**: Validates the username and password for the URL (/login) with the default credentials provided at startup.
- **BasicAuthenticationFilter**: This filter is responsible for processing any request that has an HTTP request header of Authorization, Basic Authentication scheme, Base64 encoded username- password.
- **BearerTokenAuthenticationFilter**: Extracts JWT tokens for authentication.
- **FilterSecurityInterceptor** : This filter is responsible for authorizing every request that passes through the filter chain before the request hits the controller.

```
@Bean
        public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws
Exception {
                httpSecurity
                                .csrf(csrf -> csrf.disable())
                                .authorizeHttpRequests(
                                                request -> request
                                                                .requestMatchers("regist
                                er","login").permitAll()
                                                                .anyRequest().authentica
                                ted()
                                )
                                .httpBasic(Customizer.withDefaults())
                                        .addFilterBefore(jwtAuthenticationFilter,
                        UsernamePasswordAuthenticationFilter.class);
```

Explanation for above code Snippet:

authorizeHttpRequests: It sets authorization rules.

requestMatchers is used to define which request URLs or HTTP methods a rule should apply to. Basically, it tells Spring Security, "for these specific paths, apply the following access rules.

From the above code, Requests to /register and /login are public means no authentication needed. Other than these APIs, remaining all should go through authentication process.

Behind the scenes, this adds a FilterSecurityInterceptor into the chain, which checks permissions before letting the request proceed.

httpBasic: This enables HTTP Basic Authentication. Spring Security will insert a BasicAuthenticationFilter into the filter chain. This filter checks the Authorization: Basic ... header, extracts username/password, and tries to authenticate the user.

addFilterBefore allows us to insert the custom filter into the SecurityFilterChain before another specific filter.

addFilterBefore(myFilter, UsernamePasswordAuthenticationFilter.class)

Here myFilter runs *before login/authentication* happens. Commonly used for JWT validation so that requests are authenticated without hitting the username-password login flow.

Indirect Questions:

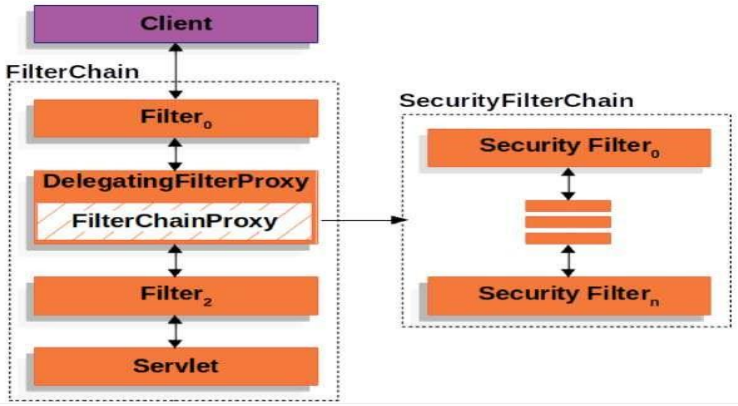
1. How you make certain APIs skip authentication? Using requestMatchers
2. How can you disable the CSRF

   Using CSRF filter and csrf.disable() method

325 Spring 74: Internals on SecurityFilterChain?

1. DelegatingFilterProxy

Spring's security filters live inside the Spring ApplicationContext (managed beans). But the



servlet container (like Tomcat, Jetty) doesn't know anything about that

So DelegatingFilterProxy acts as a bridge.This is a Servlet Filter that comes from the Spring Framework, not just Spring Security. It acts as a bridge between the Servlet container (like Tomcat, Jetty) and the Spring-managed beans

Servlet container calls > DelegatingFilterProxy > delegates to Spring bean (FilterChainProxy).

2. FilterChainProxy

- This is a Spring Security-specific filter.
- Inside Spring, all the security rules (authentication, authorization, CSRF, JWT filters, etc.) are arranged as a list of security filter chains.
- FilterChainProxy is responsible for picking the right chain based on the request (URL matching) and executing it.

Flow:
Request > DelegatingFilterProxy > FilterChainProxy > Security Filters > DispatcherServlet > Controller

326 Spring 75: What is UserDetailsService and how it works?

It is just an interface provided by Spring Security.

Its main job is to fetch user information (like username, password, roles) from the data source (database, LDAP, in-memory, etc.) so that Spring Security can use it during authentication.

So, if Spring Security wants to log someone in, it calls

loadUserByUsername(). We return a UserDetails object that represents

that user.

```
public interface UserDetailsService {
        UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException;
```

How does it work in the authentication process?

1. User tries to log in (for example, submitting a username/password).
2. AuthenticationManager receives the credentials.

3. AuthenticationManager asks a UserDetailsService to fetch the user by username.
4. The implementation of UserDetailsService loads the user details from database (or wherever).
5. Spring Security wraps it in a UserDetails object.
6. The AuthenticationProvider checks if the password matches and assigns roles/authorities.
7. If it all checks out, authentication succeeds and a SecurityContext is created for that user.

```
@Service
public class MyUserDetailsService implements UserDetailsService {

        private final UserRepository userRepository; // JPA repo

        public MyUserDetailsService(UserRepository userRepository) {
                this.userRepository = userRepository;
        }

        @Override
        public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
                UserEntity user = userRepository.findByUsername(username)
                        .orElseThrow(() -> new UsernameNotFoundException("User not
            found: " +
username));

                return org.springframework.security.core.userdetails.User
                                .withUsername(user.getUsername())
                                .password(user.getPassword())    // already encrypted
                                .authorities("ROLE USER")


        }
}
```

Key points

- UserDetailsService does not authenticate. It only fetches user data.
- Password check is done by an AuthenticationProvider (like DaoAuthenticationProvider).
- We can have multiple UserDetailsService implementations if needed.
- If we don't configure one, Spring can use an in-memory we by default.

327 Spring 76: Explain about Password Encryption and why it is important?

When a user registers or logs in, their password is the most sensitive piece of data. If we store or transmit it as plain text, anyone with database access (admins, hackers, even accidental log dumps) can see it. That is a massive security risk.

Instead, Spring Security (and security best practices in general) never stores plain passwords. Instead, it stores a hashed and salted representation of the password.

- Hashing means converting the password into a fixed-length string using an algorithm (like BCrypt).
- Salting means adding a random string before hashing, so that even if two users have the same password, their stored hashes are different.

How Password Encryption Works in Spring Security

PasswordEncoder Interface

- Spring Security provides PasswordEncoder, an interface for encoding and matching passwords.
- The most commonly used implementation is BCryptPasswordEncoder.

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
String encodedPassword = encoder.encode("mySecretPassword");
```

328 Spring 77: What is CSRF and how it works in Spring?

Cross-Site Request Forgery (CSRF) is a web security vulnerability that tricks a user's browser into performing unwanted actions on a website where the user is authenticated.

For example:

- A user is logged into his bank site in one tab.
- In another tab, the user visits an attacker's site (Thinking that it is normal website).
- That site secretly submits a money transfer request to the bank using user session cookies.
- Since the browser automatically sends cookies with every request, the bank thinks the user triggered the request.

Spring Security protects against CSRF by using a synchronizer token pattern:

How it works in Spring Security:

- When the server renders a form or page, it generates a unique CSRF token and stores it in the user's session.
- This token is also embedded into the HTML (for example, as a hidden input field or in a meta tag for AJAX).
- When the user submits a form or makes a state-changing request (POST, PUT, DELETE), the token must be included in the request.
- Spring Security checks if the token in the request matches the token stored in the session. If the tokens don't match > request is rejected.
If they match > request is valid and processed.

329 Spring 78: What is CORS, how we need to resolve it?

CORS stands for **Cross-Origin Resource Sharing**.

- Browsers enforce a security policy called the **Same-Origin Policy**, which prevents a webpage's scripts from contacting or loading data from other websites.
- For example: If frontend is at http://localhost:3000 and backend API is at http://localhost:8080. A fetch request from frontend to backend is **cross-origin**.
- By default, the browser blocks such requests unless the server explicitly allows them. That is where CORS comes in.

Using NGINX to handle CORS is common, especially when we want to manage it outside the application.

NGINX acts as a reverse proxy in front of the Spring Boot app. we can configure it to add the CORS headers in HTTP responses so browsers allow cross-origin requests.

How CORS works in Spring (Refer Question No. 378, Spring 126)

330 Spring 79: What is Outh2 and how does it differ from JWT?

OAuth2 is an authorization framework that allows a user to grant a third-party application limited access to their resources without sharing credentials.

- It is not an authentication protocol by itself, though it is often used with OpenID Connect

(OIDC) for authentication.

- OAuth2 defines roles and flows for issuing access tokens to clients.
- It is widely used for APIs, social logins, and microservices.

Core components in OAuth2:

1. **Resource Owner**: The person who owns the data or resources being accessed (e.g., the Google account or Facebook profile).
2. **Client**: The app or service that requests access to the user's resources (e.g., a travel booking app that wants access to your Google Calendar).
3. **Authorization Server**: The server responsible for authenticating the user and issuing access tokens (e.g., Google's OAuth server).
4. **Resource Server**: The server that hosts the user's data (e.g., Google, Facebook, Twitter). It verifies and grants access to the requested resources based on the token provided.

Flows / Grant Types:

- **Authorization Code**: Web apps redirect users to login at the provider.
- **Client Credentials**: Service-to-service communication.
- **Password Grant**: User provides username/password directly to client (less secure, rarely used).
- **Implicit Grant**: For single-page apps (now discouraged).

Where as

- JWT (JSON Web Token) is a token format, not a framework.
- JWT can be used inside OAuth2 as the access token.

Understand with example:

Scenario: A third-party travel app wants to access your Google Calendar to book trips.

1. Resource Owner: It is the user, the Google account holder. The user owns the calendar data.
2. Client: The third-party travel booking app that wants to read the user calendar to avoid scheduling conflicts.
3. Authorization Server: Google’s OAuth server. It asks the user to log in and consent: “Do you allow the travel app to read your calendar?”
4. Resource Server: Google Calendar API. Once the app has a valid access token, this server returns the user calendar data to the travel app.

Flow in Simple Terms

1. Travel app redirects us to Google login page (Authorization Server).
2. You log in and consent.
3. Google issues an access token to the travel app.
4. Travel app calls Google Calendar API (Resource Server) with the token.
5. Resource server validates the token and sends back your calendar data.

331 Spring 80: How do you implement Oauth2 in Spring?

Reference: #39 Spring Security | Google and Github Login using OAuth2
Code Link: SpringBoot-Oauth2-Demo/Oauth2-springdemo at master · VarshaDas/SpringBoot-Oauth2- Demo

332 Spring 81: What is role-based access?

Role-Based Access Control (RBAC) is a way to manage user permissions based on roles. Instead of assigning permissions directly to each user, we assign roles (like ADMIN, USER, MANAGER) to users, and each role has specific permissions.

How it Works

1. **Define Roles** For example: ADMIN, USER, EDITOR.
2. **Assign Permissions to Roles**: Each role has specific access

   rights. ADMIN, can create, read, update, delete

   everything.

   USER, can only read their own data.
3. **Assign Roles to Users** A user can have one or multiple roles.
4. **Check Access** When a user tries to access a resource, the system checks if the user's role allows that action.

Example in Real Life

- **Let's say in Banking App:**

  USER view account balance, transfer money.

  ADMIN approve loans, manage accounts, view all users.
- A user logs in; Spring Security populates their roles (authorities) in the SecurityContext.
- When the user tries to access /admin/dashboard, the system checks if they have ROLE_ADMIN.

Implementation in Spring Security

- Each role is represented as an authority (GrantedAuthority).
- We can secure endpoints using:

  Method-level security:

  @PreAuthorize("hasRole('ADMIN')") HTTP

  configuration:

```
http.authorizeHttpRequests()
.requestMatchers("/admin/**").hasRole("ADMIN")
.anyRequest().authenticated();
```

- Roles are usually prefixed with ROLE_ internally (ROLE_ADMIN).

Additional Questions:

1. How do you secure sensitive data in a Spring boot application that is accessed by multiple users with different roles?

Securing sensitive data in a Spring Boot application accessed by multiple users with different roles requires a multi-layered approach, primarily using Spring Security.

Authentication and Authorization with Spring Security:

- **User Authentication:** Implement robust authentication mechanisms. This can involve username/password authentication (using BCryptPasswordEncoder for secure password
  hashing), JWT-based authentication for stateless APIs, or integrating with external identity providers.
- **Role-Based Access Control (RBAC):** Define distinct roles (e.g., ADMIN, USER, GUEST) and assign them to users.
- **Method-Level Security:** Use annotations like @PreAuthorize, @PostAuthorize, and @Secured on service methods or controller endpoints to restrict access based on roles or custom expressions.
- **URL-Based Security:** Configure SecurityFilterChain to secure specific URL patterns based on roles.

333 Spring 82: Difference between Role and Grant?

Authority

- An authority represents a specific permission or action a user can perform.
- It is a more fine-grained concept.
- In Spring Security, it is represented by the interface GrantedAuthority.
- Examples:
  - READ_PRIVILEGE
  - WRITE_PRIVILEGE
  - DELETE_USER

2. Role

- A **role** is just a group of permissions.
- Roles are higher-level and bundle related permissions together.
- In Spring Security, roles are usually written as ROLE_<ROLE_NAME>. Examples:
  - ROLE_ADMIN might include READ_PRIVILEGE, WRITE_PRIVILEGE, DELETE_USER
  - ROLE_USER might include READ_PRIVILEGE only

Reference: Spring Security Roles: A Comprehensive Guide | Medium

334 Spring 83: What is method level security?

Method-level security in Spring Security is a way to protect individual methods in the code by specifying who can access them. Instead of securing only URLs or endpoints, we can secure specific service or controller methods.

We need to enable method-level security in the configuration class with

@EnableGlobalMethodSecurity(prePostEnabled = true) , Before (Spring Security 5.x ,From 6.x it is deprecated.

@EnableMethodSecurity from After Spring Security

6. We annotate methods with security annotations:

@PreAuthorize: Check before executing the

method. @PostAuthorize: Check after executing

the method.

@Secured: Basic role-based check before method execution.

@RolesAllowed: Standard JSR-250 role-based check.

335 Spring 84: What is Spring Boot Actuator, and why is it useful?

Spring Boot Actuator is a sub-project of Spring Boot that provides production-ready features to monitor and manage thw application.

- It adds a set of built-in endpoints that expose information about the application, like health, metrics, environment properties, and more.
- These endpoints helps to understand, monitor, and manage the app without writing extra code.

Key Features

1. **Health Checks**: Verify if the app and its dependencies (database, messaging queues, etc.) are working properly.
2. **Metrics**: Access information about memory usage, CPU, active threads, HTTP request statistics, etc.
3. **Environment & Config Info**: View active profiles, properties, and configuration settings.
4. **Logging**: Check and change logging levels at runtime.
5. **Tracing & Auditing**: Track HTTP requests and audit application events (with optional integrations).

How do you check if a running service is up in microservice

We can REST endpoint like /health or /actuator/health.

336 Spring 85: How do you enable and configure Actuator endpoints?

1. Add Dependency

Add the Actuator dependency in the pom.xml (Maven) or build.gradle (Gradle):
Maven:

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

2. Enable Endpoints

- By default, **only a few endpoints are exposed** (/actuator/health, /actuator/info).
- We can enable more endpoints in application.properties or application.yml.

```
management.endpoints.web.exposure.include=health,info,metrics,loggers

# Exclude certain endpoints if needed
management.endpoints.web.exposure.exclude=env,beans
```

337 Spring 86: List out some important Actuator endpoints?

1. /actuator/health

This endpoint checks the health of the application. It tells if the app is running fine, DB is connected, disk space is available, etc.

2. /actuator/info

This endpoint shows application info that we configure in application.properties (like version, description).

3. /actuator/metrics

The `/actuator/metrics` endpoint provides various application metrics. For example, to see JVM memory usage, we can query: localhost:8080/actuator/metrics/jvm.memory.used

4. /actuator/beans: Displays a complete list of all Spring beans in the application.

Indirect Question:

1. How do you check how many times each API endpoint was called, along with timings? We need to /actuator/metrics/http.server.requests

   For a particular API: actuator/metrics/http.server.requests?tag=uri:/hello-world&tag=method:GET Reference: A Comprehensive Guide to Spring Boot Actuator | by Pratik T | Medium

338 Spring 87: What is Micrometer?

Micrometer is basically the metrics collection library that Spring Boot (and many other frameworks) use under the hood. Think of it as the middleman between the code and monitoring systems like Prometheus.
Here is the thing:

- Spring Boot Actuator doesn't collect metrics all by itself.
- Instead, it uses Micrometer as the core library to record, manage, and expose metrics.
- Micrometer gives a vendor-neutral API , Means We can switch between different monitoring systems (like Prometheus, Datadog, New Relic, AWS CloudWatch, etc.)

339 Spring 88: How to create Custom Endpoint in Actuator?

We can create our own custom actuator endpoints using **@Endpoint** annotation on a class. Then we have to use **@ReadOperation**, **@WriteOperation**, or **@DeleteOperation** annotations on the methods to expose them as actuator endpoint bean.

```
@Component
@Endpoint(id = "custominfo")
public class CustomInfoEndpoint {

        @ReadOperation
            public Map<String, Object> customInfo()
        { Map<String, Object> data = new
        HashMap<>(); data.put("appName", "Payment-
        Service"); data.put("version", "1.0.3");
                data.put("activeUsers", 153);
                data.put("lastDeployed", "2025-08-20 14:30:00");
                return data;
        }
}
```

The above code creates a custom actuator endpoint "custominfo".

1. We should register this endpoint in application.properties

```
management.endpoints.web.exposure.include=health,info. customInfo
```

2. When we access it ` http://localhost:8080/actuator/custominfo` , It gives

```
{
    "appName": "Payment-Service",
  "version": "1.0.3",
    "activeUsers": 153,
    "lastDeployed": "2025-08-20 14:30:00"
}
```

Real time use case:

Example: If we want to see Business Metrics of an Application like Number of active users, current load, pending transactions.

Instead of DB checking in the DB or writing SQL, we can write /actuator/activeUsers.

```
@Component

@Endpoint(id = "activeUsers") // Exposed at /actuator/activeUsers
public class ActiveUsersEndpoint {

        @ReadOperation
            public Map<String, Object> activeUsers() {
        Map<String, Object> metrics = new HashMap<>();

                // Pretend fetching from service or cache
                int activeUsers = 125;
                int pendingTransactions = 48;
                double currentLoad = 0.67; // like CPU/memory load, etc.

                metrics.put("activeUsers", activeUsers);
        metrics.put("pendingTransactions", pendingTransactions);
        metrics.put("currentLoad", currentLoad);

                return metrics;
        }
}
```

http://localhost:8080/actuator/activeUsers This will give all the metrics

340 Spring 89: How to secure Actuator endpoints and why to secure?

Actuator exposes very sensitive information about the application:

- /actuator/health: can tell if the app is up/down (attackers can use this).
- /actuator/metrics: exposes internal performance metrics.
- /actuator/env or /actuator/configprops: might reveal, API keys, or DB URLs if not properly masked.
- Custom endpoints: could leak business data if left open.

If we don't secure them, anyone with the URL can hit those endpoints and can use the information. That is why in production we always secure actuator endpoints.

To secure them, we need to use SecurityFilterChain in Spring security

```
@Configuration
@EnableWebSecurity
public class ActuatorSecurityConfig {

        @Bean
            SecurityFilterChain securityFilterChain(HttpSecurity http) throws
        Exception { http
                        .authorizeHttpRequests(auth -> auth
                                .requestMatchers("/actuator/health",
                    "/actuator/info").permitAll()
                                .requestMatchers("/actuator/**").hasRole("ADMIN")
                                .anyRequest().authenticated()



                        )
                        .httpBasic();
                return http.build();
        }
}
```

Here:

- /actuator/health and /actuator/info are public (safe for probes).
- All other Actuator endpoints need **ADMIN role**.
- Authentication is via **basic auth**.

341 Spring 90: What is AOP?

AOP is a programming technique where we separate cross-cutting concerns from the main business logic.

Cross cutting concerns are nothing but

- Logging
- Security checks
- Transactions
- Caching
- Performance monitoring

If we write these inside every class/method, our code gets messy and repetitive.

So instead of writing the same logging or security code in every method, we write it in separately.

Our business logic stays in different files and cross cutting concerns stays in separate files, both can be managed separately

Example: Let's say we have a payment service where we want to log when Payment started and after payment ended.

Without AOP: (Note: Used System.out.print() just for example purpose, In prod we should not use it.)

```
class PaymentService {
            public void processPayment() {
        System.out.println("Starting payment..."); // logging
                // business logic
                System.out.println("Payment done."); // logging
        }
}
```

With AOP:

```
class PaymentService {
    public void processPayment() {
        // only business logic
    }


}

@Aspect
class LoggingAspect {
        @Before("execution(* PaymentService.*(..))")
            public void logBefore() {
        System.out.println("Starting payment...");
        }

        @After("execution(* PaymentService.*(..))")
            public void logAfter() {
        System.out.println("Payment done.");
        }
}
```

Here the logging concern is completely separate and placed in something called @Aspect (Will see next questions).

Reference: Spring AOP Explained: How to Implement Aspect-Oriented Programming in Your Spring Application | by Alex Klimenko | Medium

Indirect Questions:

1. When will you use the AOP?

When we want to separate cross-cutting concerns from the main business logic.

2. What are cross cutting concerns?

3. How you implement AOP in Spring boot application?

To implement AOP, start with including the *spring-boot-starter-aop* module in the application dependencies.

4. What is @EnableAspectJAutoProxy and when to use?

We use @EnableAspectJAutoProxy when we need to enable Spring's ability to automatically create proxies for the beans. It used to implement AOP features like @Transactional, @Secured, or the own custom @Aspect classes.

In practice, we often don't need to use it explicitly because many Spring modules (like Spring Boot, Spring Transaction Management) already auto-configure it for us.

In a Non-Boot, Java-based Configuration Application, if we are building a standard Spring application (not using Spring Boot) with @Configuration classes, and we want to use AOP, then we must add this annotation to one of the configuration classes.

Code Link: Spring-Samples/aop at main · CodingLyf-Fullstack/Spring-

Samples Run this code and hit ` GET /users/1`. You will see the

output

```
Around (Before): getUserById
```

```
Before method: getUserById | Args: [1]
After method: getUserById | Returned: CodingLyf
Around (After): getUserById
```

342 Spring 91: What are the core components of AOP?

Core components of AOP:

- Aspect.
- Join Point.
- Point Cut.
- Advice.
- Weaving.
- Proxy.

Aspect: It is basically a class where we define cross-cutting code. Ex: The class where our log statements exist.

```
class PaymentService {
    public void processPayment() {
        // only business logic
    }
}

@Aspect
class LoggingAspect {
        @Before("execution(* PaymentService.*(..))")
            public void logBefore() {
        System.out.println("Starting payment...");
        }

        @After("execution(* PaymentService.*(..))")
            public void logAfter() {
        System.out.println("Payment done.");
        }
}
```

Here, LoggingAspect is the **Aspect**.

Join Point: A specific point in the application, like a method execution or exception handling, where we can apply additional behaviour.

In the above example, processPayment() is a Join Point because we applying logging there in the method execution

Advice: The actual code inside the aspect that runs at certain points.

Types of advice:

1. **@Before**: Runs before the method execution.
2. **@After**: Runs after the method execution.
3. **@Around**: Wraps the method, running both before and after. This is the most powerful advice type. It surrounds a join point, such as a method invocation, and has the ability to prevent the actual method execution. It takes a ProceedingJoinPoint as a parameter, which is useful to
   execute the target method. By calling the proceed() method on the ProceedingJoinPoint, we can proceed with the original method execution.

4. **After Returning**: Runs only if the method completes successfully. We can use it for Auditing or logging after methods execution or resource clean up.
5. **After Throwing**: Runs only if the method throws an exception.

```
@Before("execution(* PaymentService.processPayment(..))")
    public void logBefore() {
    System.out.println("Payment starting...");
}
```

Here logBefore() is an advice which runs before the method execution.

Pointcut: A rule that decides where our extra code (aspect) should run.

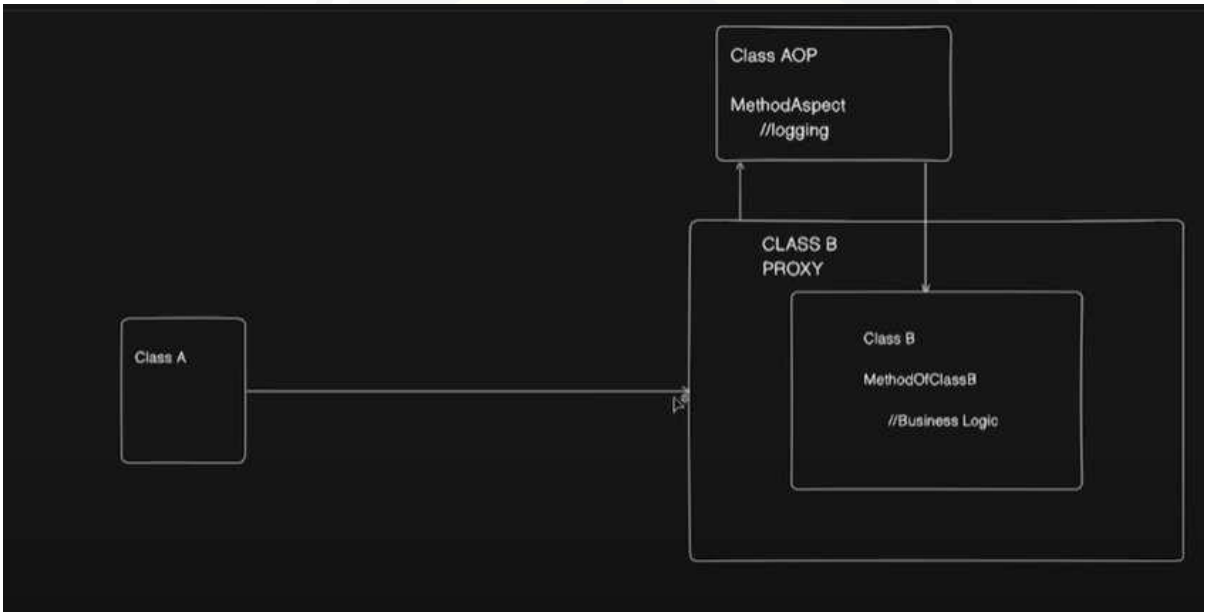
If we say execution(* PaymentService.*(..)), it means "apply this aspect to all methods in PaymentService."

Weaving: The process of applying aspects to the target objects. Spring does this dynamically at runtime using proxies.

Proxy: This is very important in Spring. All the cross-cutting concerns use Proxy. We will see it as separate question (Refer Question No 362 Spring 110. then go to next question)

343 Spring 92: How proxy works in AOP?

Let's understand with an example:



Here we have Class A, calling a method in classB and class AOP has a method which is responsible for logging.

1. Class A

- This is a normal class in the application.
- It calls a method of Class B (let's say methodOfClassB()).

2. Class B

- This contains the business logic. For example, it may process payments, calculate discounts, etc.
- Normally, Class A would just call Class B directly.
- But with AOP, it doesn't call Class B directly; instead, it calls a proxy of Class B.

3. Class B Proxy

- A proxy is like a wrapper around Class B.
- When Class A makes a call, it doesn't go straight to Class B.
  Instead, it goes to the proxy, which has the power to Decide whether to call the actual method in Class B.
    - Add extra behaviour before or after the call.
- In this case, that extra behaviour is logging.

4. Class AOP (Aspect-Oriented Programming)

- This defines the Aspect, it does the extra behaviour we want to apply.
- In the diagram, the Aspect is logging (MethodAspect > logging).
- The logging aspect will run before or after methodOfClassB() executes.

5. Flow of execution

1. Class A calls methodOfClassB() : but it hits the proxy instead of the real class.
2. The proxy checks if any aspect is attached (here: logging).
3. The logging aspect runs (e.g., "Method started" / "Method ended").
4. Then the proxy calls the real method in Class B to execute business logic.
5. Optionally, the aspect can also run after the method (e.g., log execution time, exceptions).

344 Spring 93: Which classes or methods in Spring cannot be declared as final, and why??

Spring creates proxies for certain features like @Transactional, @Async, @Configuration, and AOP in general.

Why final matters in Spring

Spring often uses CGLIB proxies to add extra behaviour to the classes at runtime. CGLIB works by subclassing the class and overriding methods.

If a class or method is final, it cannot be subclassed or overridden, so CGLIB cannot apply its proxy. That breaks things like:

- @Transactional
- @Async
- @Cacheable, @CacheEvict
- @Configuration (when calling @Bean methods

internally) Rules

1. Classes
    - We cannot make a class final if Spring needs to create a proxy for it (like a @Service or @Configuration with transactional beans).
    - We can make it final if no proxying is needed (simple component with no AOP features).
2. Methods
    - Any method annotated with @Transactional, @Async, @Cacheable, etc., cannot be final, because the proxy needs to override it.
    - Final methods are ignored by proxies.

```
@Service
@Transactional
public final class BankService { // final class breaks transactional proxy
        public void transferMoney() { … } // if method is final, proxy cannot intercept
}
```

```
@Service
@Transactional
public class BankService { // class is non-final
        public void transferMoney() { … } // method is non-final
}
```

345 Spring 94: What is Self-Invocation Problem?

In Spring AOP, aspects (like logging, transactions, security) are applied using proxies.

But if a method inside the same class calls another method of that class, the proxy is bypassed, and the aspect will not run.

That is the self-invocation problem.

Example

```
@Service
public class PaymentService {

        @Transactional    // AOP-based annotation
            public void processPayment() {
        System.out.println("Processing payment...");
                saveTransaction();     // calling another method inside same class
        }

        @Transactional
            public void saveTransaction() {
        System.out.println("Saving transaction...");
        }
}
```

Now when we call: paymentService.processPayment();

We might expect both processPayment() and saveTransaction() to run inside a transaction. But here is the catch:

- processPayment() is called through the proxy so Transaction applies.
- saveTransaction() is called directly inside the same class, not through the proxy so transaction does not apply.

So saveTransaction() runs without the @Transactional aspect.

Why does this happen?

- Spring AOP works by wrapping the bean with a proxy.
- When an external class calls paymentService.processPayment(), the proxy intercepts the call.
- But when one method in the class calls another (self-invocation), it goes straight to the real object (this.saveTransaction()), by skipping the proxy.

Ways to Fix It

1. **Move the method to another bean (class)**
    - So, the call goes through the proxy.

2. **Use AopContext to call through proxy**

   ((PaymentService) AopContext.currentProxy()).saveTransaction();

   Requires @EnableAspectJAutoProxy(exposeProxy = true) in config.

346 Spring 95: How to do Scheduling in Spring boot?

In Spring, scheduling basically means running tasks automatically at fixed times, intervals, or according to cron expressions. Spring makes this super simple with its scheduling support.

1. Enable Scheduling

First, we need to turn on scheduling in the Spring Boot app with @EnableScheduling.

```
@SpringBootApplication
@EnableScheduling  // Enables scheduling
public class MyApp {
          public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
         }
}
```

2. Use @Scheduled Annotation

Now we can mark methods that should run on a schedule. Spring will run them in the background automatically.

3. Scheduling Strategies

We have Scheduling Strategies like:

- fixedDelay,
- fixedRate
- and using cron expressions.

```
@Scheduled(cron = "0 0 9 * * ?") // every day at 9 AM
    public void morningTask() {
    System.out.println("Good Morning! It is 9 AM");
}
```

Indirect Question:

1. You have configured scheduled task only once but it is running twice. What might be the issue?

Multiple application instances: if the app is running on multiple servers or multiple Spring contexts, each instance runs its own scheduler.

Use Distributed cache lock like Redisson's RLock to ensure only one instance runs the task.

Multiple bean definitions: if the same @Scheduled method is present in multiple beans or contexts, it runs multiple times.

2. You have configured scheduled but it is not running. What might be the issue?

- @EnableScheduling is missing in the configuration.
- The scheduled method must be public and inside a Spring-managed bean (e.g., annotated with @Component, @Service, or defined as a @Bean). If it is private, protected, or in a class Spring doesn't manage, the scheduler won't detect or run it.
- The cron/fixedRate/fixedDelay expression is incorrect or misconfigured.

347 Spring 96: What is the difference between fixedDelay and fixedRate?

fixedDelay: This is used to give specific delay between the completion of one task execution and the start of the next.

- The next run starts only after the current run finishes + the delay time.
- Example: If a task takes 4 seconds and delay is 3 seconds: then the next run starts at 7 seconds.
- Use Case: Good for tasks where execution time may vary (like sending emails, file processing, DB cleanup) and we always want a gap between runs.

fixedRate: This property schedules the task to run at a consistent, fixed interval, regardless of how long the previous execution took.

- Example: If rate is 5 seconds, it runs at 0s, 5s, 10s, 15s… even if the previous task is still running.
- Use Case: Best for tasks that need **strict periodic execution**, like health checks, monitoring, refreshing cache, or fetching data at fixed times.

348 Spring 97: How can you make tasks run in parallel?

By default, Spring runs all @Scheduled tasks in a single thread. That means if one task takes time, the next one waits. To make them run in parallel, we need to give Spring a thread pool for scheduling.

We can use ThreadPoolTaskScheduler, a scheduler that supports various scheduling options and manages a thread pool for task execution. The ThreadPoolTaskScheduler uses a java.util.concurrent.ScheduledThreadPoolExecutor to manage a pool of threads.

We can configure the ThreadPoolTaskScheduler to suit the application's needs. Some common configuration options include:

- **Pool size:** The number of threads in the thread pool
- **Thread name prefix:** A prefix for thread names, useful for debugging
- **Wait for tasks to complete on shutdown:** A flag to indicate whether the scheduler should wait for scheduled tasks to complete before shutting down

```
@Configuration
@EnableScheduling
    public class
    SchedulerConfig { @Bean
        public TaskScheduler taskScheduler() {
                ThreadPoolTaskScheduler scheduler = new
        ThreadPoolTaskScheduler(); scheduler.setPoolSize(5);
        scheduler.setThreadNamePrefix("MyTaskScheduler-");
        scheduler.setWaitForTasksToCompleteOnShutdown(true);
                return scheduler;
        }
}
```

Tasks:

Without the thread pool: task2 will be delayed until task1 finishes. With the thread pool: both can run at the same time.

349 Spring 98: How can we handle scheduled task failures?

1. Try-Catch Inside the Task

The simplest way, just wrap the task logic with try-catch and log or recover gracefully

2. Use a Custom ErrorHandler

```
@Component
public class MyTasks {

        @Scheduled(fixedRate = 2000)
            public void task1() throws InterruptedException {
        System.out.println(Thread.currentThread().getName() + " running task1");
        Thread.sleep(4000); // simulate long work
        }

        @Scheduled(fixedRate = 2000)
        public void task2() {
                System.out.println(Thread.currentThread().getName() + " running task2");
        }
}
```

Spring provides an **ErrorHandler** interface to handle exceptions thrown out of @Scheduled methods. We can create a custom error handler by implementing this interface:

```
@Component
public class MyErrorHandler implements ErrorHandler {

        @Override


        public void handleError(Throwable t) {
                // Custom logic for handling errors
                System.err.println("Error encountered in scheduled task: " +
        t.getMessage());
        }
```

To use this custom error handler, we would need to set it in the task scheduler.

```
@Configuration
@EnableScheduling
public class SchedulerConfig {

        @Bean
            public ThreadPoolTaskScheduler taskScheduler() {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5);
        scheduler.setThreadNamePrefix("scheduler-");
        scheduler.setErrorHandler(new MyErrorHandler()); //Error Handler
        return scheduler;
        }
}
```

3. Add Retry Logic (Spring Retry)

If a task should retry automatically on failure, we can integrate Spring Retry (Refer Question No. 379 Spring 127 to know more about this)

350 Spring 99: How to run a task in the background in Spring boot?

In Spring Boot, if we want to run a task in the background (async execution), we typically use @Async.

Step 1: Enable Async Support

In the main application class (or any config class), enable async execution:

```
@Configuration
@EnableAsync
public class Config {


}
```

Step 2: Mark Methods with @Async

When we annotate a method with @Async, Spring will execute it in a separate thread (it won't block blocking the main thread)

```
@Service
public class Service {

        @Async
        public void process(String userEmail) {



            try {
                    Thread.sleep(5000); // simulate delay
        System.out.println("Delayed task");
                } catch
        (InterruptedException e) {
        Thread.currentThread().interrupt();
            }
        }
```

Real-world uses of @Async

- Sending emails, SMS, or push notifications.
- Triggering heavy reports.
- Calling external APIs that can run in parallel.
- File uploads, image processing, or background data

syncs. **Reference:** **Spring Boot @Async Controller with CompletableFuture Indirect Question**

1. CompletableFuture vs @Aysnc

CompletableFuture is Java 8 feature (java.util.concurrent.CompletableFuture) for asynchronous computation. It runs task in the background and handle results, exceptions, or chaining multiple tasks.

Key Points:

- It is Fully programmatic; we create, execute, and combine futures yourself.
- It supports chaining, combining, and callback methods (thenApply, thenAccept, exceptionally).

- It Works in any Java class, not just Spring beans.

@Async is a Spring annotation that enables a method to be executed in a separate thread, managed by Spring's task execution infrastructure.

It provides a convenient way to mark a method for asynchronous execution and here we don't need manually manage the threads**.**

2. How Spring manages the thread pool in @Async task?

Spring manages @Async tasks using a **TaskExecutor** (default is SimpleAsyncTaskExecutor), which handles a thread pool to run methods asynchronously. We can customize the pool size and behaviour by defining a @Bean of type Executor. Spring submits each @Async method call to this executor, allowing it to run in a separate thread without blocking the main thread.

```
@Bean(name = "taskExecutor")
public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new
    ThreadPoolTaskExecutor(); executor.setCorePoolSize(5); //
    Minimum threads executor.setMaxPoolSize(10);    // Maximum
    threads executor.setQueueCapacity(25);    // Queue size
    executor.setThreadNamePrefix("AsyncThread-");


        executor.initialize();
        return executor;
}
```

Reference: How Spring Boot Configures Thread Pools | Medium

351 Spring 100: How @Async works behind the scenes?

When we put @Async on a method in Spring Boot, here is what actually happens behind the scenes:

1. Spring creates a proxy (Refer Question no 362 Spring 110 to know more about the proxy**)**

- When the class has a method annotated with @Async, Spring doesn't call the method directly.
- Instead, it creates a proxy object (using JDK dynamic proxies or CGLIB, depending on the class).
- The proxy intercepts method calls and decides how to execute them.

2. Intercepts the call

- When we call a method, instead of running it on the caller's thread, the proxy hands the task over to a **TaskExecutor** (basically a thread pool).
- By default, spring uses SimpleAsyncTaskExecutor if no executor is provided, but in real projects we usually configure the own ThreadPoolTaskExecutor.

3. Method execution happens on a different thread

- The proxy submits the method as a task (Runnable or Callable) to the executor.
- The original caller thread **doesn't wait** for the task to finish (unless we return Future/CompletableFuture).
- This is how it becomes asynchronous.

4. Return type decides behavior

- void: fire and forget. Caller has no clue if the task succeeded/failed.
- Future / CompletableFuture : Caller can track progress, get results, or handle

exceptions. Point to remember:

Due to the proxy-based nature of Spring's AOP framework, if we call an @Async method from another method within the same class, it will not actually execute asynchronously. This is because the call won't go through the proxy but it is just a direct method call. Always be aware of

this limitation. (Self-invocation problem Refer Question 346 No. Spring 94)

```
@Service
public class MyAsyncService {

        @Async
            public void asyncVoidTask() throws InterruptedException {
        System.out.println(Thread.currentThread().getName() + " running void task");
        Thread.sleep(1000);
                System.out.println(Thread.currentThread().getName() + " finished void
        task");
        }


        @Async
            public CompletableFuture<String> asyncFutureTask() throws
        InterruptedException { System.out.println(Thread.currentThread().getName() + "
        running future task"); Thread.sleep(1000);
                System.out.println(Thread.currentThread().getName() + " finished future
        task");
                return CompletableFuture.completedFuture("Task Done");
        }
```

In this example, first method is not returning anything whereas second method is returning CompletableFuture

```
@GetMapping("/runAsync")
    public String run() throws Exception {
    service.asyncVoidTask(); // fire-and-forget
        CompletableFuture<String> result = service.asyncFutureTask(); // trackable
    System.out.println("Controller thread: " + Thread.currentThread().getName());
    return "Async tasks triggered";
}
```

352 Spring 101: Can you run Database transactional tasks in Async?

It takes too long to write here. I would suggest you to go through the link, worth reading.

Link: Handling Asynchronous Execution with Transactions in Spring: A Common Pitfall and How to Solve It - DEV Community

353 Spring 102: How cache works in spring boot?

Caching in Spring Boot is a mechanism to improve application performance by storing frequently accessed data in a temporary, faster-access storage location, such as in-memory or a dedicated cache server. This reduces the need to repeatedly fetch data from slower sources like databases or external APIs.

How it works:

Enabling Caching:

Caching is enabled in a Spring Boot application by adding the **@EnableCaching** annotation to a configuration class or the main application class. This activates Spring's caching infrastructure.

Cache Abstraction:

Spring provides a cache abstraction layer that allows developers to use various cache providers (e.g., Caffeine, Ehcache, Redis) without changing the core application logic.

354 Spring 103: How @cachable works?

@Cacheable is a Spring annotation used to **store the result of a method call in a cache**, so the next time we call that method with the same arguments, Spring will fetch the result directly from the cache instead of executing the method again.

Basically, it saves computation time and database calls.

```
@Service
public class ProductService {

        @Cacheable(value = "products", key = "'allProducts'")
        public Product getProductById(Long id) {
                // Imagine this is a slow DB call
        System.out.println("Fetching from DB for id: " + id);
        return new Product(id, "Laptop");
        }
}
```

value = "products" is the cache name where Spring will store and look up the cached

data key = "'allProducts'" means Spring will always use the string "allProducts" as the

cache key

355 Spring 104: What is the importance of the key while caching?

- Keys are critical in caching because they uniquely identify the cached data.
- When we call a cached method, Spring creates a key based on the method parameters (by default all arguments are considered).
- If two calls have the same key, Spring returns the cached result instead of executing the method.
- Without proper keys, data could be incorrectly reused or duplicated. We can customize keys using SpEL (@Cacheable(key = "#id")) to cache only the values we need.
- Good key strategy ensures accurate retrieval, avoids collisions, and optimizes memory usage for the cache.

356 Spring 105 Can you cache based on the condition?

Yes, Spring allows conditional caching with the condition and unless attributes.

- **Condition**: Evaluated before method execution, decides whether to cache.
- **Unless**: Evaluated after execution, decides whether to store the result.

```
@Cacheable(value = "products", key = "#id", condition = "#id > 10")
public Product getProduct(Long id) {

}
```

Here, caching happens only if id > 10.

```
@Cacheable(value = "products", unless = "#result == null")
public Product getProduct(Long id) {

}
```

This prevents caching null results.

357 Spring 106: How to update the cache?

When we want to refresh the cache with a new value after updating data, use @CachePut. Unlike @Cacheable, it always executes the method and updates the cache with the new

result. Example:

```
@CachePut(value = "products", key = "#product.id")
public Product updateProduct(Product product) {
        // Save to DB
        return productRepository.save(product);
}
```

Whenever this method runs, the cache is updated with the latest product, ensuring consistency between DB and cache.

358 Spring 107: How to delete the cache?

```
@CacheEvict(value = "products", key = "#id")
    public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}
```

To remove data from the cache, use @CacheEvict. This is important when cached data is stale or invalid. This removes the cache entry for the given product

ID. We can also clear the entire cache:

```
@CacheEvict(value = "products", allEntries = true)
public void clearCache() {}
```

359 Spring 108: How cache work behind the scenes?

When Spring Boot starts, it creates a CacheManager bean. @Cacheable methods are wrapped in a proxy. When the method is called:

1. Proxy checks the cache using the generated key.
2. If a value is found, it returns it immediately (method body is skipped).
3. If not found, the method executes, result is stored in the cache, and then returned. The actual storage is managed by the configured cache provider (ConcurrentMap, Redis, EhCache, etc.).

The default CacheManager is ConcurrentMapCacheManager

360 Spring 109: How CocurrentMapCacheManager works?

ConcurrentMapCacheManager is Spring's default cache manager. It uses Java's in-memory **ConcurrentHashMap** to store cached data. It is lightweight, thread-safe, and requires no external configuration. Each cache is basically a ConcurrentMap.

```
@Bean
public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager("products", "users");
}
```

It is great for development and testing, but not recommended for production because the cache is limited to the JVM, not distributed, and is lost when the app restarts.

Point to remember: For production, Redis or Caffeine is usually preferred.

361 Spring 110: What is proxy and how does it work Spring?

A proxy is like a middleman (or wrapper) that Spring creates around the actual object (the target).

When we call a method, we are not calling the real object directly but we are calling the

proxy. The proxy can do extra work before or after calling the real method.

Point to remember: This is how Spring applies proxies like AOP, transactions, security, or caching without writing extra code in our class.

Let's see an example:

We have an interface called 'Service' and we created Payment Service class implementing the 'Service'.

```
interface Service {
        void processPayment();
}

class PaymentService implements Service {
            public void processPayment() {
        System.out.println("Processing payment...");
        }
}
```

When we call new PaymentService().processPayment(); Spring creates a proxy for PaymentService like this,

```
//Proxy class
  }

  @Override
  public void processPayment() {
          System.out.println("Before: Logging before
     payment..."); target.processPayment(); // call the real
     object System.out.println("After: Logging after
     payment...");
  }
```

And this is how Spring calls:

```
Service service = new PaymentServiceProxy(new PaymentService());
service.processPayment();
```

In our example, `PaymentServiceProxy` has been created and it will pass the real service to the proxy. Now Proxy can do extra code before or after the original method (processPayment()) execution.

There are two types of proxies in Spring:

1. JDK Dynamic Proxies (Requires an Interface)

If the target class implements an interface, Spring will create a dynamic proxy that implements the same interface.

2. CGLIB Proxies (No Interface Required)

If there is no interface, Spring generates a subclass proxy using CGLIB (Code Generation Library).

Reference: Understanding the Proxy Pattern in Spring Boot with Practical Insights | by Davoud Badamchi | Medium (It contains how proxies work in AOP, @Transactional and Caching).

362 Spring 111: How a CGLIB Proxy works with @Bean?

When we mark a class with @Configuration, Spring doesn't just use it as-is. Behind the scenes, Spring creates a CGLIB-enhanced subclass of that class. This proxy ensures that calls to @Bean methods inside the same configuration class still return the same singleton bean from the container, not a new object.

Example:

```
@Configuration
public class AppConfig {

        @Bean
        public UserService userService() {
                return new UserService(orderService()); // direct method call
        }

        @Bean


        public OrderService orderService() {
                return new OrderService();
        }
}
```

At first glance, we might think:

- Every time userService() calls orderService(), a new OrderService is created.
- But Spring wants OrderService to be a singleton by default

What actually happens

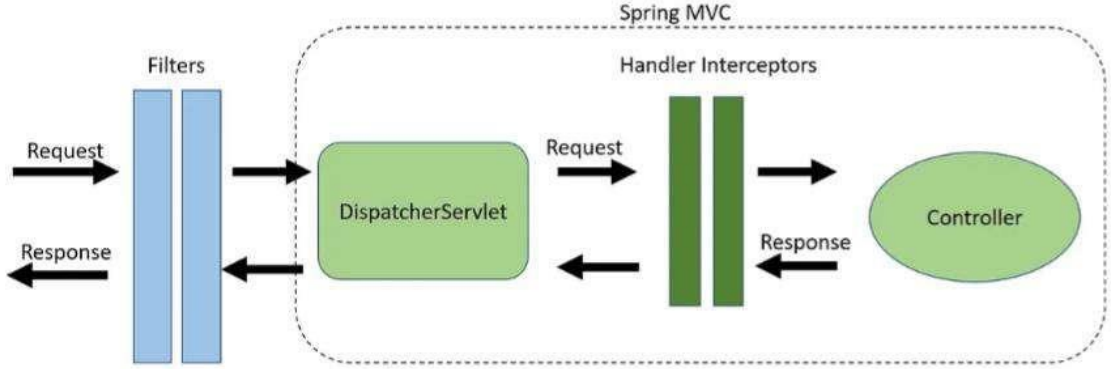
Spring uses **CGLIB** to generate a subclass of AppConfig.

- When userService() calls orderService(), it doesn't directly execute the method.
- Instead, the CGLIB proxy intercepts the call, checks the container, and returns the already cached singleton OrderService bean.

So even though it looks like a normal method call, it is actually routed through Spring's bean factory.

363 Spring 112: What is filter in Spring boot and how it will used?

In Spring Boot, a Filter is a component that intercepts and processes HTTP requests and responses at the web container level, before they reach the Spring DispatcherServlet and after the DispatcherServlet has handled them. Filters are part of the Java Servlet API and are used to implement cross-cutting concerns in web applications.



Common use cases include:

- **Authentication and Authorization:** Filters can verify user credentials and permissions before allowing access to resources (e.g., Spring Security utilizes filters).
- **Logging and Auditing:** Recording request details, response status, and other relevant information for monitoring and debugging.
- **Request/Response Modification:** Altering request headers, parameters, or response content (e.g., compression, content negotiation).
- **Data Validation:** Pre-processing requests to validate input data before it reaches the application logic.
- **CORS (Cross-Origin Resource Sharing):** Managing access control for requests from different origins.

364 Spring 113: How do you create a custom filter? Steps to create the filter

- Implement the jakarta.servlet.Filter interface (or javax.servlet.Filter for older versions).
- Override the doFilter() method, which contains the logic for processing the request and response.

Using @Component and @Order: Annotate the filter class with @Component to make it a Spring bean. Use @Order to define the order of execution if we have multiple filters.

```
@Component
public class MyCustomFilter implements Filter {

        @Override
        public  doFilter(ServletRequest request, ServletResponse response, FilterChain
chain)
                throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        System.out.println("Incoming request URL: " + req.getRequestURI());

        // Continue the request
        chain.doFilter(request, response);

        System.out.println("Outgoing response processed");
    }
}
```

- Use chain.doFilter(request, response) to pass the request to the next filter in the chain or to the target servlet.

Registering the Filter:

Point to remember: If we annotate the filter with @Component, Spring Boot automatically adds it to the filter chain.

Use cases of filters.

- **Authentication & Authorization**: Check JWT tokens, session IDs, or API keys.
- **Logging & Auditing**: Log request URLs, headers, or response times.
- **Compression or Transformation**: Compress responses, modify headers, or transform JSON.

- **Rate Limiting**: Block excessive requests.
- **CORS Handling**: Add headers for cross-origin requests.

Indirect Question:

1. How can you implement JWT token validation for all APIs without modifying each controller?

Use a custom filter (e.g., extending OncePerRequestFilter) and register it in the Spring Security filter chain.

2. If you want to log every incoming API call and its response time, which component would you use? Implement a Spring Filter or HandlerInterceptor (Refer Question no 100 for Interceptor)

3. How does Spring Boot decide the order in which filters are executed? Using @Order annotation

365 Spring 114: What is interceptor and its uses?

Interceptors in Spring Boot are components within the Spring Web MVC framework that are used to intercept and process HTTP requests and responses. They provide a mechanism to execute custom logic before a request reaches a controller, after the controller has processed the request but before the response is sent.

Interceptors are generally used for operations such as security, authentication, authorization, logging and monitoring

Both Filters and Interceptors serving the same purpose. But what is the difference between them (Refer Question No 368, Spring 116)

Key Points About Interceptors

1. Interceptors are tied to Spring MVC and work with HandlerMapping and HandlerExecutionChain.
2. They can pre-process requests, post-process responses, and handle after-completion logic used for cleanup tasks.
3. Unlike filters, interceptors have access to the controller's handler method and can examine method arguments, model data, and view.

366 Spring 115: How to create an Interceptor?

Step 1: Create a component by Implementing the HandlerInterceptor

Interface Spring provides the HandlerInterceptor interface. It has **three main**

methods:

```
@Component
public class MyInterceptor implements HandlerInterceptor {

		// Runs before the controller
	method @Override


	public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler)
					throws Exception {
		System.out.println("Pre-handle logic: Incoming request URL -> " +
request.getRequestURI());
				// Return true to continue to the controller, false to block
		the request
				return true;
		}

		// Runs after the controller method but before view
	rendering @Override
	public void postHandle(HttpServletRequest request, HttpServletResponse response,
Object handler,
					ModelAndView modelAndView) throws Exception {
				System.out.println("Post-handle logic: Controller executed,
		preparing
response");
		}

		// Runs after the complete request is finished and view
	rendered @Override
	public void afterCompletion(HttpServletRequest request, HttpServletResponse
response, Object handler, Exception ex)
					throws Exception {
				System.out.println("After completion logic: Request
		finished");
				if (ex != null) {
					System.out.println("Exception occurred: " +
```

- **preHandle():**

  When an interceptor is implemented, any request before reaching the desired controller will be intercepted by this interceptor and some pre-processing can be performed like logging, authentication, redirection, etc.

  Must return true to continue to the controller; false stops the request.

- **postHandle()**:

  This method is executed after the request is served but just before the response is sent back to the client. It intercepts the request in the final stage, giving us a chance to make any final trivial adjustments.

- **afterCompletion():**

  This method is executed after the request and response mechanism is completed. This method can turn out to be very useful in cleaning up the resources once the request is served completely.

Step 2: Interceptors need to be registered with Spring MVC using WebMvcConfigurer:s

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

        private final MyInterceptor myInterceptor;



        public WebConfig(MyInterceptor myInterceptor) {
                this.myInterceptor = myInterceptor;
        }

        @Override
            public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(myInterceptor)
                                .addPathPatterns("/**"); // Apply to all endpoints
        }
}
```

Here, we have registered myInterceptor using `InterceptorRegistry`.

367 Spring 116: Difference between Filter and Interceptor?

Feature	Filter	Interceptor
API Level	Part of the Servlet API (javax.servlet.Filter)	Part of Spring MVC (HandlerInterceptor)
Scope	Works on all HTTP requests, even outside Spring MVC (e.g., static resources, JSPs)	Works only on requests handled by Spring MVC controllers
Execution Point	Pre-processing before the request enters the servlet and post-processing after response leaves the servlet	Pre-processing before controller, post-processing after controller but before view, and after completion after view rendered
Access to Handler	Does not know about controllers or handler methods	Has access to handler (controller method) and ModelAndView
Use Cases	Cross-cutting concerns like logging, authentication, compression, CORS, security	Controller-specific tasks like logging request/response, authorization, metrics, exception handling
Order of Execution	Controlled by @Order or FilterRegistrationBean	Controlled by order of interceptor registration in WebMvcConfigurer
Dependency on Spring	Independent of Spring; works at servlet container level	Dependent on Spring MVC; integrated with Spring lifecycle
Example	JWT token validation before reaching any controller	Measuring execution time of controller method

368 Spring 117: Have you written any custom Annotation, if yes how you have written it?

Annotations are created by using '@' sign, followed by the keyword interface, and followed by annotation name as shown in the below example.

```
public @interface EnableRestCallLogs{

}
```

Generally, each Annotation can contain meta-annotations. Both of these are not mandatory. If not applied, default values will be considered.

- @Retention
- @Target.

Retention policy(@Retention)

It specifies the Retention policy. A retention policy determines at what time annotation should be discarded. Mainly java defined 3 types of retention policies.

Those are **SOURCE, CLASS** and **RUNTIME**.

- The **SOURCE** policy will be retained only with source code, and discarded during compile time. It effected on before compile the source code. Besides that very common java annotations
  like @Override, @Deprecated, @FunctionalInterface,@SuppressWarnings also uses retention policy as **SOURCE**.
- The retention policy **CLASS** will be retained until compiling the code, and discarded during runtime.
- Retention policy **RUNTIME** will be available to the JVM through runtime. If we not specify the retention policy the default retention policy type is **CLASS**.

Target(@Target)

- @Target annotation definition defines where to apply the annotation .
- It takes ElementType enumeration as its only argument.
- The ElementType enumeration is a constant which specifies the type of the program element declaration (class, interface, constructor, etc.) to which the annotation can be applied.

  Type -> Class, interface or

  enumeration Field -> Field

  Method -> Method

  Constructor -> Constructor

@Target(ElementType.METHOD , @Target({ElementType.***METHOD***, ElementType.***TYPE***})

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;



// Custom annotation
@Retention(RetentionPolicy.RUNTIME) // Annotation should be available at runtime
@Target(ElementType.METHOD)      // Can be applied only on methods
public @interface LogExecutionTime {
}
```

369 Spring 118: @Configuration vs @Component?

1. @Component

- @Component is a generic stereotype annotation in Spring.
- It tells Spring, *"this class is a Spring bean, please register it in the application context."*
- We usually use it for service classes, repositories, controllers, or any bean we want Spring to manage.
- Methods inside a @Component class are not special by default means if we add a @Bean method here, Spring will still create a bean, but it won't guarantee singleton behaviour when one @Bean method calls another.

```
@Component
public class AppConfig {

        @Bean
        public UserService userService() {
                return new UserService(orderService()); // creates new OrderService each
        time
        }

        @Bean
        public OrderService orderService() {
                return new OrderService();
        }
```

Here, userService() will get a **new instance** of OrderService() instead of the Spring-managed one.

@Configuration

- @Configuration is a specialized form of @Component.
- It not only registers the class as a bean but also enables full configuration features.
- Specifically, it uses CGLIB proxies to intercept calls between @Bean methods and ensure singleton semantics which means even if one @Bean method calls another, we still get the same Spring-managed bean instance.

```
@Configuration
public class AppConfig {

        @Bean
        public UserService userService() {
                return new UserService(orderService()); // gets the Spring-managed
        singleton



        @Bean
        public OrderService orderService() {
                return new OrderService();
        }
}
```

Here, userService() will get the **same OrderService instance** from the

container. Point to remember:

@Component: No proxying means calling one @Bean method from another creates a new object.

@Configuration: Proxying enabled means Spring ensures that all @Bean methods return proper container-managed singletons.

370 Spring 119: What are best techniques to handle exceptions in Spring?

1.**Try-Catch Blocks (Local Handling)**

- The simplest approach is to use **try-catch** within a method.
- Pros: Quick for simple cases.
- Cons: Leads to **duplicate code** if many methods need similar handling.

Example:

```
try {
    userService.deleteUser(id);
} catch (UserNotFoundException e) {
    // handle exception locally
}
```

2. @ExceptionHandler (Controller Level)

- It is used to handle exceptions **within a specific controller**.
- Annotate a method with @ExceptionHandler to intercept exceptions.
- We can return custom response objects for better API consistency.

Example:

```
@RestController
public class UserController {

        @ExceptionHandler(UserNotFoundException.class)
        public ResponseEntity<String> handleUserNotFound(UserNotFoundException ex) {
                return
        ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
        }
```

3. @ControllerAdvice (Global Exception Handling)

- It is useful for centralized exception handling across all controllers.
- Annotate a class with @ControllerAdvice and define @ExceptionHandler methods.
- It is best for REST APIs to send uniform error responses.

```
@ControllerAdvice
public class GlobalExceptionHandler {

        @ExceptionHandler(UserNotFoundException.class)
            public ResponseEntity<ErrorResponse>
        handleUserNotFound(UserNotFoundException ex) { ErrorResponse error = new
        ErrorResponse("USER_NOT_FOUND", ex.getMessage()); return new
        ResponseEntity<>(error, HttpStatus.NOT_FOUND);
        }

        @ExceptionHandler(Exception.class)
        public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
                ErrorResponse error = new ErrorResponse("INTERNAL_ERROR", "Something
        went
wrong");
                return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
```

4. ResponseEntityExceptionHandler

- Spring provides a base class ResponseEntityExceptionHandler with default handling for common exceptions.

- We can extend it in the @ControllerAdvice to customize default exception handling.
- Useful for exceptions like MethodArgumentNotValidException (validation errors).

```
@RestControllerAdvice
public class CustomResponseEntityExceptionHandler extends ResponseEntityExceptionHandler
{

	@Override
	protected ResponseEntity<Object> handleMethodArgumentNotValid(
			MethodArgumentNotValidException ex,
			HttpHeaders headers,
			HttpStatusCode status,
			WebRequest request) {

		Map<String, Object> body = new
	  LinkedHashMap<>(); body.put("timestamp",
	  LocalDateTime.now()); body.put("status",
	  status.value());

	  List<String> errors = ex.getBindingResult()
	                          .getFieldErrors()
	                          .stream()
	                          .map(x -> x.getField() + ": " + x.getDefaultMessage())
	                          .collect(Collectors.toList());

	  body.put("errors", errors);


		   System.out.println(body);
		   return new ResponseEntity<>(body, status);
		}

}
```

5. Custom Exceptions

- We can define our own exception classes to make the code readable and meaningful.
- Combine with @ExceptionHandler or @ControllerAdvice.

```
public class UserNotFoundException extends RuntimeException {
	public UserNotFoundException(String message) {
		super(message);
	}
}
```

Indirect Questions

1. Define @ExceptionHandler or @ControllerAdvice

@ExceptionHandler handles exceptions inside one controller, while @ControllerAdvice applies globally across controllers.

2. If I define @ExceptionHandler for a custom exception in a controller and also in a @ControllerAdvice, which one gets executed and why

If both exist, the local @ExceptionHandler in the controller wins, because Spring gives priority to controller-scoped handlers.

3. If you don't define any @ControllerAdvice or @ExceptionHandler, what kind of response does Spring Boot return for an exception thrown in a REST API?

Without custom handlers, Spring Boot returns a default JSON error response (status, error, message, timestamp, path) using **BasicErrorController.**

4. If multiple @ControllerAdvice classes handle the same exception type, how does Spring decide which one to use?

If multiple @ControllerAdvice match, Spring picks the most specific one (by exception hierarchy or @Order annotation).

5. If a service method annotated with @Async throws an exception, will your @ControllerAdvice catch it? If not, how would you handle it

Exceptions from @Async methods don't reach @ControllerAdvice. We need to handle them via **AsyncUncaughtExceptionHandler**

```
public class CustomAsyncExceptionHandler implements AsyncUncaughtExceptionHandler
{ @Override
public void handleUncaughtException(Throwable ex, Method method, Object...
params) {
                    System.err.println(
                                        "Exception occurred in async method:


                                        + " with message: " +
                    ex.getMessage());
                    // Log the exception, send notifications, etc.
        }
```

6. How do you handle Validation Errors.

Validation errors (from @Valid) are caught as **MethodArgumentNotValidException**. Typically, we handle them in @ControllerAdvice and return structured error messages (field, rejected value, reason).

```
@ControllerAdvice
    public class ValidationHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
        handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
                    .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
            return ResponseEntity.badRequest().body(errors);
        }
}
```

7. How can you restrict @ControllerAdvice only to controller or exceptions from specific packages

We can restrict @ControllerAdvice with basePackages, basePackageClasses, or annotations so it only applies to specific controllers or packages.

```
//Applies only to controllers in package
"com.example.api" @ControllerAdvice(basePackages =
"com.example.api") public class ApiExceptionHandler {
@ExceptionHandler(Exception.class)
  public ResponseEntity<String> handle(Exception ex) {
          return ResponseEntity.internalServerError().body("API error: " +
     ex.getMessage());
  }
```

8. What is ProblemDetails?

ProblemDetails is a standard format that is defined in RFC 7807 for describing errors and exceptions in HTTP APIs. The format includes a set of predefined fields, such as the error type, error code, error message, and additional details about the error.

```
@RestControllerAdvice
public class CustomExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(value = { CustomException.class })
  protected ResponseEntity<Object> handleCustomException(CustomException ex, WebRequest
request) {
        ProblemDetails problemDetails = new ProblemDetails();
    problemDetails.setStatus(HttpStatus.BAD_REQUEST);
    problemDetails.setTitle("Custom Exception");


        problemDetails.setDetail(ex.getMessage());
    return handleExceptionInternal(ex, problemDetails, new HttpHeaders(),
HttpStatus.BAD_REQUEST, request);
    }
```

Reference: Using ProblemDetails in Spring Boot | by Duncan Roydon | Medium

9. Mention 5 Spring Exceptions that you came across.

- **NoSuchBeanDefinitionException**: Thrown when we ask Spring for a bean that doesn't exist inthe ApplicationContext.
- **NoUniqueBeanDefinitionException**:Thrown when multiple beans match a type and Spring can't decide which one to inject.
- **BeanCurrentlyInCreationException**: Happens during circular dependency issues.
- **BeanInstantiationException**: Raised when Spring fails to create a bean instance, often due to missing constructor or abstract class.
- **ApplicationContextException**: A generic runtime exception for problems starting or using the Spring ApplicationContext.

371 Spring 120: How Database indexing improves the performance?

An index in a database is a data structure (usually a B-tree or hash table) that allows faster retrieval of rows from a table. It works like an index in a book, rather than scanning the entire book to find a
topic, we go to the index, find the page number, and jump directly to

it. SELECT * FROM users WHERE email = 'abc@example.com';

Without Index:

The database does a full table scan.

It checks every single row, one by one, to see if email

matches. This is very slow for large datasets.

With Index:

The database goes to the index on email, finds the location of the matching row(s) quickly, and jumps directly there.

It is like going straight to the correct shelf in a library rather than checking every

book. This is much faster, often reducing lookup time from seconds to

milliseconds.

```
CREATE INDEX idx_email ON users(email);
```

Please note: If we create too many indexes, it may hurt write performance and increase storage usage.

Reference: How Does Indexing Work | Atlassian

372 Spring 121: What is externalization and how would you achieve it?

When we build applications, one of the fundamental principles is separation of configuration from code. We don't want to hardcode things like database URLs, API keys, file paths, or feature toggles directly into the Java classes. Why? Because those values often change between environments like
development, testing, staging, and production.

Externalization is the practice of moving configuration values outside the application code so they can be changed without modifying or recompiling the application. Instead of editing Java classes, we adjust a configuration file, an environment variable, or even a command-line argument.

How Spring Achieves Externalization

1. Using Application Properties/YAML Files: Configurations placed in application.properties or application.yml

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
```

2. Using Command-Line Arguments: we can override properties at runtime without touching the code.

```
java -jar myapp.jar --server.port=9090
```

3. Using Environment Variables: It is useful in cloud or containerized deployments. Example: setting SPRING_DATASOURCE_PASSWORD in Docker or Kubernetes.

How Spring read external configurations?

1. Using @Value annotation

The @Value annotation in Spring Boot is used to inject values into fields, method parameters, or constructors of Spring-managed beans.

```
@Value("${db.url}")
private String dbUrl;
```

Main disadvantage of @value is , It is not type-safe; wrong values can cause runtime

errors. To overcome this, we can use @ConfigurationProperties.

2. @ConfigurationProperties

Instead of injecting individual values like @Value, it binds a whole set of related properties into a strongly typed POJO.

```
@ConfigurationProperties(prefix = "app")
public class AppProperties {


        private String name;
        private int timeout;

        // getters and setters
}
```

application.properties:

```
app:
    name: MyApplication
  timeout: 5000
```

Here, name and timeout are mapped to the fields of AppProperties file.

373 Spring 122: Auditing in Spring JPA?

When we build applications that store data, we often need to keep track of who created a record, when it was created, and who last modified it. This is where *auditing* comes in. Instead of manually setting these fields everywhere in the code, Spring Data JPA provides a neat way to handle it automatically.

Spring Data JPA auditing hooks into the entity lifecycle. When we save or update an entity, Spring automatically fills in fields like:

- **CreatedDate**: when the entity was created
- **LastModifiedDate**: when the entity was updated
- **CreatedBy**: who created the entity
- **LastModifiedBy**: who last updated it

Step 1: Enable JPA Auditing

```
@SpringBootApplication
@EnableJpaAuditing // enables auditing support
public class AuditingApp {
            public static void main(String[] args) {
        SpringApplication.run(AuditingApp.class, args);
        }
}
```

Step 2: Create the Base Audit Class

```
@EntityListeners(AuditingEntityListener.class) // required for auditing to kick in
@Getter
@Setter
public abstract class Auditable {

        @CreatedBy
    @Column(updatable = false)
    private String createdBy;



        @CreatedDate
    @Column(updatable = false)
        private LocalDateTime createdDate;

        @LastModifiedBy
        private String lastModifiedBy;

        @LastModifiedDate
        private LocalDateTime lastModifiedDate;
}
```

Step 3: Extend Audit Fields in Entities

Any entity that extends this base class will automatically get audit fields populated.

```
@Entity
public class Product extends Auditable {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        private String name;
}
```

Step 4: Provide an AuditorAware Bean

Spring needs to know who the current user is. We do that by providing a bean of type AuditorAware<T>.

Indirect question: How to get current user logged in Spring?

```
@Configuration
public class AuditConfig {

        @Bean
        public AuditorAware<String> auditorAware() {
                // In real apps, fetch from SecurityContextHolder
                return () -> Optional.of("system_user");
        }
}
```

Step 5: Save

Whenever we save or update a Product, Spring will automatically fill in audit fields.

```
Product product = new Product();
product.setName("Laptop");
productRepository.save(product);

// createdBy = "system_user"


// createdDate = current timestamp
// lastModifiedBy = "system_user"
// lastModifiedDate = current timestamp
```

@EnableJpaAuditing: This annotation is placed on a Spring configuration class (e.g., the main application class).Its purpose is to activate the JPA auditing features provided by Spring Data.

When this annotation is present, Spring Data JPA automatically detects and processes entities that are configured for auditing.

@EntityListeners(AuditingEntityListener.class): This annotation is placed on an entity class (a class annotated with @Entity). It registers the AuditingEntityListener as an entity listener for that specific entity.

The **AuditingEntityListener** is a Spring Data JPA class that listens for JPA lifecycle events (like pre-persist and pre-update).

When these events occur, the AuditingEntityListener automatically populates the auditing fields within the entity, such as: @CreatedDate, @LastModifiedDate, @CreatedBy

Reference: Database Auditing in Spring boot with spring security context and spring data JPA | by

374 Spring 123: Explain application flow of the Spring boot or What will happen when we call run ()?

When we create a Spring Boot app, we typically start with something like:

```
@SpringBootApplication
public class MyApp {
            public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
        }
}
```

Under the hood, a lot of things will happen. Let's walk through the flow step by step.

1.Run Method.

SpringApplication.run()

2. Environment Setup.

Spring Boot sets up the Environment, means it loads all the configurations like application.properties, system variables, and command line arguments.

3. ApplicationContext Creation.

Creates the ApplicationContext, it is the heart of Spring that manages all the beans. applicationContext as the container that holds all the beans.

4. Bean Definition

Now Spring boot scans all the beans. Spring detects them and registers it in the context.

@SpringBootApplication is the combination of:

@Configuration, @EnableAutoConfiguration and @ComponentScan

@Configuration - This annotation indicates that the class is a source of bean definitions for the Spring application context.

@EnableAutoConfiguration: This enables Spring Boot's auto-configuration mechanism. Spring Boot automatically configures the application based on the dependencies present in the class path.

@ComponentScan: This annotation instructs Spring to scan for components within a specified package (and its sub-packages). It automatically discovers and registers classes annotated with @Component, @Controller, @Service, @Repository

5. Auto-Configuration

It is one of best features in Spring boot. Based on classpath and environment, Boot decides what to configure.
For example:

- If spring-boot-starter-web is present, Boot auto-configures Tomcat, DispatcherServlet, etc.
- If spring-boot-starter-data-jpa is present, it configures Hibernate, DataSource, etc.

6. Context Refresh

After bean definitions are ready

- Spring calls context.refresh().
- This triggers actual bean instantiation, dependency injection, and initialization.

7. Starting Embedded Web Server (if web app):

If this is a web application:

- Boot starts Tomcat (or Jetty/Undertow).
- Binds it to the port from the config (server.port).
- Registers the DispatcherServlet.

8. Runners Execution.

Once the context is fully ready, Spring runs CommandLineRunner or ApplicationRunner

```
@Component
    public class MyRunner implements CommandLineRunner {
    @Override
        public void run(String... args) {
                System.out.println("App started with args: " + Arrays.toString(args));
        }
}
```

9. Application Ready

Finally, ApplicationReadyEvent is published. At this point:

- All beans are ready.
- The server is up.
- And the application is running.

375 Spring 124: What are different strategies of ID generation in Hibernate?

Every entity in Hibernate usually has a primary key. Hibernate needs to know how to generate that primary key when we insert a new record. This is where *identifier generation strategies* come in. Choosing the right strategy affects not just performance, but also portability across databases, scalability, and even data integrity.

Hibernate supports a number of identifier generation strategies.

1. AUTO (Default Strategy)

JPA allows Hibernate (or the JPA provider) decide which strategy to use based on the underlying database. MySQL uses IDENTITY. Oracle uses SEQUENCE.

2. IDENTITY

The database auto-generates the primary key using an auto-increment column. The ID is assigned only after insert.

```
@Entity
    class User
    { @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;
}
```

3. SEQUENCE

Uses a database sequence object to generate IDs. Hibernate fetches the next value from the sequence before the insert. Best for Oracle, PostgreSQL, or any DB that supports sequences.

```
@Entity
    class User
    { @Id
        @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "user_seq")
    @SequenceGenerator(name = "user_seq", sequenceName = "user_sequence", allocationSize
= 1)
        private Long id;
}
```

4. UUID

Generates a globally unique ID (UUID) instead of a numeric value.

```
@Entity

    class User
    { @Id
        @GeneratedValue(generator = "uuid")
    @GenericGenerator(name = "uuid", strategy = "uuid2")
    private String id;
}
```

Point to remember: @GenericGenerator is deprecated from Hibernate 6.5, we will have to use alternatives like @UuidGenerator, @IdGeneratorType

Tricky question:

You have an entity with a @GeneratedValue(strategy = GenerationType.AUTO), but while inserting records, IDs are not generating sequentially. Why?

We are basically telling JPA: "Pick the generation strategy based on the underlying database."

- On PostgreSQL / Oracle, it might use a sequence.
- On MySQL, it usually maps to AUTO_INCREMENT.
- On H2 / some others, it could choose a table

generator. And here is why we don't see sequential IDs:

- Sequences and identity columns often allocate IDs in blocks for performance. For example, Hibernate might reserve 50 IDs at a time (allocationSize=50 by default). If the app restarts, unused IDs are skipped, so gaps appear.
- In distributed systems, different instances can pre-allocate different ID ranges, leading to non- continuous values.
- AUTO doesn't guarantee *sequential* IDs So the answer: IDs are not sequential because GenerationType.AUTO relies on database-specific mechanisms (sequence, identity, table), and Hibernate optimizes them by allocating IDs in batches, which causes gaps.

If we really want sequential IDs, we have to explicitly use GenerationType.IDENTITY (on MySQL) or a @SequenceGenerator with allocationSize = 1 But it impacts the performance.

376 Spring 125: What is singleton?

A Singleton is a design pattern where only one instance of a class is created, and that single instance is shared across the application.

Think of it like having a single coffee machine in the office. No matter how many employees come to drink coffee, they all use the same machine. we don't build a new one every time.

In programming, this helps in:

- Saving memory (one object instead of many).

- Providing a single point of control.

Singleton in Plain Java Before Spring, we create a singleton in Java using private constructors and static methods.

```
class CoffeeMachine {
        private static CoffeeMachine instance;

        private CoffeeMachine() {} // private constructor

        public static CoffeeMachine getInstance() {
                if (instance == null) {
                        instance = new CoffeeMachine();
                }
                return instance;
        }

            public void brew() {
        System.out.println("Brewing coffee...");
        }
}

public class Main {
            public static void main(String[] args) {
        CoffeeMachine c1 = CoffeeMachine.getInstance();
        CoffeeMachine c2 = CoffeeMachine.getInstance();

                System.out.println(c1 == c2); // true (same object)
        }
}
```

Here, c1 and c2 point to the same object.

How Spring Handles Singleton

In Spring, we don't have to write all that boilerplate code. By default, every bean we define in Spring is a singleton.

Example:

```
@Component
class CoffeeMachine {
        public void brew() {
                System.out.println("Brewing coffee in Spring...");
        }
}
```

Now, when we inject this bean in multiple places: Even though Employee1 and Employee2 have their own copies of coffeeMachine variable, Spring gives them the same instance of CoffeeMachine bean.

```
@Service
class Employee1 {
```

```
        @Autowired
        private CoffeeMachine coffeeMachine;

            public void drink() {
        coffeeMachine.brew();
        }
}

@Service
    class Employee2 {
    @Autowired
        private CoffeeMachine coffeeMachine;

            public void drink() {
        coffeeMachine.brew();
        }
}
```

Additional Questions:

1. In Singleton, what problems may occur with serialization and reflection? Serialization Problem

- When we serialize a singleton object and then deserialize it, Java creates a new instance of the class.
- This breaks the singleton guarantee because now we have two instances.
- Use readResolve() method to preserve

singleton: Reflection Problem

- Reflection can bypass the private constructor and create a new instance of the singleton.
- This also breaks the singleton guarantee.

Reference for reflection: Java Reflection (With Examples)

2. How to break Singleton design pattern?

Reference: How to BREAK and FIX Singleton Design Pattern | Interview Question

3. How singleton work in multithread environment?

By using synchronized keyword, we can create Singleton object in multithread environment.

Reference: Singleton Design Pattern In Java With All Scenarios Examples

4. How spring's singleton scope is different than GOF singleton?

GoF Singleton (Gang of Four Singleton Design Pattern):

- It is a creational pattern where we make sure only one instance of a class exists in the entire JVM.
- The class itself controls its lifecycle, usually with a private constructor and a getInstance() method.

```
public class GoFSingleton {
        private static GoFSingleton instance;

        private GoFSingleton() {}

        public static GoFSingleton getInstance() {
                if (instance == null) {
                        instance = new GoFSingleton();
                }
                return instance;
        }
}
```

Spring Singleton (Bean Scope):

- "Singleton" in Spring means one bean instance per Spring container (ApplicationContext), not across the JVM.
- If we create multiple Spring contexts, each will have its own bean instance.

```
@Component
public class SpringSingletonBean {
}

ApplicationContext context1 = new AnnotationConfigApplicationContext(AppConfig.class);
ApplicationContext context2 = new AnnotationConfigApplicationContext(AppConfig.class);

SpringSingletonBean bean1 = context1.getBean(SpringSingletonBean.class);
SpringSingletonBean bean2 = context1.getBean(SpringSingletonBean.class);
SpringSingletonBean bean3 = context2.getBean(SpringSingletonBean.class);

System.out.println(bean1 == bean2); // true (same context)
System.out.println(bean1 == bean3); // false (different context)
```

377 Spring 126: How Cross Origin works in Spring?

When we open a website, the browser usually only allow us to talk to the same server it came from. That is called the **Same-Origin Policy**. But sometimes we want a page to talk to another server; for example, your frontend at http://localhost:3000 needs to call a Spring Boot backend at http://localhost:8080. That is where **CORS (Cross-Origin Resource Sharing)** comes in. Here is how Spring handles it internally:

1. Browser Sends Origin

Every cross-origin request includes an Origin header.

Example: Origin: http://localhost:3000

Spring reads this header to check where the request came from.

2. Preflight Requests

If the browser wants to send something more complex than a GET or POST (say DELETE or custom headers), it sends a quick OPTIONS request first.
Example preflight request:

```
OPTIONS /api/data HTTP/1.1
Origin: http://localhost:3000
Access-Control-Request-Method: DELETE
```

3. Spring's CORS Configuration

Spring compares the request against the CORS rules we have

defined: With @CrossOrigin on a controller method:

```
@CrossOrigin(origins = "http://localhost:3000")
@GetMapping("/data")
public String getData() {
        return "Hello CORS!";
}
```

Or globally with WebMvcConfigurer:

```
@Override
    public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
                        .allowedOrigins("http://localhost:3000")
                        .allowedMethods("GET", "POST", "DELETE");
}
```

4. Spring Adds Response Headers

If the origin is allowed, Spring attaches CORS headers to the response, for example:

- Access-Control-Allow-Origin: http://localhost:3000
- Access-Control-Allow-Methods: GET, POST, DELETE
- Access-Control-Allow-Headers: Content-Type

5. Browser checks It

The browser looks at these headers. If the headers match what it needs, it allows the frontend script to read the response. If not, it silently blocks it, even though the backend replied.

378 Spring 127: How @Retryable works in Spring?

Sometimes, we call an external service or a database, and it fails, maybe because of a network glitch, a timeout. In such cases, instead of immediately throwing an error, we might want to try again a few times before giving up.

That is exactly where Spring's **@Retryable** comes in. It is part of **Spring Retry**, and it gives us a neat way to retry failed operations automatically without writing boilerplate loops.

How it Works

When we put @Retryable on a method:

1. Spring wraps that method in a proxy.
2. If the method throws an exception that we havemarked for retry, Spring will call the method again.
3. It repeats this until either:
    - The method succeeds
    - The maximum retry attempts are exhausted
4. If retries are exhausted, we can handle the failure using

@Recover. Example:

Step 1: Add Dependency

```
<dependency>
    <groupId>org.springframework.retry</groupId>
    <artifactId>spring-retry</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Step 2: Enable Retry
Step 3: Use @Retryable

```
@Service
                throw new RuntimeException("Payment failed!");
        }

        @Recover
        public void recover(RuntimeException e) {
        System.out.println("All retries exhausted. Payment failed permanently: " +
e.getMessage());
        }
}
        counter); counter++;
```

Key Points to Remember

- @Retryable works only on Spring-managed beans.
- It needs Spring AOP (that is why we added spring-boot-starter-aop).
- Use @Recover to define a fallback method after retries fail.
- We can control retries with:
    - maxAttempts means how many times
    - backoff means delay, multiplier (exponential backoff)
    - value to which exceptions trigger retry

Reference: Implementing Method-Level Retry in Spring Boot |

Medium Indirect Question:

1. One of your microservice is down then how do you handle? Use Retry or circuit breaker

379 Spring 128: How proxyMode works in Spring and how does it help? Problem: Injecting a Short-Lived Bean into a Long-Lived Bean

Imagine we have a **singleton bean** (default scope) and a **request-scoped bean**. We want to inject the request-scoped bean into the singleton:

Singleton bean:

```
@Service
public class SingletonService {

        @Autowired
        private RequestService requestService; // Problem!

            public void printRequestId() {
        System.out.println(requestService.getId());
        }
}
```

Request-scoped bean:

```
@Component
@RequestScope
public class RequestService {
        private final String id = UUID.randomUUID().toString();
```

```
        public String getId() {
                return id;
        }
}
```

What happens: Spring will throw an error or inject the same instance of the request bean into the singleton. That is wrong, because request-scoped beans are supposed to be new for each HTTP request.

The core issue: singleton lives longer than request, so Spring doesn't know which request instance to inject.

Solution: Use proxyMode

Definition: In Spring Framework, proxyMode = ScopedProxyMode.TARGET_CLASS is used in conjunction with the @Scope annotation to define how a scoped bean (like a prototype, request, or session-scoped bean) should be injected into other beans, especially singleton beans.

Spring provides a way to inject a proxy instead of the actual request bean. The proxy acts like a placeholder and resolves the actual bean at runtime based on the current request.

```
@Component
@RequestScope(proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestService {
        private final String id = UUID.randomUUID().toString();

        public String getId() {
                return id;
        }
}
```

Reference: Bean Scopes in Java Springboot. Non-related intro | by Jayamal Jayamaha | Medium

Now, when the singleton calls requestService.getId(), Spring fetches the correct request-specific instance behind the scenes.

380 Spring 129: How to deploy Spring boot application using jar or war?

When we finish building a Spring Boot project, the next big step is **deployment**.
This means taking the application out of the IDE and running it in the real world means on a server, or even in the cloud.

Spring Boot makes this surprisingly simple, and it gives, two main ways to package and deploy the application:

- As an **executable JAR**
- As a **traditional WAR**

1. Deploying as a JAR (the modern way)

By default, Spring Boot applications are packaged as JAR files. Because Spring Boot already bundles an embedded web server (Tomcat, Jetty, or Undertow). That means the app carries its own server inside, like a backpack. We don't need to install Tomcat separately.

How to build a JAR

1. Open the pom.xml and make sure the packaging is set to jar:

   <packaging>jar</packaging>

2. Package the app with Maven: mvn clean **package**

3. We will see something like this in the target folder: myapp-0.0.1-SNAPSHOT.jar
4. Run it just like any other Java program: java -jar target/myapp-0.0.1-SNAPSHOT.jar

2. Deploying as a WAR (the traditional way)

Before Spring Boot came along, Java web apps were typically packaged as **WAR files** and deployed into external servers like Tomcat, JBoss, or WebLogic.
This approach is still used in companies that maintain older infrastructures.

How to build a WAR

Change the packaging type in pom.xml: <packaging>war</packaging>

Update the main Spring Boot class to extend

SpringBootServletInitializer:

```
@SpringBootApplication
public class MyAppApplication extends SpringBootServletInitializer {

	@Override
	protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
		return builder.sources(MyAppApplication.class);
	}

	    public static void main(String[] args) {
	SpringApplication.run(MyAppApplication.class, args);
	}
}
```

Build the WAR: `mvn clean package > myapp-0.0.1-SNAPSHOT.war`

Drop this WAR file into the external server's webapps/ directory (for Tomcat, it is usually /apache-tomcat/webapps/).

Additional Questions:

1. What is JAR and WAR?

A JAR file stands for Java Archive. It is a package file format used to aggregate many Java classes, associated metadata, and resources (like images or text) into one file for easy distribution. JAR files are primarily used for deploying standalone Java applications or libraries that other projects can reference.

A WAR file stands for Web Application Archive. It is a specialized JAR file specifically for web applications. It has a standardized structure for deploying to a Servlet Container (like Tomcat, Jetty) or Application Server (like WildFly, WebSphere).

2. When to choose JAR and WAR?

Use JAR for,

Standalone Applications: Any application that has its own main() method and is run from the command line (e.g., using java -jar myapp.jar). This is the standard for:

- Microservices
- Batch jobs (e.g., using Spring Batch)
- CLI tools and utilities

- Desktop applications (though less common now)

Modern Spring Boot Applications: This is a crucial point. Spring Boot uses an "embedded server" approach.

Our web application is packaged as a JAR.

This JAR contains an embedded Tomcat, Jetty, or Undertow

server. We can run it instantly without needing a separately

installed Tomcat.

This is the default and preferred approach for most modern Spring Boot development due to its simplicity and portability.

Use WAR for

Traditional Web Application Deployment: When we need to deploy the application to an existing, external Servlet Container or Application Server.

- We have a dedicated Tomcat server where we need to deploy multiple applications.
- And Company's infrastructure team manages the server, and we just provide the .war file.

Reference: **Difference Between JAR and WAR. Java Archive(Jar) File vs Web… | by Abdul Kadar | Medium**

381 Spring 130: Explain different kind of paging techniques?

Imagine your database has 1 million rows. If your API return all of them in one shot, your client (and your server) cannot handle. How can you implement.

Solution:

This is where Pagination comes into picture, means fetching only a set of records at a time. Means “Reading records like a page where each page contains fixed number of records”.

Pagination Techniques

1. Offset-based Pagination (Page & Size)

This is the most common technique, here we need specify the page number and page size.

- Page number means ‘which page?’
- Page size means ‘how many records per

page?’. Example: GET /products?page=2&size=5

Here we are mentioning, “skip first 5 records, then give me next

5”. In SQL, it handles with LIMIT and OFFSET.

```
SELECT * FROM products ORDER BY id LIMIT 5 OFFSET 5;
```

LIMIT “tells the database how many rows to return.” In this example 5 will be returned.

OFFSET “tells the database to skip the specified number of rows before applying the LIMIT”. In this example as we mentioned offset 5, it skips first 5 rows and return next 5 rows

```
SELECT * FROM products LIMIT 5 OFFSET 10;
```

In this example, it skips first 10 rows, then return next 5 rows (rows 11 to 15).

Remember like OFFSET is “start point,” LIMIT is “how many to fetch.”

In Spring, Pagination and sorting is handled by PagingAndSortingRepository.

```
Pageable pageable = PageRequest.of(1, 5); // page index starts at 0
Page<Product> products = productRepository.findAll(pageable);
```

The PageRequest.of(1, 2) means page index 1 (remember it starts at 0), with 5 items per page. Spring takes care of building the LIMIT and OFFSET query for us.

Reference: Spring Boot Pagination and Sorting Example

Offset pagination works for small datasets, imagine if we request for 10,000th page, then Database has to scan first 10000 records to skip them when slows down the process. For very large datasets we use Keyset/Cursor pagination.

2. Keyset Pagination (Also known as Cursor Pagination):

Keyset pagination retrieves records based on a specific key (usually a unique, indexed column like ID or timestamp). This makes it faster and more scalable, especially for large datasets.Example, Let's say we are using "timestamp" as key:

In the first API call clients send without any key: GET /products?&size=10, so Backend has to first 10 records, SQL for this is

```
SELECT * FROM products ORDER BY timestamp ASC LIMIT 10;
```

The backend returns those 10 products and also includes the last product's timestamp in the response (let's say 2025-01-01).

From next request, Client to has to send latest timestamp in the key GET /products?key=2025-01-01&size=10. SQL:

```
SELECT * FROM products WHERE timestamp > 2025-01-01 ORDER BY timestamp ASC LIMIT 10;
```

Which means client wants the 10 records after Jan 1st 2025. From there every request should contain the key.

Code Example:

```
public interface OrderRepository extends JpaRepository<Order, Long> {

        // First page (no cursor)
        @Query("SELECT o FROM Order o ORDER BY o.timeStamp
    ASC") List<Order> findFirstPage(Pageable pageable);

        // Next pages (cursor provided)
    @Query("SELECT o FROM Order o WHERE o. timeStamp > :cursor ORDER BY o.createdAt
ASC")
        List<Order> findNextPage(@Param("cursor") Instant cursor, Pageable pageable);
}
```

Below service decides which query to call depending on whether a cursor is provided.

```
@Service
public class OrderService {

        @Autowired
        private OrderRepository orderRepository;

        public List<Order> getOrders(Integer size, Instant cursor) {
                Pageable pageable = PageRequest.of(0, size,
        Sort.by("createdAt").ascending());

                if (cursor == null) {
                        return orderRepository.findFirstPage(pageable);
                } else {
                        return orderRepository.findNextPage(cursor, pageable);
                }
        }
```

REST APIs

382 Rest 1: What is REST API and what are its uses?

REST (Representational State Transfer) API is a way for two systems (like a frontend app and a backend server) to talk to each other over HTTP using a set of rules. Instead of inventing new ways to exchange data, REST relies on standard HTTP methods like:

- **GET**: fetch data
- **POST**: create data
- **PUT/PATCH**: update data
- **DELETE**: remove data

The data is usually sent in formats like **JSON** or **XML** (JSON is the most common).

Example:

GET /users/1: Get details of user with ID 1
POST /users: Create a new user

Uses of REST API

1. **Communication between applications**: Web app or mobile app can fetch data from a backend server via REST APIs.
2. **Platform-independent**: Since it is based on HTTP, any system (Java, Python, React, Android, iOS) can use it.
3. **Scalability**: Each request is stateless (server doesn't store session info), making it easier to scale.
4. **Integration**: REST APIs allow connecting third-party services (payment gateways, maps, social media).
5. **Reusability**: Once an API is built, multiple clients (web, mobile, desktop apps) can use the same endpoints.

Point to Remember: You may be asked to write sample code for any API, so practice GET, POST, and PUT

Indirect Question:

1. What is stateless and stateful in REST?

In REST, stateless means every request from the client must contain all the info needed, and the server doesn't remember past interactions.
Stateful means the server stores session data about the client, so requests can rely on the server's memory of previous calls.

Stateless example:

- A GET /users/123 request always includes authentication (like a token in headers).
- The server doesn't remember the user logged in; every request carries the needed info.

Stateful example:

- we log in once, the server creates a session and stores it in memory.
- On the next GET /users/123, we just send a session ID, and the server uses its stored state to know the user who logged in.

2. How can you maintain a session in a stateless REST API

Use tokens instead

The common approach is JWT (JSON Web Tokens):

- Client logs in then server generates a token with user info and expiry.
- Client sends the token on every request (usually in the Authorization header).
- Server validates the token, without needing to store session state.

3. Why should you handle response timeout while calling any API and how to handle in Spring?

If we don't handle response timeouts when calling an API, our app can just wait there forever if the other service is slow or dead. That means bad user experience, and even system crashes under load.

In Spring, we handle this by setting **timeouts** in the HTTP client. For example, with RestTemplate or WebClient, we can configure connectTimeout and readTimeout. If the API doesn't respond in that time, the call fails fast, and we can retry, show an error, or switch to a fallback.

```
@Configuration
public class AppConfig {

        @Bean
        public RestTemplate restTemplate(RestTemplateBuilder builder) {
                return builder
                                .setConnectTimeout(Duration.ofMillis(3000)) // Example:
                Set connection
timeout
                                .setReadTimeout(Duration.ofMillis(3000))      //
                Example: Set read timeout
                                .build();
```

4. Difference between Connection Timeout and Read Timeout?

Connection Timeout: How long the client will wait to establish a connection with the server.
Example: if the server is down or unreachable, this timeout kicks in.

Read Timeout: How long the client will wait for data after the connection is established.
Example: server accepted the connection but is too slow to send the response.

383 Rest 2: What is difference between PUT and PATCH?

PUT: It Replaces the entire resource with the new one. If we don't provide some fields, they can be overwritten with defaults or null.

```
@PutMapping("/{id}")
public ResponseEntity<User> updateUser(@PathVariable Long id, @RequestBody User newUser)
{


        return userRepository.findById(id)
                        .map(user -> {
                                // replace all fields
                user.setName(newUser.getName());
                user.setAge(newUser.getAge());
                user.setCity(newUser.getCity());
                                return ResponseEntity.ok(userRepository.save(user));
                        })
                        .orElseGet(() -> {
                               // if not found,
                create new newUser.setId(id);
                                return ResponseEntity.ok(userRepository.save(newUser));
                        });
}
```

PATCH: It is used for Partially updates means only the fields we send are updated, the rest stay unchanged.

Tricky Question:

In both the methods we are using `userRepository.save(user), then how does PUT different from PATCH?`

Let's see How save() works under the hood

- In JPA/Hibernate, save() doesn't directly do an SQL UPDATE of everything.
- It checks if the entity already exists (id is present).
    - If **new**, it runs INSERT.
    - If **existing**, it compares fields in the managed entity with what's changed and generates an UPDATE ... SET ... WHERE id=?.

Here the thing is it is not about what Hibernate does, it is about semantics of the API contract.

Technically, both may call save()

```
@PatchMapping("/{id}")
public ResponseEntity<User> patchUser(@PathVariable Long id, @RequestBody Map<String,
Object> updates) {
        return userRepository.findById(id)
                        .map(user -> {
                                updates.forEach((key, value) -> {
                                        switch (key) {
                                                case "name":
                                user.setName((String) value); break; case "age":
                                user.setAge((Integer) value); break; case "city":
                                user.setCity((String) value); break;
                                        }
                                });
                                return ResponseEntity.ok(userRepository.save(user));
                        })
                        .orElse(ResponseEntity.notFound().build());
```

But from a REST design perspective:

- Use PUT if clients must send the entire object.
- Use PATCH if clients can send only the fields they want to change.

384 Rest 3: Can you tell what are the annotations you used while developing REST APIs?

@RestController: Marks the class as a REST controller (returns JSON/XML instead of views). @RequestMapping: Defines the base URL for all methods in the controller.

@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping: HTTP-specific shortcuts for CRUD endpoints.

@PathVariable: Extracts values from the URI path.

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {

}
```

@RequestParam: Extracts query parameters from the URL.

```
@GetMapping("/users")
public List<User> findUsers(@RequestParam String city) {

}
```

@RequestBody: Maps the request JSON payload to a Java object.

@ResponseEntity: Builds custom responses with status codes and headers.

```
ResponseEntity
.status(HttpStatus.CREATED)
.body(savedUser);
```

385 Rest 4: Difference between @RestController and @Controller? @Controller

- It is a specialization of @Component; it marks class as a web request handler. It is used to declare common web controllers that can return HTTP responses
- By default, methods in a @Controller return views (like JSP, Thymeleaf templates) unless we add @ResponseBody.
- Typically used in web applications where we return HTML pages.

@RestController

- It is the combination @Controller + @ResponseBody. It is used to create controllers for REST APIs that can return JSON responses.
- Every method automatically returns JSON/XML instead of a view.
- Typically used in REST APIs.

Reference: What is the difference between @Controller vs @RestController in Spring Boot?

Tricky Questions:

1. What happens if we use both @Controller and @RestController on the same class?

- @RestController itself is a combination of @Controller + @ResponseBody.
- If we add @Controller along with @RestController, it doesn't break anything.
- But it is redundant, Means Spring will still treat it as a @RestController.

2. What happens if we don't use @ResponseBody with @Controller?

Without @ResponseBody, Spring will assume the method is returning a **view name** (HTML/JSP/Thymeleaf template).

```
@Controller
    public class
    TestController {
    @GetMapping("/hello")
    public String hello() {
                return "Hello World"; // interpreted as view name "Hello World"
        }
```

Spring will look for a template called Hello World.html instead of returning "Hello World" as text.

To return raw data (like JSON, String, etc.), we must use @ResponseBody or just switch to @RestController.

3: Can we use @RestController and still return a view (HTML)?

By default, no. @RestController forces @ResponseBody on all methods, so it always returns JSON/XML. If we really want to return a view, we must explicitly use ModelAndView.

```
@RestController
    public class PageController {
    @GetMapping("/page")
        public ModelAndView getPage() {
                return new ModelAndView("home"); // renders home.html
        }
}
```

4. What happens if a method in @RestController returns void?

- If the method is void and we don't write anything to the response, the client just gets an empty 200 OK response.
    - Useful in cases like a DELETE endpoint where we don't want to return data.

386 Rest 5: Difference between @RequestMapping and @GetMapping? @RequestMapping

- It is a general-purpose annotation to map HTTP requests to handler methods.
- It can handle **all HTTP methods** (GET, POST, PUT, DELETE, etc.) via the method attribute.

```
@RequestMapping(value = "/users", method = RequestMethod.GET)
public List<User> getUsers() {

}
```

@GetMapping

- It is the Shortcut (specialization) of @RequestMapping(method = RequestMethod.GET).
- It is Cleaner, more readable when the method is only handling GET requests.

```
@GetMapping("/users")
public List<User> getUsers() {

}
```

Tricky Questions

1. Can @RequestMapping handle multiple HTTP methods in a single method? Yes. Example:

```
@RequestMapping(value = "/users", method = {RequestMethod.GET, RequestMethod.POST})
public String handleUsers() {
        return "Handled GET or POST";
}
```

2. What happens if you use @RequestMapping without specifying a method? It will match **all HTTP methods** (GET, POST, PUT, DELETE, etc.) for that URL.

3. Can you combine @RequestMapping at class level with @GetMapping at method level? Yes, that is common. The class-level @RequestMapping acts as a **base path**

```
@RestController
@RequestMapping("/api")
public class UserController {

	@GetMapping("/users")
	public List<User> getAllUsers() {
		return List.of(new User(1L, "CodingLyf"));
	}
}
```

Final endpoint = /api/users.

387 Rest 6: Difference between @PathVariable and @RequestParam? @PathVariable

It is used to extract values from the URL path itself.

Typically used when the value uniquely identifies a resource.

```
@GetMapping("/users/{id}")
public String getUserById(@PathVariable Long id) {
	return "User ID: " + id;
}
```

Request: /users/10 : Response: User ID: 10

@RequestParam

- Used to extract values from **query parameters** in the URL.
- Typically used for filtering, searching, sorting, or optional values.

```
@GetMapping("/users")
public String getUsersByCity(@RequestParam String city) {
	return "Users in city: " + city;
}
```

When to use which

Use @PathVariable when the value is part of the resource identifier.

/users/1/orders/99 userId = 1, orderId = 99

Use @RequestParam for optional or filtering parameters.

/users?city=Hyd&sort=asc

Tricky Questions:

1. What happens if you don't pass a @RequestParam that is required?

By default, it throws an exception (MissingServletRequestParameterException). To avoid this, we can make it optional:

@RequestParam(required = false) String city or

@RequestParam(defaultValue = "Hyderabad") String city

2. What happens if you don't pass a @PathVaraible that is required?

If the value is required (default behavior) and it is missing from the request URL, we will get a 400 Bad Request error because Spring can't bind it.

If we make it optional (required = false) and it is missing, Spring will pass null to the method parameter.

```
// Optional PathVariable
@GetMapping({"/info", "/info/{id}"})
public String getUserInfo(@PathVariable(required = false) Integer id) {
```

```
        return id == null ? "No ID provided" : "User Info for ID: " + id;
}
```

388 Rest 7: What is Response Entity and how it will be used?

ResponseEntity is a Spring class that represents the whole HTTP response.It allows us to set not only the body of the response, but also the status code and headers.

In other words, instead of just returning data, we get full control over what the client will receive. Why to use ResponseEntity?

- To customize HTTP status codes (200 OK, 201 CREATED, 404 NOT FOUND, etc.)
- To set custom headers (like authentication tokens, cache-control, etc.)
- To send data + metadata together

```
@GetMapping("/user/{id}")
    public ResponseEntity<User> getUser(@PathVariable int id)
    { User user = userService.findById(id);
        if (user != null) {
                return ResponseEntity.ok(user); // 200 OK with user data
        } else {
                return ResponseEntity.status(HttpStatus.NOT_FOUND).build(); // 404
        }
}
```

It returns data with a status code

```
@PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody LoginRequest request)
    { String token = authService.authenticate(request);

        return ResponseEntity.ok()
                        .header("Authorization", "Bearer " + token)
                        .body("Login successful");
}
```

This login APIresponse includes a header + body + status.

1. Can we return ResponseEntity without a body? If yes, what will happen? Use ResponseEntity.ok().build()

2. How do you map exceptions to different HTTP status codes in a RESTFul Spring boot application?

1. @ResponseStatus on custom exceptions

Attach the status code directly to an exception class.

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
        public ResourceNotFoundException(String message) {
                super(message);
        }
}
```

Whenever this exception is thrown, Spring will return **404 Not Found**.

2. @ExceptionHandler inside a @ControllerAdvice

Centralized place to catch exceptions and return proper status codes.

```
@ControllerAdvice
    public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
        public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
                return
        ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
        }

        @ExceptionHandler(IllegalArgumentException.class)
        public ResponseEntity<String> handleBadRequest(IllegalArgumentException ex) {
                return
        ResponseEntity.status(HttpStatus.BAD_REQUEST).body(ex.getMessage());
```

389 Rest 8: Different types of HTTP status codes?

Role of HTTP codes:

HTTP status codes provide a standardized way to indicate the success or failure of an HTTP request. In REST APIs, they inform the client about the result of their request, such as success (2xx codes), client errors (4xx codes), or server errors (5xx codes). Proper use of these codes enhances API usability and debugging.

200 OK : Request succeeded and response contains the expected data

201 Created : A new resource was successfully created (commonly in POST APIs)

204 No Content : Request succeeded but no response body (e.g., DELETE success)

301 Moved Permanently : Resource permanently moved to a new URI

302 Found : Temporary redirect to another URI

304 Not Modified : Resource not changed, client can use cached version

400 Bad Request : Client sent an invalid request (missing params, invalid data)

401 Unauthorized : Authentication required or failed (invalid token/credentials)

403 Forbidden : Authenticated but not allowed to access the resource

404 Not Found : Requested resource doesn't exist

405 Method Not Allowed : HTTP method not supported for the resource

409 Conflict : Request conflicts with server state (e.g., duplicate entry)
415 Unsupported Media Type : Request format not supported (e.g., sending XML instead of JSON)

429 Too Many Requests : Rate limit exceeded

500 Internal Server Error : Generic server-side failure

502 Bad Gateway : Server acting as a proxy/gateway got invalid response

503 Service Unavailable : Server temporarily overloaded or down for maintenance

504 Gateway Timeout : Server acting as a proxy/gateway didn't get a timely response

390 Rest 9: How Spring send JSON by default / How Spring converts Java object to JSON while returning response from API?

Here is what happens behind the scenes:

Spring MVC + @RestController

When we return a Java object from a controller method (inside a @RestController or with @ResponseBody), Spring doesn't just return the object directly. It delegates to a **HttpMessageConverter** to serialize it into the desired format (usually JSON).

Default JSON Conversion

- Spring Boot includes **Jackson** (com.fasterxml.jackson) in its starter dependencies (spring-boot- starter-web).
- Jackson automatically converts the Java object (POJO) into JSON if the request's Accept header includes application/json.

```
@RestController
    public class
    UserController {
    @GetMapping("/user")
    public User getUser() {
                return new User("CodingLyf", 1);
        }
}

class User {
        private String name;
        private int age;

        // getters & setters
```

Response JSON:

```
{
    "name": "CodingLyf",
  "age": 1
}
```

391 Rest 10: Different types of headers?

1. Content-Type

Defines the format of the request or response body (JSON, XML, text). Example: Content-Type: application/json

When a client sends data to a server, the Content-Type header informs the server about the format of the data in the request body.

When a server sends data back to a client, the Content-Type header informs the client about the format of the data in the response body

2. Accept

The Accept header is a request header used by clients to specify which media types they are willing to accept as a response from the server.
Example: Accept: application/json

3. Authorization

It carries credentials like tokens or API keys for authentication. Example: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

4. Cache-Control

It is used for caching behavior like no-store, no-cache, or max-age.
Example: Cache-Control: no-cache

392 Rest 11: What is content negotiation?

Content Negotiation is how Spring (or any web framework) decides what format (JSON, XML, HTML, etc.) to send back to the client based on the request.

There are three main ways:

1. **Header-based (Accept Header)**

   Client sends Accept: application/json , Server responds with JSON.

   GET /products Accept: application/xml

   Response: XML format.

2. **Parameter-based (URL query parameter)**

   Format is specified in URL.

   GET

   /products?format=json

3. **Path extension (deprecated in Spring)**
   - File-like extension in

   URL. GET /products.json

Indirect Questions:

1. Can a controller return String instead of JSON? Yes.

- In a @RestController, we can return a string as plain text
- In a @Controller, a String is usually treated as a view name (we need to annotate the method with @ResponseBody, then we can send as plain text).

2. What if we return a plain string without @ResponseBody? A String is usually treated as a view name

3. Suppose you need to send a Java object over a network in a distributed application. What challenges might arise, and how would you handle them?

Challenges

1. Serialization: To send an object, it must be converted into bytes. If the class doesn't implement Serializable, it can't be sent directly.
2. Versioning issues: If sender and receiver have different versions of the class (e.g., extra fields), deserialization can fail.
3. Performance overhead: Default Java serialization is slow and creates large payloads.
4. Security risks: Deserialization can be exploited if untrusted data is processed.

How to handle

- Use Serializable or Externalizable for converting objects to bytes.
- Define serialVersionUID to manage version compatibility.
- For performance and portability, prefer protocols like JSON, XML, or Protobuf instead of Java's built-in serialization.
- Validate inputs and avoid deserializing from untrusted sources to reduce security risks.

4. If you want to return both HTML views and JSON responses from a single controller class, how can you achieve this?

@Controller can return HTML views (via Thymeleaf, JSP, etc.) when you return a String (view name). It can also return JSON if you annotate specific methods with @ResponseBody.

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping("/users")
public class UserController {

        // Returns HTML view
    @GetMapping("/page")
    public String userPage() {
                return "userView"; // mapped to a template like userView.html
        }


        // Returns JSON
    @GetMapping("/json")
    @ResponseBody
        public User getUserJson() {
                return new User(1, "CodingLyf");
        }
}
```

393 Rest 12: How can Spring send XML if default is JSON?

By default, Spring Boot uses Jackson to convert Java objects to JSON. But it can also produce XML if we include the right dependency.

Steps:

1. Add Jackson XML dependency: If we use Maven, add this to pom.xml:

```
<dependency>
        <groupId>com.fasterxml.jackson.dataformat</groupId>
        <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

2. Controller Example

```
@RestController
@RequestMapping("/api")
public class EmployeeController {

    @GetMapping(value = "/employee", produces = {MediaType.APPLICATION_XML_VALUE,
MediaType.APPLICATION_JSON_VALUE})
        public Employee getEmployee() {
                return new Employee(1, "CodingLyf", "Software Engineer");
        }
}
```

If client sends Accept: application/json , returns JSON. If client sends Accept: application/xml , returns XML.

394 Rest 13: What is Idempotency?

Idempotency means **making the same request multiple times gives the same result without causing side effects**.

In HTTP:

- **GET** is idempotent, calling it multiple times returns the same data.
- **DELETE** is idempotent, deleting the same resource multiple times still results in "deleted".
- **PUT** is idempotent, updating a resource with the same payload gives the same

state. Example:

- DELETE /users/123 : first call deletes, next calls return "user not found", but the system state doesn't change further.

395 Rest 14: Why Patch is not idempotent?

PATCH is not guaranteed idempotent.

Reason: PATCH applies a partial update. If the patch operation is additive (like "increment balance by 100"), sending it multiple times changes the state each time.

But if the patch operation is a direct replacement (like "set name to 'CodingLyf"), then it behaves idempotent.

PATCH may or may not be idempotent depending on how you design the patch

semantics. By HTTP spec, PATCH is not required to be idempotent (unlike PUT).

396 Rest 15: Can we make POST as idempotent?

Normally, POST is not idempotent because each call creates a new resource (like adding a new

record). But yes, we can design a POST API to behave idempotently using Idempotency Keys.

How it works

- While invoking the API, Client generates a unique key (say Idempotency-Key: abc123).
- Server stores this key with the request and its response.
- If the client retries the same POST with the same key, the server returns the stored response instead of creating a new resource again.

Real-world use case:

- Payment APIs (Stripe, PayPal, Razorpay, etc.)
    - When we POST a payment request, client include an idempotency key.
    - If the client retries due to timeout, the server won't charge the customer

twice. So, POST can be made idempotent through server-side logic + idempotency keys.

397 Rest 16: How do REST APIs handle versioning, and why is it important?

Once we release an API, people use it. If we change the endpoints, request/response formats then clients can break mean they may get errors eventually apps fail.

To avoid failures to the existing clients, we use something called API Versioning.

REST API versioning is all about how we manage changes to the API without breaking existing

clients There are various approaches to versioning a RESTful API, including:

- URL Versioning: Adding a version number to the API endpoint (e.g., /api/v1/resource).

- Query Parameter Versioning: Specifying the version as a query parameter (e.g., /api/resource?version=1).
- Header Versioning: Sending the version information in a custom header (e.g., X-API-Version: 1).

398 Rest 17: How do you validate the Request from client in Spring boot?

Spring Boot provides a clean way to handle validation using **annotations**, **binding**, and **exception handling**.

1. Using Validation Annotations

Spring Boot supports the Jakarta Validation API with annotations like:

- @NotNull – ensures the value is not null
- @Size(min=, max=) – validates the length of strings or collections
- @Email – ensures the value is a valid email address
- @Min / @Max – validates numeric ranges

These annotations are added directly to the request DTOs (Data Transfer Objects).

2. Enabling Validation in Controllers

To trigger validation automatically when a client sends a request, we need to use the @Valid annotation in the controller method

```
@PostMapping("/users")
public ResponseEntity<String> createUser(@Valid @RequestBody UserRequest request) {
        return ResponseEntity.ok("User created successfully");
}
```

@Valid tells Spring to check the DTO annotations.

If the request is invalid, Spring throws a MethodArgumentNotValidException.

3. Handling Validation Errors

We can handle errors gracefully using @ControllerAdvice and a global exception handler:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

        @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<String> handleValidationErrors(MethodArgumentNotValidException
ex) {
                String errorMessage = ex.getBindingResult()
                                                                .getFieldErrors()
                                                                .stream()
```

```
                                                        .map(err ->
                                err.getField() + ": " +
err.getDefaultMessage())
                                                        .collect(Collectors.join
                                ing(", "));
                        return ResponseEntity.badRequest().body(errorMessage);
```

This way, we can send a clear message about what went wrong instead of a generic server error.

399 Rest 18: How to create custom validators in spring?

Creating custom validators in Spring involves defining a custom annotation and implementing the validation logic in a separate class.

Let's say we are creating custom validator to check Strong password. Password should contain at least 8 chars, 1 digit, 1 uppercase.

Steps to Create a Custom Validator:

1. Add the Validation Dependency.

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

2. Create a Custom Constraint Annotation.

Define an annotation that will be used to mark fields or classes for validation. This annotation needs to specify the validator class and other metadata.

```
import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import java.lang.annotation.*;

@Documented
@Constraint(validatedBy = StrongPasswordValidator.class)
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface StrongPassword {
        String message() default "Password is too weak"; //Default error
    message Class<?>[] groups() default {};
        Class<? extends Payload>[] payload() default {};
}
```

- @Constraint declares the class which will implement our interface and define its behaviour, `StrongPasswordValidator is the class which will implement StrongPassword interface`
- @Target defines the program element types to which our custom annotation will apply,
- @Retention defines when our annotation will be available.

3. Implement the Validator Class.

Create a class that implements the ConstraintValidator interface. This class contains the actual validation logic.

```
import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

public class StrongPasswordValidator implements ConstraintValidator<StrongPassword,
String> { // StrongPassword is the annotation, String is the type to validate


        @Override
        public boolean isValid(String password, ConstraintValidatorContext context) {
                if (password == null) return false;
                // simple example: at least 8 chars, 1 digit, 1 uppercase
                        return
               password.length() >= 8 &&
               password.matches(".*[A-Z].*") &&
               password.matches(".*\\d.*");
        }
```

4. Apply the Custom Annotation.

Annotate the fields or parameters in the model or DTOs where you want to apply the custom validation.

```
    public class UserRequest
    { @StrongPassword
        private String password;

        // getters and setters
}
```

5. Trigger Validation.

Ensure that validation is triggered, typically by using @Valid or @Validated annotations on method parameters in the controllers or services.

```
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;

@RestController
@RequestMapping("/users")
public class UserController {

        @PostMapping
        public String createUser(@Valid @RequestBody UserRequest request) {
                return "User created with password: " + request.getPassword();
        }
}
```

That is it. Now if the password doesn't meet the rule, Spring Boot will throw a MethodArgumentNotValidException with the custom message.
Reference: Spring Boot Custom Validation. Creating custom validation in a Spring… | by Bereket Berhe | Medium

Creating Spring Boot Custom Validators | by Catherine Edelveis | Medium

400 Rest 19: How do you implement caching in REST APIs?

Implementing caching in REST APIs involves various strategies across different layers to improve performance and reduce server load.

1. Client-Side Caching (Browser Caching):

HTTP Headers:

- We can achieve using HTTP heaers, Cache-Control, Expires, ETag, and Last-Modified headers in the API responses.
- Cache-Control: Specifies caching policies (e.g., public, private, max-age, no-cache).

- Cache-Control: max-age=3600, public: Means the response can be cached by anyone for 1 hour.

- Expires: Provides a date/time after which the response is considered stale.
  - Expires: Wed, 23 Aug 2025 12:00:00 GMT
- ETag: A unique identifier for the resource version, allowing clients to send If-None-Match to check for updates.
- Last-Modified: The last modification date of the resource, used with If-Modified-Since.

2.Server-side Caching

- Store frequently accessed responses in Redis, Memcached, or in-memory caches.
- Example: Cache all product details for 10 minutes in Redis, so repeated API calls don't hit the database.

401 Rest 20: How can you handle rate limiting in REST API?

Rate limiting is the process of restricting the number of requests a client can make to an API within a given time frame. It helps prevent abuse, protects server resources, and ensures fair usage. Rate limiting can be implemented by tracking the number of requests per client and enforcing limits using techniques like token bucket or fixed window algorithms

Refer Question No 434, Spring Scenario 23 for the implementation

402 Rest 21: Rate limiting vs API throttling?

Both these are used to protect the misuse or overload of the APIs

1. Rate Limiting

Rate limiting is about setting a maximum number of requests a client can make within a fixed time window.
If the limit is 100 requests per minute, the 101st request is immediately rejected. If the limit is crossed, the API usually returns 429 Too Many Requests.

2. Throttling

Throttling is about controlling the speed at which requests are processed, rather than blocking them.

If the system allows 5 requests per second and we send 20 at once, only 5 are processed immediately. The rest are queued, delayed, or dropped based on policy.

403 Rest 22: What are different types of authentications used in REST APIs?

Basic Authentication: The client includes the username and password in the request headers. This method is simple but not recommended for sensitive data as it transmits credentials in plaintext.

Token-based Authentication: The server generates and returns a token (e.g., JSON Web Token or JWT) upon successful login. The client includes this token in subsequent requests to access protected resources.

OAuth: A protocol that allows users to authorize third-party applications to access their resources without sharing credentials. It involves obtaining an access token from an authorization server and using it to make authenticated requests.

404 Rest 23: Difference between URI and URL?

A URI (Uniform Resource Identifier) is a general term that identifies a resource. It can identify a resource by its name, its location, or both. Think of a URI as a way to uniquely identify something.

A URL (Uniform Resource Locator) is a specific type of URI that identifies a resource by its location

and also describes the means of accessing it. In essence, a URL tells where a resource is and how to get it.

Key Differences:

- **Scope**: All URLs are URIs, but not all URIs are URLs. A URI can be a URN (Uniform Resource Name) which identifies a resource by its name without specifying its location
  (e.g., urn:isbn:0451450523 identifies a book by its ISBN).
- **Purpose**: URIs are for identification, while URLs are for location and access.
- **Components**: URLs always include a protocol (like http://, https://, ftp://), a domain name, and potentially a path and file name, all of which are essential for locating and accessing the resource. URIs can be simpler, focusing solely on identification.

405 Rest 24: How do you implement pagination in REST API?

In a REST API, pagination is often used to limit the amount of data returned in a single request. There are different approaches to handling pagination.

One common approach is to use query parameters, such as "page" and "limit", to specify the desired page number and the number of items per page.
The API can then use this information to retrieve and return the appropriate data subset.

Pagination is important for improving performance and ensuring that large data sets are processed efficiently.

406 Rest 25: What is the difference between synchronous and asynchronous communication in RESTful web services?

When building RESTful APIs, how the client and server communicate matters. Communication can either be synchronous or asynchronous.

Synchronous Communication

In synchronous communication, the client sends a request and **waits for the server** to respond before doing anything else. The client is essentially “blocked” until the response arrives.

For example, when a client requests user information with GET /users/123, it waits until the server returns the user data. This approach works well for operations where the result is needed immediately, such as retrieving, updating, or deleting a resource.

Asynchronous Communication

In asynchronous communication, the client sends a request but **does not wait for the server** to respond. Instead, the client continues with other tasks. Later, it can check back for the response or receive a callback when the server has completed processing.

For instance, if a client requests generating a large report with POST /generate-report, the server might immediately return a job ID. The client can continue with other work and later query the status of the report or get notified when it is ready.

407 Rest 26: What are the best practices while designing the REST API?

Designing a RESTful API involves following several best practices. These practices ensure a reliable, scalable, and easily understandable RESTful API design:

- Use meaningful and consistent resource names for the API endpoints.
- Use HTTP verbs (GET, POST, PUT, DELETE) correctly to perform the corresponding actions on the resources.
- Ensure that the API is stateless and authentication is handled using tokens or OAuth.

- Use descriptive error messages and correct HTTP status codes for responses.
- Versioning the API can help manage changes over time.
- Lastly, document the API comprehensively, including endpoint details, request and response formats, and examples.

408 Rest 27: How do you document the APIs and their details?

When building RESTful APIs, proper documentation is crucial. It helps developers understand how to use the API, what endpoints are available, what parameters are required, and what responses to expect. One of the most popular tools for this purpose is **Swagger**, now known as **OpenAPI**.

Swagger allows developers to describe APIs in a structured JSON or YAML format. This description includes all the endpoints, request parameters, response formats, authentication methods, and even error codes. Once the API is described, Swagger can automatically generate human-readable documentation that is easy to navigate.

409 Rest 28: SOAP VS REST?

When designing web services, two popular approaches are SOAP (Simple Object Access Protocol) and REST (Representational State Transfer). While both allow applications communicate over the web, they differ in philosophy, design, and use cases.

1. SOAP (Simple Object Access Protocol)

- **Protocol-based**: SOAP is a strict protocol with specific rules and standards.
- **Message Format**: Uses **XML** for both requests and responses.
- **Features**: Built-in error handling, security (WS-Security), transactions, and ACID compliance.
- **Transport**: Usually HTTP, but can also use SMTP, TCP, or JMS.
- **Use Case**: Enterprise applications requiring high security, reliable messaging, and formal contracts (like banking, payment gateways).

2. REST (Representational State Transfer)

- **Architectural style**: REST is not a protocol; it is a set of principles for designing APIs.
- **Message Format**: Uses **JSON** (most common) or XML.
- **Features**: Lightweight, stateless, supports CRUD operations via standard HTTP methods (GET, POST, PUT, DELETE).
- **Transport**: Only HTTP/HTTPS.
- **Use Case**: Web and mobile applications requiring fast, scalable, and flexible APIs (like social media apps, e-commerce platforms)

Additional Question:

Why do financial services prefer SOAP over rest?

Strong Contracts: SOAP uses WSDL (Web Services Description Language) which is like a strict contract between client and server. In finance, this ensures no ambiguity in request/response formats.

Built-in Security: SOAP supports WS-Security (signing, encryption, authentication) out of the box. REST relies on external mechanisms like OAuth, JWT, or HTTPS.

Transaction Support: SOAP has standards for things like ACID transactions (WS-AtomicTransaction), which is critical for money transfers where operations must succeed or fail together.

410 Rest 29: How do you invoke the other APIs or Inter-service communication in a microservice in Spring?

In modern applications, it is common for any service to call other APIs, whether internal microservices or third-party services. Spring provides simple, flexible ways to make these HTTP requests and handle responses.

1. Using RestTemplate

RestTemplate is the classic way to call REST APIs in Spring. It is synchronous, meaning the call will block until a response is received.

```
@Autowired
private RestTemplate restTemplate;

public String getUserData() {
        String url = "https://api.example.com/users/123";
        return restTemplate.getForObject(url, String.class);
}
```

2. Using WebClient

WebClient is part of Spring WebFlux and supports reactive, non-blocking calls. It is the preferred approach for modern, scalable applications.

```
@Autowired
private WebClient webClient;

public Mono<String> getUserData() {
        return webClient.get()
                                        .uri("https://api.example.com/users/123")
                                        .retrieve()
                                        .bodyToMono(String.class);
}
```

Mono<String> represents a single asynchronous value.

retrieve() handles the HTTP response.

Point to remember:

Spring has announced RestTemplate is in maintenance mode. That means:

- It will still work in current and future Spring versions but No new major features are planned.
- Spring recommends using WebClient (from Spring WebFlux) for new projects because it supports non-blocking, reactive calls and is more flexible for modern applications.

411 Rest 30: How do you implement search functionality?

1. Using Query Parameters

The most common way. We pass search criteria as query parameters in the URL.

Example**:** Search users by name and city:

```
@GetMapping("/users")
        public List<User> searchUsers(
        @RequestParam(required = false) String name,
                @RequestParam(required = false) String city) {
        return userService.searchUsers(name, city);
}
```

Request looks like: GET /users?name=CodingLyf&city=Hyderabad

2. Using Request Body (for complex searches)

For more advanced filters (ranges, multiple fields, or nested conditions), we can send a POST with a search payload.

```
@PostMapping("/users/search")
public List<User> searchUsers(@RequestBody UserSearchCriteria criteria) {
        return userService.searchUsers(criteria);
}
```

Request payload:

```
{
    "name": "CodingLyf",
    "city": "Hyderabad",
  "ageMin": 25,
    "ageMax": 35
}
```

Spring Scenario Based Questions

412 Spring Scenario 1: Get by User ID or Email.

You are designing an API to fetch the user details. How would you like to fetch, will you use user- id or user-email and why?

Explanation: User ID is preferred.

Why?

- Every user has a unique ID, usually an auto-generated integer or UUID. There's no risk of collision.
- Database queries by primary key (like userId) are extremely fast.
- IDs don't reveal personal information, unlike emails.

413 Spring Scenario 2: Debug the slow API.

Imagine there is an existing API but it takes too long to respond. How you debug it?

1.Check if it is really slow

First, confirm the API is slow. Use **Postman** or **cURL** and measure the response time. If it is only slow in the browser, it might be a frontend issue. If Postman also shows slowness, then it is the
backend.

2. Check Latency.

- Network latency (time taken to reach the server).

  We can use "curl -w" command to check the network latency

- Server processing time (time spent inside the backend).

We place loggers to check the server processing time.

```
@GetMapping("/users")
public List<User> getUsers() {
  long start = System.currentTimeMillis();

  List<User> users = userService.getUsers();

  long end = System.currentTimeMillis();
  log.info("Server processing took {} ms", (end - start));

    return users;
}
```

- Database or external calls (time spent waiting for other services). We place loggers at repository to check the DB processing time.

```
long dbStart = System.currentTimeMillis();
List<User> users = userRepository.findAll();
long dbEnd = System.currentTimeMillis();

log.info("Database call took {} ms", (dbEnd - dbStart));
```

We can use commons-lang3 **StopWatch** instead of loggers

4. Look at server-side profiling

- We can use an APM (Application Performance Management) tool (like New Relic, Spring Boot Actuator).
- These tools show a detailed breakdown: how much time was spent in controller, service, repository, etc.

5. If server processing is slow, we might need to optimize the code.

6. If DB processing is slow, we need to optimize the query. This is how we debug the API.

414 Spring Scenario 3: Performance optimization.

You have found an API is very slow and you need to optimize it, what are the steps you are going to take?

1. Check loops: Inefficient loops can slow down the process for example, looping over thousands of records in Java when the database could filter them.

```
//Bad: filtering in Java
List<User> users = userRepository.findAll();
List<User> active = new ArrayList<>();
for (User u : users) {
      if (u.isActive()) {
     active.add(u);
  }
}
```

Better approach: push filtering to DB

2. Creating objects inside a loop slows things down

```
for (int i = 0; i < 100000; i++) {
        String s = new String("Hello"); // unnecessary new object
}
```

3. Use Asynchronous Processing: Use asynchronous processing for long-running tasks to improve responsiveness

4. Caching: Implement caching to reduce database load and improve response times

5. Pagination: For large datasets implement the pagination (Refer Question No.)

6. Optimize the Queries

```
-- Bad
SELECT * FROM users;

-- Better (only fetch what you need)
SELECT id, name FROM users;
```

7. Have proper indexes: If we query on a column without an index, DB scans the whole table (Refer Question No. 372, Spring 120)
8. **Address the N+1 Problem:** Fetching the data inside a loop can make N number of queries. Try to fix using JOIN Fetch (Refer Question No 303, 304, Spring 51,52).

9. Connection Pooling: Use connection pooling to manage database connections efficiently.

415 Spring Scenario 4: DB Migrations

You need features to an existing Spring Boot application, which involves changing the database schema. How do you ensure smooth database migrations?

Steps:

1. To handle database migrations, we can use migrations tools like Flyway or Liquibase.

2. Write SQL changes in separate files and place them in `src/main/resources/db/migration`:

- V1_create_users_table.sql
- V2_add_email_to_users.sql

The numbering (V1, V2, V3) ensures order, and the tool keeps track of what's already been applied.

3. Spring Boot can run Flyway or Liquibase migrations at startup. That means whenever the app boots, it checks which migrations are pending and applies them. This keeps development and test environments in sync automatically.

4. Always run them in a staging database first. This helps catch performance issues (like adding an index on a huge table) or mistakes (like dropping a column that is still used).

5. It always good have rollback mechanism in place. Tools like Liquibase useful to define rollback scripts. Flyway doesn't encourage rollbacks but suggests creating a new "fix" migration if needed.

416 Spring Scenario 5: File upload and download

You are designing an APIs for upload and download the files; how would you achieve this?

Explanation:

For file upload, we need to use MultipartFile. It represents the file sent in a multipart/form-data request.

- Client sends file via HTTP POST request.
- Server receives it as MultipartFile.

- Server processes and stores the file (in DB, cloud, or local storage).
- Return a success response.

```
@PostMapping("/upload")
ublic ResponseEntity<String> uploadFile(@RequestParam("file") MultipartFile file) {
        try {
                // Save to disk (demo purpose)
                Path path = Paths.get("uploads/" +
        file.getOriginalFilename()); Files.write(path, file.getBytes());



        return ResponseEntity.ok("File uploaded successfully: " +
file.getOriginalFilename());
        } catch (IOException e) {
                return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                                                        .body("Could not upload
                        file");
        }


  @GetMapping("/download/{fileName}")
  public ResponseEntity<Resource> downloadFile(@PathVariable String fileName) {
          try {
                  Path path = Paths.get("uploads/" +
          fileName); Resource resource = new
          UrlResource(path.toUri());

                  if (!resource.exists()) {
                          return ResponseEntity.notFound().build();
                  }

                  return ResponseEntity.ok()
                                  .contentType(MediaType.APPLICATION_OCTET_STREAM)
                                  .header(HttpHeaders.CONTENT_DISPOSITION,
                                                  "attachment; filename=\"" +
                          resource.getFilename() + "\"")
                                  .body(resource);

          } catch (Exception e) {
                  return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
          }
```

For File downloads, two headers matter:

- Content-Disposition: tells the browser to download instead of preview.
- Content-Type : defines file type (application/pdf, application/vnd.ms-excel, etc.).

Sample URL downloads the file: http://localhost:8080/download/report.pdf

Additional Questions.

1. What is the maximum file size that spring accepts?

By default, Spring Boot accepts file uploads up to **1MB** (per file) and **10MB** (per request).

2. How do you increase the acceptable file size ?

You can increase this by setting properties like spring.servlet.multipart.max-file-size and spring.servlet.multipart.max-request-size in application.properties.

```
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

3. What is the maximum file size that Spring can accept?

Spring itself doesn't impose any maximum size, but very large uploads are limited by memory, disk, and timeout constraints.

417 Spring Scenario 6: Asynchronous Processing

Suppose you need to a large number of data and save data in DB. It is very long running task. How would you handle it?

Use a background worker

This is where tools like **Spring Batch** or **@Async with ExecutorService** comes into picture.

- Spring Batch is built exactly for this kind of job like chunked reading, processing, and writing to DB.
- If we don't want the overhead of Spring Batch, we can write our own async task using @Async (Refer Question no.351 Spring 99) or even put it on a **queue system** (like RabbitMQ, Kafka).

418 Spring Scenario 7: Handling Configuration

Suppose you need to develop an APP for different environments where each environment has its own configurations like database and endpoints. How would you configure it?

Explanation:

When the app runs in different environments (development, testing, production), each one has its own settings: different databases, different API endpoints.

Spring Boot provides a flexible way to manage configuration properties using **application.properties** or **application.yml** files.

1. Create Environment-Specific Property Files

For each environment, you create a dedicated file inside src/main/resources.

application-dev.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/devdb
spring.datasource.username=devuser
spring.datasource.password=devpass
```

application-prod.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/proddb
spring.datasource.username=produser
spring.datasource.password=prodpass
```

2. Activate the Right Profile

We tell Spring Boot which environment to use by setting the **active profile**. This can be done in different ways:

Option A: Inside application.properties, use this spring.profiles.active=dev

Option B: java -jar app.jar --spring.profiles.active=prod (for prod)

3. Inject Properties.

Use @ConfigurationProperties, this annotation helps us to map the entire content of the properties file to a POJO (Plain Old Java Object). A property file can be either application.properties or application.yml.

```
@Configuration
@ConfigurationProperties(prefix = "spring.datasource")
    public class
    DataSourceConfig { private
    String url; private String
    username; private String
    password;

        // Getters and Setters
```

Spring will automatically map the right values depending on the active profile.

4. @Profile for Environment-Specific Beans

Sometimes, you want different beans for different environments.
Example: a dummy email service in dev, and a real one in prod.

```
public interface EmailService {
        void send(String to, String msg);
}

@Profile("dev")
@Service
public class DummyEmailService implements EmailService {
        public void send(String to, String msg) {
                System.out.println("DEV mode: Email not really sent to " + to);
        }
}

@Profile("prod")
@Service
public class RealEmailService implements EmailService {
        public void send(String to, String msg) {
                // actual implementation
        }
}
```

419 Spring Scenario 8: Apply configuration without reboot

You are running a Spring Boot application in production. One day, your manager comes and says: “We need to change the log level to DEBUG for one application/service to investigate an issue, but you should not restart the service because it is handling live traffic.”

In other words, you must **apply configuration changes without rebooting** the application.

Explanation:

Normally, Spring Boot reads its configuration from application.properties or application.yml files at startup. So, if we change them, we need to restart the app

To solve this, Spring provides ways to dynamically refresh configurations without restarting**.**

To do this we need to follow these things:

1. Store configurations centrally in a separate Git repo.
2. Set up a Spring Cloud Config Server
3. Connect your main applications to Config Server
4. Enable actuator > refresh API

1. Store configurations centrally in a separate Git repo.

Create a Git repository (say config-repo). Inside it, place environment-specific property files:

```
application-dev.properties
application-test.properties
application-prod.properties
```

2. Set up a Spring Cloud Config Server

Create a new Spring Boot app with @EnableConfigServer and add

```
<dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

Point it to the Git repo (config-repo) in application.properties:

```
spring.cloud.config.server.git.uri=https://github.com/org/config-repo
```

```
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer


public class ConfigServer {
            public static void main(String[] args) {
        SpringApplication.run(ConfigServer.class, args);
        }
}
```

@EnableConfigServer. This annotation tells Spring Boot to enable the Config Server functionality for the application.

3. Connect your main applications to Config Server

- Add spring-cloud-starter-config dependency
- To connect the application to config server, we need to give below property pointing to the config server URL.

```
spring.cloud.config.uri=http://config-server:8888
spring.profiles.active=prod
```

4. Enable actuator > refresh API:

- Add `spring-boot-starter-actuator` dependency.
- Annotate beans that should reload on config change with @RefreshScope. Example:

```
@RestController
@RefreshScope
    public class ConfigController {
    @Value("${featureX.enabled}")
    private boolean featureEnabled;

        @GetMapping("/feature-status")
        public String featureStatus() {
                return "Feature X is " + (featureEnabled ? "ENABLED" : "DISABLED");
        }
}
```

The @RefreshScope annotation in Spring Cloud is used to enable dynamic refreshing of configuration properties within a Spring Boot application without requiring a full application restart.

How it works?

- Suppose you want to disable the feature in prod.
- Edit application-prod.properties in config-repo branch: `featureX.enabled=false`
- Commit and push changes to Git.
- Go to main running app.
- Call the actuator refresh endpoint:
- POST `http://main-app:8080/actuator/refresh`
- Spring reloads properties from Config Server into your beans without reboot.

Reference: Complete Guide to Spring Cloud Config | Medium

420 Spring Scenario 9: Large file handling.

Client might upload a large file through the API. How would you handle this to avoid memory issues?

Option 1: Streaming Uploads

Instead of loading the whole file in memory, stream the data chunk by chunk and @Async to process it in the background.

```
@PostMapping("/upload-large")
public ResponseEntity<String> uploadLargeFile(HttpServletRequest request) {
        try (ServletInputStream inputStream = request.getInputStream();
                OutputStream outputStream = new
         FileOutputStream("uploads/largefile.bin")) {

                byte[] buffer = new byte[8192]; // 8KB buffer
                int bytesRead;
                    while ((bytesRead =
            inputStream.read(buffer)) != -1) {
            outputStream.write(buffer, 0, bytesRead);
                }
                return ResponseEntity.ok("File uploaded successfully (streamed).");
        } catch (IOException e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Upload
failed.");
```

Option 2: Direct-to-Cloud Uploads

- In many production systems, we don't use REST API handle GBs of data directly.
- Instead, we issue a pre-signed URL from S3, GCS, or Azure.

- Client uploads file directly to cloud while your API just coordinates.

Sample code for GCS (Google cloud storage):

```
import com.google.cloud.storage.*;
import org.springframework.web.bind.annotation.*;
import java.net.URL;
import java.util.concurrent.TimeUnit;

@RestController
@RequestMapping("/files")
public class FileController {

        private final Storage storage;

        public FileController(Storage storage) {
                this.storage = storage;
        }



        @GetMapping("/presigned-url")
        public String getPresignedUrl(@RequestParam String fileName) {
                BlobInfo blobInfo = BlobInfo.newBuilder("my-bucket-name",
        fileName).build();

                        URL signedUrl =
                storage.signUrl( blobInfo,
                                15, TimeUnit.MINUTES,      // URL
                valid for 15 minutes
                Storage.SignUrlOption.httpMethod(HttpMethod.PUT),
                Storage.SignUrlOption.withV4Signature()
                );

                return signedUrl.toString(); // return signed URL to client
```

- The API generates a signed URL valid for 15 minutes.
- Clients use this URL to upload directly to GCS with an HTTP PUT request.
- Our server never touches the file

bytes. Client Upload (Example with cURL)

```
curl -X PUT --upload-file ./bigfile.zip \
"https://storage.googleapis.com/my-bucket-name/bigfile.zip?X-Goog-Algorithm=GOOG4-RSA-SHA256&X-Goog-Credential=..."
```

421 Spring Scenario 10: Caching.

Your application is having performance issues due to frequent queries to Database. How would you cache the data

Explanation: Spring boot provides various caching solutions such as EhCache, Redis, Hazelcast and In- memory (for small data).

We need to use these Annotations: @EnableCaching, @Cachable, @CachePut, @CacheEvict.

@EnableCaching:

This annotation is placed on a configuration class or the main application class to enable Spring's annotation-driven cache management.

@Cacheable:

This annotation marks a method whose result can be cached. When a method annotated with @Cacheable is invoked, Spring checks if the result for the given arguments is already present in the cache. If found, the cached value is returned, avoiding method execution. If not found, the method is executed, and its result is stored in the cache for future use.

@CachePut:

This annotation is used to update the cache with the new result of a method execution without skipping the method execution itself.

Unlike @Cacheable, @CachePut always executes the method and then places its return value into the cache, ensuring the cache is refreshed with the latest data.

@CacheEvict:

This annotation is used to remove one or more entries from a cache. It is typically applied to methods that modify data, ensuring that stale cached data is removed after an update or deletion operation. This forces subsequent requests for that data to fetch it from the underlying source (e.g., database), ensuring data consistency.

422 Spring Scenario 11: Validations.

Imagine you have a POST API that accepts user details. If the client sends bad data (like missing required fields or too-short passwords), your API should validate, reject gracefully, and return meaningful errors.

Explanation: Spring Boot makes this simple with **Bean Validation (JSR-380)** + annotations like @NotNull, @Size, @Email.

Step 1: Add Validation Dependency

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Step 2: Create a DTO with Validation Annotations

```
import jakarta.validation.constraints.*;
    public class UserRequest {
    @NotBlank(message = "Name is required")
        private String name;

        @Email(message = "Email must be valid")
    @NotBlank(message = "Email is required")
    private String email;

        @Size(min = 8, message = "Password must be at least 8 characters")
        private String password;

        // getters and setters
}
```

Step 3: Use @Valid in Controller

```
@RestController
@RequestMapping("/users")
```

```
public class UserController {

        @PostMapping
    public ResponseEntity<String> createUser(@Valid @RequestBody
UserRequest userRequest) {
                return ResponseEntity.ok("User created: " + userRequest.getName());
        }
}
```

Here:

- @Valid triggers validation on the incoming JSON.
- If validation fails, Spring throws MethodArgumentNotValidException.
- Then catch the exception with @ExceptionHandler

423 Spring Scenario 12: Scheduling the task.

Imagine you are building an e-commerce application. Every midnight, you need to generate a sales report and send it to management. Doing this manually is painful and error-prone. How would you achieve this?

Explanation:

Spring has a mechanism to schedule the tasks using @EnableScheduling and @Scheduled(Refer Question no. 347,348, Spring 95,96)

1. We need tell Spring to turn on the scheduler by using @EnableScheduling.

2. Now, let's see the job that will run every night at midnight.

```
@Component
public class SalesReportScheduler {

        @Scheduled(cron = "0 0 0 * * ?") // Every day at midnight
        public void generateDailySalesReport() {
                System.out.println("Generating sales report... " + LocalDateTime.now());
                // logic: fetch sales data, generate PDF, send email
        }
}
```

Cron: **Cron expression generator by Cronhub**

3. Spring will handle them in the background using a default thread pool. If we need a custom thread pool, configure a TaskScheduler bean.

424 Spring Scenario 13: Dependency Injection.

Imagine you have a **UserService** that handles all the logic for fetching users, creating users, updating users, etc. Now, multiple controllers in your application (say AdminController and CustomerController) need to use this same service. How do you make it work cleanly in Spring?

1. Mark the service as a Spring Bean

- We do this with @Service (or @Component).
- This tells Spring to manage the lifecycle of this service and make it available in the application context.

```
@Service
public class UserService {
        public String getUserDetails() {
                return "User details from service";
        }
}
```

2. Inject the service into controllers

- Since the service is now a bean, we don't need to new it manually.
- We just declare a dependency using @Autowired (or constructor injection, which is the better practice).

Example in AdminController:

We don't have to write any extra code.

Spring sees UserService in the context, sees that both controllers need it, and injects the same bean instance into both places (because by default, Spring services are **singleton scoped**).

425 Spring Scenario 14: Custom Bean.

Imagine You are building an **e-commerce application**. You need a **utility class** to calculate discounts. it is just a reusable piece of business logic , Like DiscountCalculator.

How would you inject into to the Spring context?

Explanation:

```
@RestController
@RequestMapping("/admin")
public class AdminController {

        private final UserService userService;

        public AdminController(UserService userService) {
                this.userService = userService;
        }

        @GetMapping("/user")
        public String getUser() {
                return userService.getUserDetails();
        }
}
```

Since it is a utility class, it would be idle to use @Bean in Configuration class.

```
public class DiscountCalculator {
@Configuration
public class AppConfig {

        @Bean
        public DiscountCalculator discountCalculator() {
                return new DiscountCalculator();
        }
}
```

This is a plain Java class without any annotation. And then we can inject like this@Configuration tells Spring this class provides bean definitions.

@Bean tells Spring "whenever someone needs DiscountCalculator, use this method to create and manage it."

Additional Question:

Why don't you use @Service or @Compnent to DiscountCalculator?

Service and @Component are meant for general Spring-managed beans. Generally, @Service class is to maintain the business logic. DiscountCalculator is more like a utility/helper without business logic. So, annotating it with @Service or @Component adds no real value here.

Common examples are

- RestTemplate: we usually create it as a @Bean in a config class so we can reuse the same instance across the app.
- DataSource : Defined as a @Bean to configure database connections (URL, username, pool size).
- ObjectMapper: Registered as a @Bean when we want custom JSON serialization/deserialization rules.

426 Spring Scenario 15: Inject bean on condition.

You have two payment services, one is for CreditCardService and one is for UPIService. You want to inject only service one depending on config.

Explanation: We need to use @ConditionalOnProperty to achieve this (Refer Question No 278, Spring 26).

Example:

```
public interface PaymentService {
        void pay(double amount);
}


@Service
@ConditionalOnProperty(name = "payment.type", havingValue = "card")
class CardPaymentService implements PaymentService {
            public void pay(double amount) {
        System.out.println("Paid by Card: " + amount);
        }
}

@Service
@ConditionalOnProperty(name = "payment.type", havingValue = "upi")
class UpiPaymentService implements PaymentService {
            public void pay(double amount) {
        System.out.println("Paid by UPI: " + amount);
        }
}
```

Only one bean is created at runtime, depending on the condition (payment.type in application.properties file). This avoids conflicts and makes bean injection flexible.

Additional Question:

1. Why can't you use @Profile.

@Profile will be used for environment specific like to load beans based on environment (Dev or Prod)

427 Spring Scenario 16: Inject bean on multiple conditions.

Suppose your app needs to connect to different message brokers (like Kafka or RabbitMQ) but only under certain conditions:

- Broker type should match the property value from application.properties(broker.type).
- The app should also be running on a specific OS (e.g., Linux

only). So, we want the bean created only when multiple conditions

are satisfied. **Explanation:**

We need use @ConditionalOnExpression or custom Condition class

```
@Bean
@ConditionalOnExpression("'${broker.type}' == 'kafka' and
'${os.name}'.contains('Linux')")
public MessageBroker kafkaOnLinux() {
        return new KafkaBroker();
}
```

Suppose if we have more complex logic that can't be resolved in @ConditionalOnExpression and same condition is being used in multiple times , then we need to go with custom Condition class

```
    public class LinuxAndPropertyCondition implements Condition
    { @Override
            public boolean matches(ConditionContext context, AnnotatedTypeMetadata
        metadata) { String os = System.getProperty("os.name").toLowerCase();
                String brokerType = context.getEnvironment().getProperty("broker.type");
                return os.contains("linux") && "kafka".equalsIgnoreCase(brokerType);
        }
}
```

And we will have to use @Conditional to trigger the custom Condition class.

```
@Bean
@Conditional(LinuxAndPropertyCondition.class)
public MessageProducer kafkaProducer() {
        return new KafkaProducer();
}
```

428 Spring Scenario 17: Dynamic bean selection, based on parameter from client.

Imagine you are building a payment service. Your app supports multiple payment gateways like PayPal, Stripe, and Razorpay.

- At runtime, the client request parameter (say paymentMode=paypal) should decide which bean (payment processor) to use.
- You cannot solve this with @Profile or @ConditionalOnProperty because that work only at application startup based on config/environment.

Means, when client send paypal (POST /pay?type=paypal&amount=100), paypal service should be invoked.

Explanation:

In this case, we inject all candidate beans and pick one dynamically at runtime. Spring won't create/destroy beans per request, but we can route requests to the right bean.

We can achieve this using, Strategy Pattern + @Autowired Map or List

- Spring injects all beans implementing an interface.
- You select the right one using the client parameter.

Strategy pattern means: instead of writing many if-else for different logics, we keep each logic in a separate class and pick the one we need at runtime. This makes code clean, flexible, and easy to change without touching the main class.

Step 1: Define Interface:

```
public interface PaymentProcessor {
        void processPayment(double amount);
}
```

Step 2: Implement multiple beans

```
@Component("paypal")
class PaypalProcessor implements PaymentProcessor {
            public void processPayment(double amount) {
        System.out.println("Paid " + amount + " via PayPal");
        }
}

@Component("stripe")
class StripeProcessor implements PaymentProcessor {
            public void processPayment(double amount) {
        System.out.println("Paid " + amount + " via Stripe");
        }
}
class PaymentService {
        private final Map<String, PaymentProcessor> processors;

        public PaymentService(Map<String, PaymentProcessor> processors) {
                this.processors = processors;
        }

        public void pay(String type, double amount) {
                PaymentProcessor processor = processors.get(type.toLowerCase());
                if (processor == null) {
                        throw new IllegalArgumentException("Invalid payment type: " +
    type);
                }
                processor.processPayment(amount);
        }
```

Step 3: Use Map Injection for dynamic selection

Step 4: Controller

```
@RestController
class PaymentController {
        private final PaymentService paymentService;

        public PaymentController(PaymentService paymentService) {
                this.paymentService = paymentService;
        }



        @PostMapping("/pay")
            public void pay(@RequestParam String type, @RequestParam double amount) {
        paymentService.pay(type, amount);
        }
}
```

Now On POST call /pay?type=paypal&amount=100 , Spring routes to PaypalProcessor.

Reference: Dynamic Bean Switching in Spring Boot ! No If-Else Hell @Javatechie

429 Spring Scenario 18: Load initial cache data.

Suppose, you have a CacheService which will load some data into cache. How would you achieve this, will you use @PostConstruct or CommandLineRunner.

Explanation:

@PostConstruct is useful when we want to run some initialization logic right after Spring creates and injects the bean, without needing a CommandLineRunner or ApplicationRunner. It Runs right after bean creation and dependency injection is done. Best when we need to initialize resources

CommandLineRunner: Runs after the whole Spring context is ready. Best when we need application- wide startup logic. Example: inserting the database

Why can't we use @Postcontruct to insert into the DB?

- @PostConstruct runs right after the bean is created, before the full Spring Boot context is ready.
- At that time, all beans (like EntityManager, TransactionManager, Repositories) may not be fully available. This can cause errors or missing transaction issues when we try inserting data.
- On the other hand, CommandLineRunner or ApplicationRunner run after the entire Spring context is ready, meaning our DB connections, repositories, and transactions are fully initialized. Perfect for inserting.

430 Spring Scenario 19: Logging in Spring boot

You need to add logging to your application to track its behaviour and troubleshoot issues. How do you implement logging in Spring Boot?

Spring Boot provides support for various logging frameworks such as Logback, Log4j2, and SLF4J.

SLF4J

SLF4J stands for Simple Logging Facade for Java.

It is not a logging implementation. It is a facade means a common API that routes all the logging calls to a real logging backend like Logback or Log4j.
Think of SLF4J as: "The common interface the code uses to log, while the actual logging engine does the actual work."

Logback

It is the default logging implementation in Spring Boot.
Created by the same author as Log4j.
Features: faster startup, good async support, easy configuration via logback-spring.xml.

Log4j2

Successor of Log4j (fixed performance & security issues).

Offers advanced features like asynchronous logging (much faster in high-load apps), garbage-free logging (less memory churn).

Additional Questions:

1. Why can't you use simple System.out.println()?

System.out.println() is slow because it writes directly to the console (standard output) synchronously, meaning the main thread waits for each print to complete before moving on.

No log levels: We can't distinguish INFO, DEBUG, WARN, or ERROR easily, with logging frameworks, we can filter messages by level.

2. How would you ensure logging is a singleton in multithreaded programming

In practice, we use existing logging frameworks (SLF4J, Log4j, Logback) since they already guarantee singleton behaviour internally.

Example:

431 Spring Scenario 20: Monitoring the Spring application

Your application is in production, and you need to monitor its health, metrics, and logs to ensure it is performing optimally.

```
public class Logger {
        private Logger() {}
        private static class Holder {
                private static final Logger INSTANCE = new Logger();
        }
        public static Logger getInstance() {
                return Holder.INSTANCE;
        }
            public void log(String msg) {
           System.out.println(msg);
        }
}
```

Explanation:
Spring Boot Actuator provides a powerful set of tools for monitoring and managing applications. Add Actuator Dependency.

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Enable Actuator Endpoints: management.endpoints.web.exposure.include=*

Access Actuator Endpoints.

- Health Check: http://localhost:8080/actuator/health
- Metrics: http://localhost:8080/actuator/metrics
- Environment: http://localhost:8080/actuator/env

We can also integrate with external monitoring tools like Prometheus and Grafana for more advanced monitoring capabilities.

432 Spring Scenario 21: Post idempotency

A user clicks Place Order multiple times quickly. If your backend just accepts every request and creates an order record each time, you will get duplicate orders.

But HTTP POST is not idempotent by design (every call can cause a new resource).
So you need to make POST "behave idempotently" in your scenario. How do you achieve it?

Solutions

1. Idempotency Key

- The client (e.g., frontend or mobile app) generates a unique key (like UUID) for every "intent to place order".
- It sends this key in the request header or body:
- POST /orders Headers: Idempotency-Key: abc123
- The server stores this key.
    - If the same key comes again, it returns the same order response instead of creating a new one.

2. Database Constraints (Transactional Safety)

- Put a unique constraint in DB (e.g., user_id + cart_id must be unique).
- If user clicks multiple times, the DB prevents duplicates.
    - The backend can catch the DB exception and return the existing order.

433 Spring Scenario 22: Custom Annotation + Interceptor

Imagine in your application there was a custom annotation called @Adminonly.

Now You need to make sure, if any delete/* API is not annotated with, should reject the request. Kind of centralized check for delete APIs.

Solution:

We have already had custom annotation.

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface AdminOnly {

}
```

1. Create Interceptor

In Pre Handle method.

```
@Component
	public class DeleteApiInterceptor implements HandlerInterceptor {
	@Override
	public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) {
			if (handler instanceof HandlerMethod handlerMethod) {
				AdminOnly adminOnly =
		handlerMethod.getMethodAnnotation(AdminOnly.class);
				if (adminOnly == null) {
					// Check if user has ADMIN role
			response.setStatus(HttpServletResponse.SC_FORBIDDEN);
			return false;
				}
			}
			return true;
		}
```

2.Register the Interceptor

```
@Configuration
        }
}
        private final DeleteApiInterceptor deleteApiInterceptor;

        public WebConfig(DeleteApiInterceptor deleteApiInterceptor) {
                this.deleteApiInterceptor = deleteApiInterceptor;
        }

        @Override
            public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(deleteApiInterceptor)
                                .addPathPatterns("/delete/*"); // Apply only to
```

3. Controller

```
@RestController
@RequestMapping("/api/v1/users")
public class UserController {

        // This delete API requires @AdminOnly
    @DeleteMapping("/{id}")
        @AdminOnly
        public ResponseEntity<String> deleteUser(@PathVariable Long id) {
                // Imagine delete logic here
                return ResponseEntity.ok("User with ID " + id + " deleted
        successfully");
        }

        // This endpoint does not have
    @AdminOnly @GetMapping("/{id}")
        public ResponseEntity<String> getUser(@PathVariable Long id) {
                // Just dummy return
                return ResponseEntity.ok("Fetched user with ID " + id);
        }
```

We have registered the DeleteApiInterceptor so that it runs for every request under delete/*.

Inside the interceptor, we use HandlerMethod to check if the controller method handling this request is annotated with @AdminOnly.

- If the method **has the annotation**, the request continues as normal.
- If the method **does not have the annotation**, we block the request and return a 403 Forbidden.

This way, even though the interceptor is invoked for all delete/* APIs, only the methods explicitly marked with @AdminOnly are allowed to execute.

434 Spring Scenario 23: Rate Limiting

You have a public API `/api/search` that clients use to query your system. Some clients start abusing it, sending hundreds of requests per second, which puts unnecessary load on your backend. You want to ensure:

1. Each client (based on IP or API key) can only make 5 requests per second.
2. If they exceed this limit, they should receive an HTTP 429 Too Many Requests.
3. This should not affect other clients.

We can achieve this using Custom Filter.

Code Link: Spring-Samples/RateLimiting at main · CodingLyf-Fullstack/Spring-Samples Steps

- Create a custom filter called RateLimitingFilter
- The RateLimitingFilter class implements the Filter interface to create a custom filter for rate limiting.
- A ConcurrentHashMap is used to store request counts per IP address for thread-safe operations.
- The doFilter method increments the request count and checks if it exceeds the defined limit. If so, it responds with a 429 Too Many Requests status.

FilterRegistrationBean

When we write a filter in Spring Boot, we usually extend OncePerRequestFilter or implement Filter. That filter can intercept every HTTP request and response.

But here is the problem:
By default, Spring Boot will pick up the filter and apply it globally to all the URLs. But when we want more control like deciding the order, Apply the filter to specific URL patterns (like /api/*) then we will use FilterRegistrationBean

```
@Bean
    public FilterRegistrationBean<CustomFilter> customFilter() {
    FilterRegistrationBean<CustomFilter> reg = new FilterRegistrationBean<>();
    reg.setFilter(new CustomFilter());
        reg.addUrlPatterns("/api/*"); // only apply to
    /api reg.setOrder(1); // set execution order return
    reg;
}
```

435 Spring Scenario 24: Custom Query method

You are building a Spring Data JPA repository and you want to fetch all users that belong to a certain email domain. For example, get everyone with an email ending in @gmail.com but without JPQL.

How would you do this?

Solution

Spring Data JPA has a feature called **Derived Query Method**s. Instead of writing the whole query, we just follow a naming convention in our repository method. Spring automatically interprets it into SQL/JPQL for us.

Example:

Repository:

```
    public interface UserRepository extends JpaRepository<User,
    Long> { List<User> findByEmailEndingWith(String domain);
}
```

When we call the method

```
List User> gmailUsers = userRepository.findByEmailEndingWith("@gmail.com");
```

Spring Data JPA translates it into something like:

```
SELECT * FROM user WHERE email LIKE '%@gmail.com';
```

436 Spring Scenario 25: Custom Query

You are building an application to manage Orders placed by Users. Now, you need to fetch all orders placed by users older than 25, where the order status is 'DELIVERED',

Solution

Here we need to write query that involves joins, filtering, or multiple conditions that we cannot achieve with Derived Query Method.

In Spring Data JPA, we can write custom queries in two ways:

1. JPQL (object-oriented query, works with entity names and fields), using @Query
2. Native SQL (direct database query)

Example:

Our Scenario involves 2 tables (User, Orders)

```
@Entity
    public class User
    { @Id
        private Long id;
    private String name;
    private int age;
}


@Entity
    public class
    Order { @Id
        private Long id;
        private String status;
        private double productPrice;

        @ManyToOne
        private User user;


}
```

So, a User can have many Orders. That is why Order holds a reference to User.

Now, derived queries (like findByStatus) won't help here, because we need conditions across two tables. This is where @Query comes into play

```
public interface OrderRepository extends JpaRepository<Order, Long> {

            @Query("SELECT o FROM Order o JOIN o.user
        u " + "WHERE u.age > 25 AND o.status =
        'DELIVERED'")
      List<Order> findDeliveredOrdersByUsersOlderThan25();
```

- JOIN o.user u: joins the Order table with its related User.
- u.age > 25: filters by user age.
- o.status = 'DELIVERED': filters by order status.

437 Spring Scenario 26: Lazy vs Eager / N+1 problem

You have two entities:

- User
- Order

A user can have many orders. Sometimes you just want the user's details (name, age, etc.). Other times, you also want their orders.

The question is: how do you design it so you don't always drag orders out of the database unnecessarily, but can still fetch them when you actually need them?

Solution:

First let's understand the relationships.

1. A User can have many orders (OnetoMany)
2. Many Orders can be placed by a user (ManytoOne)

Entities:

```
@Entity
    public class User
    { @Id
        private Long id;
    private String name;
    private int age;

        @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
        private List<Order> orders;
}
```

Notice fetch = FetchType.LAZY.

- This means: When I load a User, don't immediately pull in their Orders.
- Orders will only load when we call user.getOrders().

Now we we can get the user, only user will be returned. But when we want to fetch the orders, a separate query will be made to fetch orders.

```
User user = userRepo.findById(1L).get();
System.out.println(user.getName()); // Only User query runs

List<Order> orders = user.getOrders(); // triggers a separate query
```

If there are 100 users, for each user one order will be made, which creates N+1 problem (Refer Question No. 303,304 Spring 51,52);

Two solve this, we have two approaches.

1.Use JOIN FETCH: This fetches all the data in one query (Like a join query in SQL)

```
@Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
User findUserWithOrders(@Param("id") Long id);
```

2. Use @EntityGraph: This is cleaner, no custom query needed:

```
@EntityGraph(attributePaths = "orders")
@Query("SELECT u FROM User u WHERE u.id = :id")
User findUserWithOrders(@Param("id") Long id);
```

438 Spring Scenario 27: save() or saveAll()

You are building a system where you need to persist data into the database. Sometimes it is just 1 record (like a single order). Other times, it is 10000 records at once (say, importing bulk Orders from a file).

Do you use save() or saveAll()?

Solution:

- save() is used to save single record.
- saveAll() is used to save list of the records.

```
 List<User> users =
 List.of( new User(2L,
 "Coding", 30), new User(3L,
 "Lyf", 25)
);
```

But here is the catch. saveAll() internally runs for loop and save() each record. If there are 10000 records, it will loop 10000 times. Best technique to persist N number of records is by **batching**.

Hibernate can group those inserts into batches. All we need is to configure it:

```
spring.jpa.properties.hibernate.jdbc.batch_size=500
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

Now, if we call saveAll() with 10000 records:

- Instead of 10000 separate INSERTs, Hibernate will do them in batches of 500.
- That means only 20 round trips to the database. Much faster.

439 Spring Scenario 28: Partial Update

```
@Entity
    public class
    User { @Id
        private Long
    id; private String
    name; private int
    age; private String
    status;
```

You are working with a user entity that has multiple fields:

Now imagine the requirement:

You want to update only the status of a user (say from INACTIVE to ACTIVE) without fetching the entire User object first.

Step 1: The Problem

Generally, In JPA, first we get the record and update it. Which means two queries will run to update single field.

```
User user = userRepo.findById(1L).get(); // 1 query
user.setStatus("ACTIVE");
userRepo.save(user);      // another query (update)
```

Step 2: Solution.

To avoid select query we can @Modifying + @Query for partial updates.

```
public interface UserRepository extends JpaRepository<User, Long> {

        @Modifying
        @Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
        int updateUserStatus(@Param("id") String status, @Param("id") Long id);
}
```

- @Modifying: tells Spring this is an update/delete, not a select.
- JPQL UPDATE query updates only the status field.
- int return value: number of rows updated.

Point to remember: We can achieve Delete also with @Modifying. We can also use it for bulk updates where can update the record without fetching it first

440 Spring Scenario 29: Soft Deletes

Instead of deleting, you need to mark a record as inactive How to implement soft delete in JPA?

Solution:

Step1: Add a field in the entity.

```
@Entity
    public class User
    { @Id
        private Long id;
    private String name;
    private int age;
        private boolean deleted = false; // soft delete flag
}
```

Now instead of delete we can run update using

@Modifying. Or we can use @SQLDelete

The **@SQLDelete** annotation is a Hibernate-specific annotation (not part of the standard JPA specification) which is useful to **override the default SQL statement** that Hibernate uses to delete an entity from the database.

Instead of the standard DELETE FROM table_name WHERE id = ?, we can define a custom SQL query to be executed when entityManager.remove() is called.

We define @SQLDelete in our entity like this:

```
@Entity
@SQLDelete(sql = "UPDATE user SET deleted = true WHERE id = ?")
@Where(clause = "deleted = false")
    public class User
    { @Id
        private Long id;
    private String name;
    private int age;
        private boolean deleted = false;
}
```

When we run userRepo.deleteById(1); hibernate actually runs the Update query.

```
update user set deleted = true where id = 1;
```

441 Spring Scenario 30: Auditing

You are building an Orders System. Every order has details like product, price, and status. We need to know when each order was created and when it was last updated. How would you achieve this?

Solution:

The naive way is to add two fields: createdDate and updateDate and then, every time we save an entity, set them manually:

```
@SpringBootApplication@EnableJpaAuditing      // switch ON auditing
public class App {

}
```

```
    @Entity
    @EntityListeners(AuditingEntityListener.class)
    public class Order {

            @Id
        @GeneratedValue
        private Long id;

            private String product;

            @CreatedDate
        @Column(updatable = false)
            private LocalDateTime createdDate;

            @LastModifiedDate
            private LocalDateTime updatedDate;
    }
```

But Spring Data JPA has built-in support for auditing using annotations:

- @CreatedDate: set once when the entity is created.
- @LastModifiedDate: updated every time the entity changes.

And all of this works by simply enabling auditing (Refer Question No. 361 Spring 109).

Example:

Now when we call orderRepo.save(order); `createdDate` and `updatedDate` will be saved automatically.

442 Spring Scenario 31: Handling null values

User is trying to fetch the details of the book that is not exist in the database. how would you gracefully handle this in Spring?

Approach:

- JPA uses Optional when fetching by ID.
- If the book is not found, throw a **custom exception**.
- Use @ControllerAdvice and @ExceptionHandler to return a proper error

response. Sampel code

Create Entity:

```
@Entity
    public class
    Book { @Id
        private Long id;
    private String title;
    private String author;
}
```

Create Repository:

```
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

Custom Exception:

```
public class BookNotFoundException extends RuntimeException {
        public BookNotFoundException(Long id) {
                super("Book with id " + id + " not found");
        }
}
```

Service

```
@Service
    public class BookService
    { @Autowired
        private BookRepository repository;

        public Book getBook(Long id) {
                return repository.findById(id)
                                .orElseThrow(() -> new BookNotFoundException(id));
        }
}
```

Global Exception handler:

```
@ControllerAdvice
public class GlobalExceptionHandler {

        @ExceptionHandler(BookNotFoundException.class)
        public ResponseEntity<String> handleBookNotFound(BookNotFoundException ex) {
                return
        ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
        }
```

In Service, we are invoking findById() which returns Optional, if not found we are throwing a custom exception. That custom exception will be caught by Global exception handler and returns NOT_FOUND message to User.

443 Spring Scenario 32: Transaction rollback failed

Suppose, you have a service method that saves Employee data and Department data. Both need to be saved in one transaction so that if anything goes wrong, neither is saved (rollback).

But what happened in your case?

- Employee was saved successfully.
- Department failed due to an exception.
- Rollback didn't happen means Employee record is still in the

database. Why rollback did not happen in this case?

Solution:

Springs @Transactional only rolls back automatically on **unchecked exceptions** (runtime exceptions).
By default:

- RuntimeException: triggers rollback
- Errors: triggers rollback
- Checked exception (like IOException, SQLException): does **not** trigger

rollback For checked Exceptions we need to use **rollBackFor** explicitly:

The rollbackFor attribute in Spring's @Transactional annotation is used to specify which Exception types should trigger a transaction rollback.

```
@Transactional(rollbackFor = {MyCheckedException.class, AnotherCheckedException.class})
public void doSomethingTransactional() throws MyCheckedException,
AnotherCheckedException {
        // ... business logic ...
        if (someCondition) {
                throw new MyCheckedException("Error condition met, rolling back.");
        }
}
```

When we use rollbackFor, it adds to the default rollback behaviour, rather than replacing it. This means that if we specify rollbackFor = {MyCheckedException.class}, the transaction will still roll back
on RuntimeExceptions and Errors, in addition to MyCheckedException.

444 Spring Scenario 33: Read-only transaction

Imagine You are building an **Employee Management System**. You have a method to fetch employees from the database. Since You are only reading data (no inserts, no updates),

How will make fetchEmployees() read-only?

Solution:

We need to use **@Transactional(readOnly = true)**

```
@Transactional(readOnly = true)
public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
}
```

The @Transactional(readOnly = true) annotation in Spring is used to declare that a method or class's operations within a transaction will only read data and will not modify it.

@Transactional(readOnly = true) has a lot of advantages.

1. performance improvement: read-only entities are not dirty-checked
2. memory saving: snapshots of persistent state are not maintained
3. data consistency: the changes of read-only entities are not persisted
4. database load: when we use master-slave , or read-write replica set(or cluster), @Transactional(readOnly = true) makes us enable to connect to read-only DB.

445 Spring Scenario 34: Event driven

You have a POST API:

- When a new reel gets posted, you need to send notifications to N users (could be hundreds, thousands, or even millions).
- Key challenge: if you try to notify all users synchronously inside the request, the API call will be slow or even timeout.

So how do you design it?

Solution: Event-Driven Approach

We need a system to handle notifications asynchronously. Using,

Message Queues: RabbitMQ, Kafka, Google Pub/Sub, AWS SQS.

446 Spring Scenario 35: Improve initial loading time or Cold Start Optimization Techniques

Your Spring Boot application is taking too much time during the initial load. Users notice that the application takes 10–15 seconds before the first API call responds after startup.

Solution.

When we start the application, following things will happen:

- Loading JVM
- Initializing the Spring context.
- Initiating the beans.
- Connecting to DB
- Setting up security,

filters etc Steps to improve.

Use **actuator/startup** endpoint to analyse which beans are taking time.

Enable lazy initialization using @Lazy:

In Spring Boot, all beans get initialized at startup by default.

If we have complex beans (like DataSources, caches, external API clients), startup time will increase. So se @Lazy to load non-critical beans only when needed.

Tune the auto-configuration:

Spring Boot auto-configures does many things we may not use (JDBC, JPA, Security, Actuator, etc.). This adds to startup.

Use @SpringBootApplication(exclude = {DataSourceAutoConfiguration.class}) for things that are not needed.

Use Spring AOT + Native image.

Instead of compiling bytecode into native code at runtime (like the JVM with JIT does), AOT compiles the code ahead of time into a native executable. In traditional Java applications, the Just-in-Time (JIT) compiler compiles code during runtime, which can introduce some delays. With AOT,

Spring 6 aims to overcome these delays by performing the compilation ahead of time, which improves startup
performance and resource usage.

447 Spring Scenario 36: Public some APIs

You are building an e-commerce application with Spring Boot.

- Pages like Home, Product Listing, and Contact Us should be accessible to everyone (public).
- Pages like Add Product, Delete Product, and User Management should only be accessible to Admins.

Solution:

Step 1: Define roles

In the application, we will have two main roles:

- ROLE_USER: can view products, browse, buy.
- ROLE_ADMIN: can do everything a user can, plus manage products and users.

Step 2: Secure endpoints with Spring Security

We use Spring Security (SecurityFilterChain) to configure which URL endpoints are public and which require authentication.

Using requestMatchers we can mention which APIs are public and which are should be accessed by admin.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

        @Bean
            public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
        Exception { http
                        .csrf().disable()
                        .authorizeHttpRequests()
                        // Public pages
                        .requestMatchers("/", "/home", "/products",
            "/contact").permitAll()

                        // Admin-only pages
                        .requestMatchers("/admin/**").hasRole("ADMIN")

                        // Everything else requires login
                        .anyRequest().authenticated()

                        .and()
                        .formLogin() // default login page
                        .and()
                        .logout();

                return http.build();
        }
```

448 Spring Scenario 37: Only ADMIN operation

Imagine You are building an Employee Management System for a company. The system has two main types of users:

1. Regular employees: They can log in, update their own profile, and view their payslip.

2. Admins (HR team): They can add new employees, update details of any employee, and delete employees if needed.

Now, deleting a user is a sensitive operation.

How you implement this example Delete operation should be done only by Admins?

Solution:

Spring Security gives a clean way to enforce this kind of restriction at the method level use PreAuthorize or PostAuthorize.

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long userId) {
        // delete logic here
}
```

- When anyone calls deleteUser(...), Spring Security checks the logged-in user.
- If that user doesn't have the role ADMIN, Spring throws an AccessDeniedException before the method body is even executed.
- If the user does have the role, then and only then the deletion

happens. Note: We can also achieve it using requestMatchers

```
http.authorizeHttpRequests()
.requestMatchers(HttpMethod.DELETE, "/employees/**").hasRole("ADMIN");
```

This works fine as long as delete is always triggered via the REST endpoint /employees/{id}. Any call to that endpoint requires the ADMIN role.

But, Since deleting employees is a sensitive business action, method-level security is the safer choice. Why? Because:

1. Security should be with the business logic (the delete method), not just with one endpoint.
2. If later we add another controller, a scheduled task, or an internal service call deleteEmployee(), the security rule should still apply.

449 Spring Scenario 38: Monitoring the Application

The operation teams want to monitor the application. how can you give APIs to them to monitor?

Solution:

Spring Boot provides **Actuator** which gives production-ready monitoring APIs. By enabling it, we can expose endpoints that the Operations team can consume (Refer Questions No.336 Spring 84)

450 Spring Scenario 39: Inter-service communication

Imagine you have a big e-commerce platform. It is not just one big application. Instead, you have split it into multiple services (microservices). For example:

- User Service: manages users, profiles, authentication
- Order Service: manages orders, cart, checkout
- Payment Service: processes payments
- Inventory Service: keeps track of stock

How you make communication between the services?

Approaches:

1. Synchronous Communication:

Example: The Order Service makes a REST API call to the Payment Service (POST /payments) and waits until it gets a success or failure response.

Tools we can use:

- REST (HTTP/JSON): simple and widely used
- gRPC: faster, binary protocol, good for high-performance

communication In Spring, we can use RestTemplate or WebClient(Refer

Questions No. 411 Rest 29)

2. Asynchronous Communication (Message driven)

Example: The Order Service publishes an event like OrderCreated to a message broker. The Payment Service listens for that event, processes payment

Tools we can use:

- Kafka
- RabbitMQ
- Google Pub/Sub / AWS SQS

451 Spring Scenario 40: Searching Million Records

You are building a REST API for an e-commerce app. The dataset contains millions of products. The client asks for search functionality where users can quickly find products by name, category, or description.

Explanation:

if we use simple LIKE %keyword% queries on such a huge dataset, performance will degrade badly because SQL has to look at every single row, one by one.

For large datasets, we cannot rely on plain database queries. You need a search optimization strategy.

Database indexing (Refer Question No. 372 Spring 120)

- Create indexes on frequently searched fields (e.g., product name, category).
- This makes queries like WHERE name LIKE 'abc%' much faster.

Full-Text Search

- Use native DB support (FULLTEXT INDEX in MySQL, tsvector in PostgreSQL).

External Search Engines (for very large scale)

- Use tools like **Elasticsearch or OpenSearch**.
- They're built for text search can handle millions of records efficiently.

Caching

- Cache frequently searched queries with Redis to avoid hitting DB repeatedly.

Full-Text Search:

FULLTEXT is a special index designed for text search. It helps you search big text fields (like names, descriptions, articles, blogs) much faster and smarter.

When we add a FULLTEXT index, SQL creates its own “search-friendly” copy of the text data. So instead of checking row by row, SQL jumps directly to matching results.

Reference:

MySQL FULLTEXT Indexes: Usage & Examples

MySQL and PostgreSQL for Advanced Full-Text Search.

452 Spring Scenario 41: Process Million Records (NOTE: It is about processing not searching)

How would you handle a situation where a Java application needs to process millions of records efficiently?

Explanation:

If we just load everything into memory and loop over it, the JVM will blow up with OutOfMemoryError. So, we need to design this smartly.

1. Implement pagination (Refer Question No. 382 Spring 130)

Instead of pulling all records into memory, fetch them in chunks (pagination).

2. Use Spring Data JPA Stream APIs

Instead of fetching everything at once. Fetch records little by little, and process them as they come. SQL databases can return results in a **streaming fashion** instead of dumping all rows at once

Example:

```
@Repository
interface PostRepository extends JpaRepository<Post, Long> {



    @Query("""
            select
        p from Post
        p
            where date(p.createdOn) >=
        :sinceDate """
    )
    @QueryHints(
            @QueryHint(name = AvailableHints.HINT_FETCH_SIZE, value =
        "25")
}
```

- This repository method fetches posts created on or after a given date using JPQL.
- The @QueryHint sets the fetch size to 25, so JPA loads rows in chunks instead of all at once.
- It returns a Stream<Post>, allowing lazy, memory-efficient processing of results.

Reference: Spring Data JPA — batching using Streams | by Patrik Hörlin | Predictly on Tech |

Medium The best way to use Spring Data JPA Stream methods - Vlad Mihalcea

3. Parallelize the work

Millions of records usually mean *independent tasks*. We can opt for parallel processing. Or use a

thread pool (ExecutorService) if we want more control.

```
listOfRecords.parallelStream()
.forEach(this::processRecord);
```

4. Batch processing

If the job involves writing back to DB, instead of writing them row by row better to collect and write in batches.

```
List<Customer> batch = new ArrayList<>();
    for (Customer c :
    records) { batch.add(c);
            if (batch.size() == 1000) {
        customerRepository.saveAll(batch);
        batch.clear();
        }
}
```

5. Asynchronous / Queue-based processing

We can use Kafka, RabbitMQ, etc. for high scalability. Pust the records into Queue and let consumers process them in parallel.

453 Spring Scenario 42: OutOfMemory due to huge data.

How did you reduce memory consumption in a high-load system? A system with 500,000 users had frequent OutOfMemoryError. How would you prevent it?

1. Use Streams and Pagination instead of Loading Everything

Instead of fetching all 500,000 users into memory, always use pagination (Pageable in Spring Data JPA) or Java Streams with @QueryHint(FETCH_SIZE) where supported.

This way we only hold a fraction of the dataset in memory at a time.

2. Reduce Object Retention with Weak/Soft References

Often memory leaks happen because objects stay referenced longer than needed (e.g., static caches, thread locals).

Use WeakReference or SoftReference for caches where data can be reloaded instead of holding strong references forever.

```
private Map<String, WeakReference<Product>> cache  = new ConcurrentHashMap<>();
```

3. Tune Collections and Object Sizes

Replace heavy collections with memory-efficient ones:

- Use ArrayList instead of LinkedList for large lists.
- Use EnumSet or EnumMap instead of HashMap if keys are enums.
- Avoid unnecessary String duplication , use intern() where safe, or use StringBuilder instead of + in loops.
- Prefer primitives instead of Wrapper classes when possible . Use int[], long[], etc. instead of List<Integer>, List<Long> when ever applicable .

```
EnumMap<Category, List<Product>> productsByCategory = new EnumMap<>(Category.class);
```

454 Spring Scenario 43: Writing to DB by two threads

How did you handle a situation where two threads were updating the same record in the database?

Solution:

When two threads trying to update the same record in the database at the same time. This creates a race condition. Sometimes one thread's changes would overwrite the other's, leading to inconsistent data.

We need to use a technique called "**Optimistic Locking**."
In Hibernate/JPA, this is simple to set up. We add a @Version field to the entity. Hibernate automatically manages this field whenever an update happens.

How It Works

1. Each record has a version number (integer or timestamp).
2. When Thread A reads the record, it sees version = 1. Thread B gets the same version = 1.
3. Thread A updates the record, Hibernate increments version : 2.
4. Thread B tries to update based on version = 1. Hibernate detects mismatch (because DB now has version = 2).
5. Thread B's transaction fails with an OptimisticLockException.
6. Instead of silently losing data, we catch this exception and retry the operation with fresh

data. Whenever we get OptimisticLockException, we can retry

Example:

```
@Entity
    public class Product
    { @Id
        private Long id;
        private String name;
    @Version
        private Integer version; // Hibernate manages this
}
```

```
@Transactional
public void updateProduct(Long productId, String newName) {
        boolean success =
    false; int retries = 3;

        while (!success && retries > 0) {
                try {
                        Product product =
            productRepository.findById(productId).orElseThrow();
            product.setName(newName);
                        productRepository.save
            (product); success = true;
                    } catch
            (OptimisticLockException e) {
            retries--;
                        if (retries == 0) {
                                throw e; // give up after retries
                        }
                }
```

455 Spring Scenario 44: Parallel API call

Suppose you have to make N number of APIs. for example, there could be 1000 users for each user you need to make API call. how would you do that?

Solution:

1. Async with CompletableFuture

We can do parallel calls with the combination of @Async and CompletableFuture.

Async method to run in the background.

```
@Async
public CompletableFuture<String> syncUser() {
      String response = restTemplate.getForObject("https://api.external.com/sync/",
String.class);
    return CompletableFuture.completedFuture(response);
}
```

Making API calls by iterating users.

```
List<CompletableFuture<String>> futures = users.stream()
                    .map(this::syncUser)
                    .collect(Collectors.toList());

// Wait for all to complete
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
```

2. Reactive Approach (WebClient + Flux)

If we use on Spring WebFlux, WebClient is non-blocking and much more efficient for massive N.

```
Flux.fromIterable(users)
    .flatMap(user ->
    webClient.get()
                .uri("https://api.external.com/sync/{id}", user.getId())
                .retrieve()
                .bodyToMono(String.class),
        100 // max 100 parallel calls
)
.collectList()
.block();
```

Here max concurrency is 100, means Only 100 calls will run at the same time. The remaining 900 will wait in the reactive pipeline queue.

456 Spring Scenario 45: Secure the App

How would you secure the Spring application that should be accessed only to registered users?

Solution:

1. Add `spring-boot-starter-security` starter dependency to the project.

2. Implement Authentication like BasicAuth or Token Based Authentication

3. Implement Authorization, means Apply role-based (RBAC) for access control. Example: Admin can delete users; a normal user cannot.

4. **Encrypt the sensitive data:** For example, passwords must be encrypted before saving to DB. Use BCryptPasswordEncoder

5. **Transport Security (TLS):** To protect data in transit, always use HTTPS instead of plain HTTP. Redirect all HTTP traffic to HTTPS so no user accidentally connects over an insecure channel.In Spring Boot, enable SSL in the application:

```
server:
ssl:
      enabled: true
      key-store:
  classpath:keystore.p12 key-store-
  password: yourpassword key-store-
```

6. Input Validation & Sanitization

- Validate all inputs (length, type, allowed values).
- Escape or sanitize data before storing/using.
- Protect against SQL Injection, XSS, Command Injection.
- Use parameterized queries with JPA/JDBC instead of string concatenation.

7s. Rate Limiting & Throttling

- Protect APIs from brute-force or denial-of-service attacks.
- Implement rate limiting (e.g., 100 requests/min per IP).
- Use tools like Bucket4j, Redis, or API Gateway throttling.

457 Spring Scenario 46: Prevent SQL Injection

How do you prevent SQL injections in Spring boot App?

let's understand how SQL Injection Happens, Causes and Prevention.

SQL Injection (SQLi) is a type of security vulnerability where an attacker manipulates your SQL queries by injecting malicious input. Attacker can read, modify, or delete the database without proper authorization

This can happen when user input is inserted directly into SQL queries without any safety checks. This allows an attacker to manipulate the query and run any SQL they want.

Example:

Let's say we have written query like this to accept a username from Frontend and return user details based on username

```
String username = request.getParameter("username");

String query = "SELECT * FROM users WHERE username = '" + username +"'";
```

If an attacker enters this as the username: '; DELETE FROM users; --

Always let the database handle user input safely.

```
String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, username);
ps.setString(2, password);
ResultSet rs = ps.executeQuery();
```

Here, even if the user types '; DELETE FROM users; --, it is treated as a literal value, not SQL code.

b) Use ORM Properly (JPA / Hibernate)

Avoid concatenating strings in queries. Use parameters:

```
@Query("SELECT u FROM User u WHERE u.email = :email")
User findByEmail(@Param("email") String email);
```

458 Spring Scenario 47: Prevent XSS attacks

How do you prevent XSS attacks in Spring boot App?

let's understand how XSS Injection Happens, Causes and Prevention.

Cross-Site Scripting (XSS) happens when an attacker manages to inject malicious scripts into a website so the other users will execute in their browsers. Essentially, the attacker tricks the browser into running their JavaScript.

Impact: leading to data theft, session hijacking, or redirection to malicious sites

How XSS Happens

The attacker looks for a web page where users can submit input, like a comment box, without proper validation or escaping.

They write a small JavaScript snippet to do something harmful, like stealing cookies or session info.

The attacker submits the script in the comment box. The backend stores it in the database without cleaning it.

When other users visit the page and see the comment, their browsers automatically run the attacker's script, letting the attacker steal information or manipulate the page.

Causes of XSS:

No input validation

- The app doesn't check or clean what users type before showing it.
- Example: allowing <script> in a comment.

No output escaping

- Special characters like < and > aren't converted, so the browser treats them as code instead of plain text.

Using innerHTML unsafely

- Setting innerHTML with raw user input can run scripts directly on the page.

Prevention

Input Validation

Always validate and escape all user input to ensure it is treated as text, not executable code.

Sanitize Output

Encode HTML entities when displaying user input.

String safeComment = StringEscapeUtils.escapeHtml4(userComment)

Content Security Policy (CSP):

Implement a CSP header to instruct the browser on which sources are allowed to execute scripts, effectively blocking injected, untrusted scripts
Enable XSS Protection and prevent clickjacking.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

        @Bean
            public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
        Exception { http
                        .headers(headers -> headers
                                // 1. Content Security Policy
                                .contentSecurityPolicy(csp ->
                                        csp.policyDirectives("default-src 'self';
                    script-src 'self'"))

                                // 2. XSS Protection
                                .xssProtection(xss -> xss.block(true))

                                // 3. Clickjacking prevention
                                .frameOptions(frame -> frame.sameOrigin())
                        );

                return http.build();
        }
```

- default-src 'self': only allow scripts from your own domain.
- xssProtection(true): blocks some reflected XSS attacks.
- frameOptions sameOrigin: prevents your pages from being loaded in a malicious iframe.

459 Spring Scenario 48: Best Practices while designing the REST API

When we design REST APIs, the first thing to keep in mind is **clarity**. Our endpoints should tell a story. For example, if we want all users, use `/users`. If we want a specific user, use `/users/{id}`. Avoid vague names like `/getUserInfo`.

Next comes **HTTP methods**. Use them for their true purpose. GET for fetching data, POST for creating, PUT or PATCH for updating, and DELETE for removing. Don't mix them up. A GET request should never modify data.

Status codes are the language back to the client. If something is created successfully, say 201 Created, not just 200 OK. If input is wrong, return 400 Bad Request. These codes make debugging and integration easier.

Now, **validation**. Never trust incoming data blindly. Use Spring's validation annotations (@Valid, @NotNull, etc.) to ensure the request is clean before processing it.

For **error handling**, don't scatter try-catch blocks everywhere. Centralize them with @ControllerAdvice. This way, clients get consistent error responses instead of random stack traces.

Security is important. Use authentication (JWT, OAuth2, BasicAuth if simple), and always validate who is accessing the endpoint. Also, never return sensitive details from the responses.

Finally, consider **versioning**. APIs evolve, and breaking old clients is a bad move. Start with /api/v1/... and move to /api/v2/... when needed.

And one last thing, document **the APIs**. Whether it is Swagger or OpenAPI, make sure anyone can understand and test the endpoints without guessing.

460 Spring Scenario 49: Design the Entity Relationships

Imagine you are implementing a system where Students and Departments need to be saved in DB. How would you design Entity and relationships

S

oluti

on

Entit

ies

- Department: Represents an academic department (like Computer Science, Mechanical, etc). One department can have many students.
- Student: Represents an individual student. Each student belongs to exactly one department.

So, the relationship here is:
One Department can contain Many Students (One-to-Many). At the same

time Many Students can be there in One Department (Many-to-One).

Department.java

```
@Entity
public class Department {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        private String name;

        // One Department -> Many Students
        @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
        private List<Student> students;

        // getters and setters
}
```

Student.java

```
@Entity

public class Student {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        private String name;

        // Many Students -> One Department
    @ManyToOne
        @JoinColumn(name = "department_id") // foreign key column
        private Department department;

        // getters and setters
}
```

In Department Entity, In @OnetoMany, we have used mappedBy, mappedBy tells “Don’t create a separate join table or foreign key for this collection. The ownership is on the Student entity, look at its department field for the actual mapping.”

On the Student side, we define @JoinCloumn which tells “In the students table, create a column department_id and use it as a foreign key referencing the department table.”

This is how tables look like:

department Table

department_id	name	location
1	Computer Science	Block A
2	Mechanical Engg	Block B
3	Civil Engg	Block C

student Table

student_id	name	roll_number	email	department_id
101	Ram	CS001	alice@college.edu	1
102	Charan	CS002	bob@college.edu	1
103	Mahesh	ME001	charlie@college.edu	2

student_id	name	roll_number	email	department_id
104	Diana	CE001	diana@college.edu	3
105	Raja	CS003	ethan@college.edu	1

Explanation of Relationship

- Each student has a department_id foreign key that links to the department table.
- A department can have many students, but each student belongs to only one

department. So, in DB terms:

- department_id in student is a foreign key referencing department.department_id.

461 Spring Scenario 50: End to End High level Design

Build a system like **Swiggy or Uber Eats**, where users can order food from restaurants, and restaurants manage their menu.

Interview Tip: While designing the systems:

- Start by clearly defining the problem and scope. Identify core entities and their relationships.
- Choose technologies for backend, frontend, database, cloud and include best practices.
- Design the APIs and clearly mention HTTP methods that are going to used. Explain architecture layer by layer, including async processing if needed.
- Cover scalability, reliability, CI/CD, logging and monitoring.
- Optionally, mention extra features like real-time notifications or analytics, Disaster recovery and failure alerts.

1. Backend (Server-side)

- **Framework**: Spring Boot
- **Responsibilities**:
  - Manage **Users**, **Restaurants**, **Menu Items**, **Orders**, **Delivery Agents**.
  - Business logic like calculating delivery ETA, order status updates, and payment processing.
  - Authentication & authorization for customers, restaurants, and admins.
- **Architecture**:
  - Controller > Service > Repository layers.
  - DTOs for requests/responses.
- **Best Practices**:
  - Implement Pagination for restaurant listings.
  - Have proper status codes for order creation, cancellation, etc.
  - Handling Exceptions with @ControllerAdvice.

2. Frontend (Client-side)

- **Framework**: React or Angular for web; React Native or Flutter for mobile.
- **Responsibilities**:
    - Show restaurant menus, allow searching/filtering, place orders.
    - Customer dashboard to track orders.
    - Restaurant dashboard to manage menu and order queue.
- **Integration**:
    - Communicate with backend via REST APIs or GraphQL.
- **UX Considerations**:
    - Lazy loading restaurant images and menu items.

3. Database (Storage Layer)

- **Choice**: SQL (PostgreSQL/MySQL)
- **Why SQL?**
    - Structured data: Users, Orders, Menu, Restaurants.
    - ACID compliance for payments and order status.
    - Relationships:
        - User > Orders (One-to-Many)
        - Restaurant > Menu Items (One-to-Many)
        - Order > Menu Items (Many-to-Many)
- **Optional NoSQL**: MongoDB for storing analytics, logs, or user activity streams.

4. Cloud Deployment

- **Provider**: AWS / GCP / Azure
- **Services**:
    - Backend: Spring Boot on Kubernetes / App Engine.
    - Frontend: React hosted on Kubernetes.
    - Database: Managed RDS (SQL) or Cloud SQL.
- **Benefits**: Scalability, automatic backups, monitoring, and resilience.

5. Containerization

- **Docker** for backend, frontend, and worker services (like sending notifications).
- **Kubernetes** to manipulate multiple services and scale dynamically.

6. CI/CD Pipeline

- **Tools**: GitHub Actions, Jenkins,
- **Pipeline**:
    1. Code pushed > run unit tests.
    2. Build Docker images.
    3. Push images to container registry.
    4. Deploy to staging > run integration tests.

5. Promote to production automatically or manually.

7. DevOps & Monitoring

- Logging: Cloud logging.
- Metrics & Monitoring: Prometheus + Grafana or Cloud monitoring.
- Alerts: Email, Slack, or SMS on errors.
- Backup: Have backup plans for Data.

8. Optional Extras for Scalability

- **Message Queues**: Kafka or RabbitMQ for handling notifications, order processing, and payment updates asynchronously.
- **Caching**: Redis or Memcached for popular restaurants, menu items, or user sessions.

462 Spring Scenario 51: Measuring Execution Time

Point to Note: Before going through the Scenarios on AOP, first go through AOP questions. Refer question no. 342 -344 Spring 90-92

Scenario: Your API performance is degrading, and you want to know which method is slow.
Question: How can you capture execution time automatically?

Solution:

Spring AOP gives us a cleaner way: write the logic once, and apply it across multiple methods using an **aspect**.

We need to use @Around, it defines an advice that executes both before and after a matched method execution (join point)

```
@Aspect

	@Around("execution(* com.example.service..*(..))")
	public Object measureTime(ProceedingJoinPoint joinPoint) throws Throwable {
		long start = System.currentTimeMillis();

		Object result = joinPoint.proceed(); // run the target method

		long end = System.currentTimeMillis();
		long duration = end - start;

		System.out.println(joinPoint.getSignature() + " executed in " + duration
	+
"ms");


		return result;
	}
```

The pointcut expression execution(* com.example.service..*(..)) says:
apply this advice to any method inside the com.example.service package (and subpackages).

joinPoint.proceed() is where the original method actually executes.

When we call orderService.placeOrder(), the aspect intercepts it, and we will get the output:

```
void com.example.service.OrderService.placeOrder() executed in 501ms
```

463 Spring Scenario 52: Centralized Exception Handling

Scenario: You want to log or take action whenever any service methods throws an exception
Question: How would you do it?

Solution:

We can use Spring AOP's @AfterThrowing advice. This advice runs only if a matched method throws an exception, which makes it perfect for centralized exception logging, monitoring, or even sending alerts.

```
@Aspect
@Component
public class ExceptionLoggingAspect {



    private static final Logger logger =
LoggerFactory.getLogger(ExceptionLoggingAspect.class);

        // Pointcut: match all methods in service package
    @Pointcut("execution(* com.example.service..*(..))")
    public void serviceMethods() {}

        // Advice: runs only when exception is thrown
    @AfterThrowing(pointcut = "serviceMethods()", throwing = "ex")
    public void logServiceException(Exception ex) {
                logger.error("Exception caught in service method: {}", ex.getMessage(),
        ex);
                // Here we can also add custom monitoring/alerting logic
        }
}
```

464 Spring Scenario 53: Transaction Management

Scenario: You want all service methods annotated with @Transactional to roll back on failure.

Question: How AOP helps here?

Solution:

When Spring sees @Transactional, it doesn't change the method code directly. Instead, it creates an AOP proxy around that bean. This proxy intercepts method calls, starts a transaction before the actual method runs, and then decides what to do afterwards, means it commit if everything goes fine, or roll back if an exception occurs.

So, we write plain business logic in the service methods, while the transactional concerns (begin, commit, rollback) are handled transparently by the AOP proxy.

465 Spring Scenario 54: Validate Tenant for all service methods

Scenario: Every service method should validate a tenant ID for multi-tenant architecture.

Question: How do you enforce it without duplicating code in every method?

Solution:

If we add tenant checks manually in every service method, we end up duplicating code and developers will forget it in some place. That is exactly the kind of cross-cutting concern AOP is meant to solve.

Custom Annotation:

```
@Aspect
@Component
public class TenantValidationAspect {

        @Before("within(@org.springframework.stereotype.Service *)")


        public void validateTenant() {
                String tenantId = TenantContext.getTenantId(); //Read tenant from
        Context

                if (tenantId == null || tenantId.isBlank()) {
                        throw new IllegalStateException("No tenant ID found in
            context!");
                }

                System.out.println("Validated tenant: " + tenantId);
```

The @Before advice runs before any service method is executed. If the tenant is missing, it blocks the call.

466 Spring Scenario 55: Custom Annotations

Scenario: You need to mark methods with @TrackExecution and only those should be logged.

Question: How would you achieve this with AOP??

Solution: Create custom

annotation Step 1: Create the

Annotation

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TrackExecution {
}
```

@Target(ElementType.METHOD): only valid on methods

@Retention(RetentionPolicy.RUNTIME): must be available at runtime for AOP to pick it up

@Aspect:

```
@Aspect
@Component
public class ExecutionTrackerAspect {

    private static final Logger log =
LoggerFactory.getLogger(ExecutionTrackerAspect.class);

        @Around("@annotation(com.example.TrackExecution)")
        public Object logExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
                long start =

        System.currentTimeMillis(); Object

        result = pjp.proceed();

                long duration = System.currentTimeMillis() - start;
                        log.info("Method {} executed in
                 {} ms",
                 pjp.getSignature().toShortString(),
                 duration);
```

@Around("@annotation(com.example.TrackExecution)") tells Spring to only execute when a method is annotated with @TrackExecution

467 Spring Scenario 56: Load huge data where pagination is not possible

Here we need all the data at once. And it takes multiple tables to join and fetch the data. how you implement it and how will you reduce the latency?

Solution:

Backend aggregation

Don't push raw massive datasets to the frontend. Instead, fetch and aggregate them on the backend . Return the graph-ready format (series, categories, values).

Use optimized queries

Write a single optimized SQL with proper joins instead of firing multiple queries. Ensure you only fetch the required columns.

Materialized views / pre-aggregation

For very large datasets, create materialized views.

Database level

- Add proper indexes on join keys and filter columns.
- Partition tables if data is huge (time-based partitioning is common).
- Avoid SELECT *. Only fetch what we need for the graph.

Caching

- Cache heavy results (in Redis or application cache).
- If the data changes slowly, you don't need to hit the DB every time.

Compression & serialization

- Compress large JSON responses (gzip).

Unit and Integration Testing

468 Testing: Questions on Testing

1. What are different types of testing you do as developer?

Unit Testing: Testing individual classes or methods in isolation. Example: testing a service method that calculates discount.

Integration Testing: Testing how multiple components work together. Example: making sure the Spring Service talks correctly to a Repository and database.

Functional/End-to-End Testing: Testing the actual flow from API layer to DB. Example: hitting a REST endpoint and checking the full response.

2. What is Unit Testing and how would write Junit testcases?

Unit testing means testing a small piece of code (like one method) without external systems (DB, network).

With **JUnit 5** and **Mockito** (Mockito is a popular open-source mocking framework for Java used in automated unit tests), a simple test looks like this:

```
//Service we want to test
public class CalculatorService {
public int add(int a, int b) {
          return a + b;
  }
}

//Test Class
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class CalculatorServiceTest {
  private final CalculatorService calculator = new CalculatorService();

  @Test
  void testAdd() {
```

Key points:

- Use @Test annotation to mark test methods.
- Assertions like assertEquals, assertTrue, etc., check the result.
- For Spring components, we often mock dependencies using **Mockito**.

3. What is Integration Testing and what tools you have used?

Integration testing checks how layers work together, like Controller > Service > Repository >

Database. In Spring Boot, we often use:

- **@SpringBootTest**: loads full application context.
- **@DataJpaTest**: tests JPA layer with an in-memory DB like H2.
- **MockMvc**: to test REST APIs.

```
@SpringBootTest
@AutoConfigureMockMvc
class UserControllerTest {

        @Autowired
        private MockMvc mockMvc;

        @Test
            void testGetUser() throws Exception
        { mockMvc.perform(get("/users/1"))
                                .andExpect(status().isOk())
                                .andExpect(jsonPath("$.name").value("John"));
        }
}
```

4. What is difference TDD and BDD?

TDD: Test Driven Development is an iterative development process where tests for new functionality are written *before* the actual code. The standard TDD cycle is often summarized as "Red-Green-
Refactor."

1. **Red:** Write a failing test. This ensures that the test is valid.
2. **Green:** Write just enough code to make the test pass.
3. **Refactor:** Optimize the code, ensuring that the test still passes after refactoring.

BDD (Behaviour-Driven Development)

- Extension of TDD, but focuses on behaviour of the system.
- Uses natural language (Given, When, Then).
- Example with Cucumber in Java:

```
Scenario: Add two numbers
Given I have a calculator
When I add 2 and 3
  Then the result should be 5
```

5. Explain the lifecycle of a JUnit test.

Lifecycle includes:

1. @BeforeAll (setup for all tests, runs once).
2. @BeforeEach (setup before each test).
3. Test methods (@Test).
4. @AfterEach (cleanup after each test).
5. @AfterAll (cleanup for all tests, runs once).

6. What is @Test annotation and what its difference in JUNIT4 vs JUNIT5

@Test tells the JUnit framework that a particular method should be treated as a test and executed when tests are run

In JUnit 4, @Test came from the org.junit package. Test methods had to be declared as public, and if we wanted to check for an exception, we used a special syntax inside the annotation itself

```
@Test(expected = IllegalArgumentException.class)
public void shouldThrowException() {
        // code that throws exception
}
```

JUnit 5 changed the experience by introducing a new architecture called Jupiter. The @Test annotation moved to org.junit.jupiter.api, and we no longer had to make them public. Exception handling became cleaner too, because instead of writing expected exceptions in the annotation, we could simply use an assertion like assertThrows, which made the test more readable

```
@Test
    void shouldThrowException() {
    assertThrows(IllegalArgumentException.class, () -> {
                // code that throws exception
        });
}
```

7. How do you approach testing in Spring boot applications

- We usually begin with unit tests. These tests focus on individual classes, like a service method or a utility function. They don't load the full Spring context, and dependencies are mocked.
- REST APIs can be tested in two main ways. If we want to test only the controller , we use @WebMvcTest, which loads just the web layer.
- If we want to test the whole flow like controller, service, repository, and even database then we use @SpringBootTest combined with MockMvc.
- For database-specific testing, Spring Boot gives @DataJpaTest, which brings up an in-memory database like H2 so that we can test the JPA repositories without touching a real database.

8. How do you perform unit testing in Spring Boot?

Performing unit testing in Spring Boot usually involves JUnit 5 and Mockito. The idea is to test only one class at a time while mocking out its dependencies.

Suppose we have a UserService that depends on a UserRepository. In a unit test, we don't connect to the real database; instead, we tell Mockito to return fake results when the repository is called. That way, we can verify if the service logic works as expected.

```
@SpringBootTest
class UserServiceTest {

        @MockBean
        private UserRepository userRepository;

        @Autowired
        private UserService userService;

        @Test
            void shouldReturnUserById() {
        when(userRepository.findById(1L)).thenReturn(Optional.of(new User(1L, "John")));
                User user =
        userService.getUser(1L);
        assertEquals("John", user.getName());
        }
}
```

9. Which starter package do you need to test the spring boot application?

We simply add spring-boot-starter-test in the pom.xml, and we get access to all the popular

testing libraries without having to add them one by one.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

10. What dependencies does the spring-boot-starter-test brings to the class-path?

- JUnit as the main testing framework
- Mockito for mocking dependencies,
- AssertJ for fluent and human-readable assertions
- Hamcrest for matcher-based assertions.
- For applications that deal with JSON and XML, JsonPath and XmlUnit are also included.
- Spring Test itself is part of the package, enabling integration with the Spring Boot test framework.

11. How do you perform Integration testing in Spring boot?

Integration testing in Spring Boot is all about verifying how different layers of the application work together.

For example, suppose we have a UserController that talks to a UserService, which in turn talks to a UserRepository. An integration test makes sure that the flow works correctly end to end. In Spring Boot, the simplest way to do this is with @SpringBootTest. This annotation starts the entire application context and allows us to send HTTP requests to the controllers using MockMvc or TestRestTemplate.

```
@SpringBootTest // Loads the full application context
@AutoConfigureMockMvc // Configures and injects MockMvc
class UserControllerIT {

        @Autowired
        private MockMvc mockMvc;

        @Test
            void shouldReturnUserById() throws Exception {
        mockMvc.perform(get("/users/1"))
                              .andExpect(status().isOk())
                              .andExpect(jsonPath("$.name").value("John"));
        }
}
```

12. How is @ContextConfiguration used in Spring Boot?

@ContextConfiguration comes from the older Spring testing framework. It is used to specify which configuration classes or XML files should be loaded to create the application context for a test. It is used for configuring the ApplicationContext for integration tests, For instance, before Spring Boot, we would write something like this:

```
@ContextConfiguration(classes = AppConfig.class)
class LegacySpringTest {

}
```

In Spring Boot, we rarely need @ContextConfiguration directly because @SpringBootTest

usually handles context loading. However, we might still see it in cases where we want fine-grained control over which beans are loaded, or if we are testing a very specific slice of the application without starting the full Boot context.

13. What does @SpringBootTest do? How does it interact with @SpringBootApplication and @SpringBootConfiguration?

When we annotate a test class with @SpringBootTest, we are telling Spring Boot to start up the full application context, almost as if we run the application with @SpringBootApplication. That means all the beans, properties, and auto-configurations are loaded.

Under the hood, @SpringBootTest builds on top of @SpringBootConfiguration and @SpringBootApplication. While @SpringBootApplication is what we put on the main application class to bootstrap the app, @SpringBootTest finds that class and uses it to initialize the context in the test environment

14. What is the difference between @ContextConfiguration and @SpringBootTest?

@ContextConfiguration belongs to the core Spring framework and is mainly used to load a minimal or custom context, typically pointing to specific configuration classes. It doesn't know about Spring Boot's auto-configuration or starters.

@SpringBootTest, on the other hand, is Boot related. It not only loads the application's configuration but also applies all the auto-configuration and Boot-specific features.

15. How can you test Spring Boot REST controllers?

If we only want to test the controller layer in isolation, without loading services or repositories, we use @WebMvcTest. This annotation creates a minimal context that includes only the web components. Dependencies like services can be mocked with @MockBean.

```
@WebMvcTest(UserController.class)
class UserControllerTest {

        @Autowired
        private MockMvc mockMvc;

        @MockBean
        private UserService userService;

        @Test
            void shouldReturnUserById() throws Exception {
        when(userService.getUser(1L)).thenReturn(new User(1L, "John"));

                mockMvc.perform(get("/users/1"))
                            .andExpect(status().isOk())
                            .andExpect(jsonPath("$.name").value("John"));
        }
}
```

If we prefer to test the full flow, including database calls, then we use @SpringBootTest with MockMvc or TestRestTemplate. This way we are not mocking the service or repository; instead, we are checking if everything works together.

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class UserControllerFullStackTest {

        @Autowired
        private TestRestTemplate restTemplate;

        @Test
        void shouldReturnUserById() {
                User user = restTemplate.getForObject("/users/1", User.class);
        assertEquals("John", user.getName());
        }
}
```

16. @AutoConfigureMockMvc vs @WebMvcTest

When we are testing REST controllers in Spring Boot, two annotations often come up: @AutoConfigureMockMvc and @WebMvcTest. At first glance, they might look like they serve the same purpose, but they actually live at different levels.

@WebMvcTest only work on Web layer like controllers, JSON converters, filters, and so on. Nothing else is loaded. Services and repositories don't exist in this slice unless we explicitly mock them. This is perfect when the goal is to check whether a controller maps endpoints correctly, validates input, or serializes output.

```
@WebMvcTest(UserController.class)
class UserControllerTest {

        @Autowired
        private MockMvc mockMvc;

        @MockBean
        private UserService userService;

        @Test
            void shouldReturnUserById() throws Exception {
        when(userService.getUser(1L)).thenReturn(new User(1L, "John"));

                mockMvc.perform(get("/users/1"))
                                .andExpect(status().isOk())
                                .andExpect(jsonPath("$.name").value("John"));
        }
}
```

Notice what happens here: the web context is bootstrapped, but UserService doesn't exist until we create a mock with @MockBean. That is because @WebMvcTest doesn't load the service or repository layer at all.

On the other side, @AutoConfigureMockMvc comes into play when are already using @SpringBootTest. While @SpringBootTest starts the full application context (controllers, services, repositories, database connections), adding @AutoConfigureMockMvc wires up the MockMvc bean so that we can test our
REST endpoints without starting a real HTTP server. In other words, we are testing the controller in the context of the whole application.

Example:

```
@SpringBootTest
@AutoConfigureMockMvc
class UserControllerIT {

        @Autowired
        private MockMvc mockMvc;

        @Test
            void shouldReturnUserFromDatabase() throws Exception {
        mockMvc.perform(get("/users/1"))
                                .andExpect(status().isOk())
                                .andExpect(jsonPath("$.name").value("John"));
        }
}
```

In this setup we don't create fake services or repositories. The test uses the real database (often an in- memory one like H2 or sometimes Testcontainers), runs the actual service code, and then calls the controller.

17. How do you test @Repository?

Repositories in Spring are gateway to the database. They handle queries, inserts, updates, and deletes. Testing them isn't about mocking, because mocking defeats the purpose, we want to be sure that the queries we write actually work against a real database.

The usual way is to use an **in-memory database** like H2 or a lightweight containerized database (with Testcontainers). Spring Boot makes this super easy.

```
@DataJpaTest
class UserRepositoryTest {

        @Autowired
        private UserRepository userRepository;

        @Test
        void shouldFindUserByEmail() {
                User saved = userRepository.save(new
        User("macha@test.com")); User found =
        userRepository.findByEmail("macha@test.com");

                assertThat(found.getEmail()).isEqualTo(saved.getEmail());
        }
```

Here, we didn't spin up the whole application. Just the JPA part and an in-memory DB. This test checks if the query method in the repository is actually returning what we expect.

18. When do you want @DataJpaTest for? What does it auto-configure?

We use @DataJpaTest for writing tests for repositories or JPA logic. It auto-configures everything we need for JPA:

- An in-memory database (by default, H2).
- Scans only for @Entity classes and Spring Data JPA repositories.
- Configures Hibernate and JPA mappings.
- Rolls back each test by default, so the DB stays clean.

So instead of loading the entire Spring context, You are just testing the persistence layer in isolation. It is lightweight and fast.

19. How would write unit testcases for services?

```
class UserServiceTest {

        @Mock
        private UserRepository userRepository;

        @InjectMocks
        private UserService userService;

        @Test
            void shouldReturnUserWhenEmailExists() {
        when(userRepository.findByEmail("test@mail.com"))
                                .thenReturn(new User("test@mail.com"));
                User result = userService.findByEmail("test@mail.com");

        assertThat(result.getEmail()).isEqualTo("test@mail.com");

        }
}
```

Services are where our business logic lives. Unlike repositories, here we don't want a real DB most of the time. Instead, we mock the repository layer and just test the logic.

Here, we don't test how the repository works (that is tested elsewhere). we only test if the service is working correctly or not.

20. What is Mockito?

Mockito is a Java testing framework that helps to create mocks. A mock is a fake object that simulate the behaviour of real objects, so we can test our class without worrying about dependencies. Imagine the service depends on a repository. Instead of hitting the real DB, Mockito gives a fake repository. We tell it how to behave (like "if someone calls findByEmail, return this dummy user"). That way, we test only the logic in the service, not the repository itself.

21. How do you mock external services / external APIs?

When the code talks to the outside world like a payment gateway, weather API, or an email service, we don't want our tests to actually call them. That would be slow and might even cost money. Instead

In Spring Boot, we usually wrap the external API in a **service class** (say, WeatherClient). Then in your tests, we mock that client.

```
class WeatherServiceTest {

        @Mock
        private WeatherClient weatherClient;

        @InjectMocks
        private WeatherService weatherService;

        @Test
            void shouldReturnSunnyWeather() {
        when(weatherClient.getWeather("Hyderabad"))
                                .thenReturn("Sunny");
                String result = weatherService.fetchWeather("Hyderabad");

        assertThat(result).isEqualTo("Sunny");

        }
}
```

The actual HTTP call never happens. We just “pretend” the client gave a response. This keeps our tests fast and focused only on how our code handles the response.

If we want to test the whole HTTP layer itself, we can also use tools like **WireMock** or **MockWebServer** (These act like fake servers that the code hits during the test.)

22. What is Spy annotation

The @Spy annotation in Mockito is used to create a spy instance of a real object. Spies are useful to perform partial mocking, where some methods of the object are mocked while others retain
their original behavior. This is particularly useful when we want to test a class with some real method calls and some mocked method calls.

Example:

Let’s say we have calculator class.

```
class Calculator {
        int add(int a, int b) { return a + b; }
        int multiply(int a, int b) { return a * b; }
}
```

Test class with @Spy.

Reference: Mockito @Spy Annotation Tutorial

23. What are the differences between @MockBean and @Mock annotations? Both create mocks, but they are different.

```
class CalculatorTest {

        @Spy
        private Calculator calculator;

        @Test
        void spyExample() {
                // Real add method will be called
        assertThat(calculator.add(2, 3)).isEqualTo(5);

                // Mock multiply to return a fake value
        doReturn(100).when(calculator).multiply(2, 3);

                assertThat(calculator.multiply(2, 3)).isEqualTo(100);
        }
}
```

- @Mock is from Mockito. It is used in **unit tests**, outside the Spring context. Its a plain JUnit + Mockito.
- @MockBean is from Spring Boot Test. It replaces a bean in the **Spring ApplicationContext** with a mock.

For example, if we are testing a controller with @WebMvcTest:

```
@WebMvcTest(UserController.class)
class UserControllerTest {

        @Autowired
        private MockMvc mockMvc;

        @MockBean


        private UserService userService; // replaces bean in context

        @Test
            void shouldReturnUser() throws Exception {
        when(userService.findByEmail("test@mail.com"))
                                .thenReturn(new User("test@mail.com"));

                mockMvc.perform(get("/users/email/test@mail.com"))
                                .andExpect(status().isOk());
        }
}
```

@MockBean tells Spring: “Don’t load the real UserService, use this fake one instead.”
If we use @Mock, the Spring context won’t know about it, and the test would fail because the controller still expects a bean.

24. How do you Mock the beans in Spring boot?

The easiest way is exactly what we saw above: @MockBean.

Whenever the Spring-managed class depends on another bean, and we don’t want the real one, we just replace it with @MockBean.

25. @InjectMock vs @Mock

When we start writing tests with Mockito, these two annotations often sit next to each other. At first, they look similar, but they actually serve two very different purposes.

Let's take an example. Imagine we have an OrderService that depends on a PaymentService:

```
class PaymentService {
        public String process() {
                return "real payment done";
        }
}

class OrderService {
        private final PaymentService paymentService;

        public OrderService(PaymentService paymentService) {
                this.paymentService = paymentService;
        }

        public String placeOrder() {
                return paymentService.process();
        }
}
```

Now, suppose we want to test OrderService. You don't really care about whether money is transferred , we just want to check that OrderService behaves correctly when the payment goes through. This is where @Mock and @InjectMocks come in.

@Mock: When we put @Mock on a class, Mockito creates a fake version of it. None of the real logic runs unless we tell it to. Instead, we control its behaviour in the test.

```
@Mock
private PaymentService paymentService;
```

Here, paymentService is no longer real. It is just a dummy object

@InjectMocks: Now comes @InjectMocks. This tells Mockito: *"Create the real object of this class, but if it has dependencies, inject the mocks into it."*

```
@InjectMocks
private OrderService orderService;
```

Mockito sees that OrderService needs a PaymentService, and since we already created a mock of it,

```
class OrderServiceTest {

        @Mock
        private PaymentService paymentService; // fake

    @InjectMocks
    private OrderService orderService;      // real class, mock injected

    @Test
        void shouldReturnApprovedWhenPaymentSucceeds() {
        when(paymentService.process()).thenReturn("Approved");
            String result = orderService.placeOrder();

        assertThat(result).isEqualTo("Approved");
```

it wires the fake into the real OrderService.

Complete sample:

In this test, OrderService is real, but the PaymentService inside it is fake. That is exactly what we want in a unit test: test one class in isolation, while replacing its dependecies with mocks.

When to use each

- Use **@Mock** whenever we want to create a fake version of a class and fully control its behaviour in tests. We don't want the real logic at all.
- Use **@InjectMocks** when we are testing a class that has dependencies. It creates the real object and injects the mocks automatically, so we don't need to manually set up the class.

Questions by Experience Level

Please Note: These questions are segregated based on fundamental knowledge but in real time its highly depends on company, role and interview process. So please don't stick to these it is just for an idea.

469 Junior Level Spring

Core

21. What is Spring Framework and why is it used?
22. What is Dependency Injection (DI)?
23. Difference between @Component, @Service, and @Repository.
24. What are Spring bean scopes?
25. Difference between ApplicationContext and BeanFactory.
26. How does @Autowired work?
27. What is @Qualifier in Spring?
28. What is the purpose of @Configuration?
29. Explain Spring bean lifecycle.
30. How do you define beans in XML vs using annotations?

Spring Boot

31. What is Spring Boot and its advantages?
32. Explain @SpringBootApplication.
33. What is dependency injection, and how does @Autowired help?
34. What are Spring Boot starters?
35. Difference between Spring and Spring Boot.
36. What is the role of application.properties?
37. What is an IOC container in Spring?

REST

38. What is HTTP and which methods do you commonly use?
39. What are common HTTP status codes (200, 404, 500)?
40. What is REST API?
41. Explain the difference between @RestController and @Controller.
42. Can you explain how you would create a simple REST API in Spring Boot?
43. Difference between PUT and PATCH.
44. What is Idempotency?

470 Mid-Level Developer

Spring Core

1. Explain AOP (Aspect Oriented Programming).
2. How does @Transactional work?
3. What are proxy objects in Spring?
4. Explain bean lifecycle in detail.
5. What are @Primary and @Qualifier?
6. How do you implement custom annotations in Spring?
7. How does Spring manage circular dependencies?
8. What is the difference between Singleton and Prototype beans?
9. Explain constructor injection vs setter injection.

Spring Boot

10. How does auto-configuration in Spring Boot work?
11. What is @Configuration vs @Bean?
12. How do profiles work in Spring Boot (@Profile use case)?
13. How do you handle global exception handling in Spring Boot (@ControllerAdvice)?
14. application.properties vs application.yml , when to prefer which?
15. What is the use of CommandLineRunner or ApplicationRunner?
16. What happens if two beans of the same type are created? How do @Qualifier and @Primary help?
17. How does the embedded Tomcat server work in Spring Boot?

REST API

18. Difference between @RequestMapping and @GetMapping.
19. @PathVariable vs @RequestParam.
20. How do you validate request inputs in Spring Boot REST APIs?
21. How would you secure a REST endpoint?
22. How do you handle exceptions in REST APIs and return proper status codes?
23. Difference between @RequestBody and @ResponseBody.
24. How to implement global exception handling in REST?
25. How do you secure REST APIs with Spring Security?
26. What are Spring Boot Actuator endpoints?
27. How do you enable pagination and sorting in Spring Data REST?

Database & JPA

28. What is ORM and why do we use it?
29. What is JPA vs Hibernate?

30. Explain @Entity, @Id, @GeneratedValue.
31. Explain lazy vs eager loading with an example.
32. How do you handle relationships like OneToMany in JPA?
33. What's the difference between save() and saveAndFlush() in Spring Data JPA?

Testing

34. What is JUnit, and what are its key annotations?
35. What is Mockito? When would you use @Mock vs @Spy?
36. What is @SpringBootTest vs @WebMvcTest?
37. How do you test REST controllers with MockMvc?

Security

38. Authentication vs Authorization, explain with examples.
39. What is Spring Security, and how does it integrate with REST APIs?
40. How would you secure an endpoint with JWT tokens?
41. Difference between 401 and 403 HTTP codes.\
42. Git commands, Git rebase vs Merge

471 Senior Developer 7+

Advanced Spring Boot

1. Explain the Spring bean lifecycle. How can you hook into it?
2. What are proxies in Spring (JDK vs CGLIB)? Where do they matter?
3. How do you inject a prototype bean into a singleton?
4. How does Spring Boot Actuator help in production monitoring?
5. How would you build a custom health check endpoint?
6. What is @ConditionalOnProperty, and where have you used it?
7. How do you implement custom metrics & monitoring with Micrometer/Prometheus?
8. What are Scopes in Spring? How do they affect microservices apps?
9. How does Spring Boot auto-configuration work internally?
10. How does Spring handle circular dependencies?
11. Explain difference between JDK dynamic proxies and CGLIB proxies.
12. How would you profile and optimize a Spring application?
13. How do you implement distributed caching in Spring (Redis/Hazelcast)?
14. How does @Transactional work under the hood with proxies?

Advanced JPA & Databases

15. What is the N+1 select problem in JPA? How do you fix it?
16. When would you use @Query vs Criteria API vs method naming?
17. Explain optimistic locking (@Version) vs pessimistic locking.
18. How do you handle pagination and sorting in Spring Data JPA?
19. What is connection pooling? Why is HikariCP the default in Spring Boot?
20. How do you implement DB migrations in Spring Boot (Flyway/Liquibase)?
21. How do you analyze and optimize a slow SQL query?

Advanced Security

22. Explain OAuth2 + JWT flow in Spring Boot.

23. What's the difference between stateless and stateful authentication?
24. How do you implement role-based access control (RBAC) in Spring Security?
25. What's the difference between 401 Unauthorized vs 403 Forbidden?
26. How does CSRF protection work in Spring Security?
27. How do you secure microservice-to-microservice communication?
28. How does Spring Security filter chain work internally?
29. How do you design API versioning in REST?
30. How do you implement rate limiting in REST APIs?
31. Explain JWT authentication in detail.
32. How would you implement service-to-service authentication in microservices?
33. How do you design REST APIs for high scalability?
34. How do you handle distributed transactions across microservices?
35. How do you debug and profile slow REST endpoints?

Microservices & Architecture

36. Monolith vs Microservices?
37. How do Microservices communicate? (REST, gRPC, Messaging, Event-driven).
38. What is an API Gateway? Why use it?
39. What is Service Discovery and how does it work (Eureka/Consul)?
40. What is the Circuit Breaker pattern (Resilience4j/Hystrix)? When would you use it?
41. How do you implement API rate limiting?
42. How do you design for eventual consistency in microservices?
43. How do you handle a distributed transaction (Saga pattern)?
44. How do you implement distributed logging & tracing (Zipkin, ELK)?
45. What happens when one service becomes slow? How would you isolate failures?
46. One of your microservice is down then how do you handle?

System Design & Scalability

47. How would you design an e-commerce system (catalog, orders, payments)?
48. SOLID Principles and Design patterns
49. How would you design scalable notification system?
50. How do you handle millions of users without crashing the DB?
51. What is CQRS pattern? When to use it?
52. What is database sharding? When is it required?
53. How do you ensure high availability and fault tolerance?
54. How do you design a notification system (email/SMS/push)?
55. What are the different caching strategies in distributed systems? (Redis, Hazelcast)?

Production Debugging Scenarios

56. Your Spring Boot app takes 45s to start in prod, but 5s locally. How do you debug?
57. After deployment, REST APIs are responding in >5s. What's your debugging approach?
58. Your service crashes with OutOfMemoryError when traffic spikes. How do you fix it?
59. A microservice returns 503 during peak load. What's happening?
60. Users are logged out every 30 min despite JWT being valid for 24h. Root cause?

472 Spring Cheat Sheet

1. Core Spring (IoC & DI)

- IoC (Inversion of Control): Spring manages object lifecycle.
- DI (Dependency Injection): Inject dependencies via:
    - Constructor injection (preferred)
    - Setter injection
    - Field injection (discouraged)
- Bean Scopes:
    - singleton (default)
    - prototype
    - Web scopes: request, session, application, websocket
- Bean Lifecycle:
  Instantiate: Dependency Injection > @PostConstruct > @PreDestroy.
- Annotations:
    - Stereotypes: @Component, @Service, @Repository, @Controller, @RestController
    - Config & Bean definitions: @Configuration, @Bean, @Scope, @Lazy
    - Injection & qualifiers: @Autowired, @Qualifier, @Primary, @Value
    - Scanning & imports: @ComponentScan, @Import
    - Conditional configs: @Profile, @Conditional
    - Ordering: @Order, Ordered interface
- Advanced Concepts:
    - BeanFactory vs ApplicationContext (lazy vs eager loading, features)
    - BeanPostProcessor, BeanFactoryPostProcessor (customize bean lifecycle)
    - FactoryBean (produce other beans)
    - Environment & PropertySources (externalized config)
    - Circular dependencies: resolve with @Lazy or restructuring

2. Spring Boot Basics

- Main Annotation
    - @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan
- Key Features
    - Auto-configuration (based on classpath & beans)
    - Starter dependencies (spring-boot-starter-*)
    - Embedded servers (Tomcat, Jetty, Undertow)
    - JAR packaging (spring-boot-maven-plugin)
- Configuration
    - application.properties / application.yml
    - Profiles: spring.profiles.active=dev

    - Externalized config: Environment variable, command line arguments, system properties
- Dev Tools
    - spring-boot-devtools: auto-restart, live reload
- Run Options
    - SpringApplication.run() customization
    - CommandLineRunner / ApplicationRunner for startup logic
- Actuator (Basics)
    - /actuator/health, /actuator/info
    - For monitoring & health checks

3. Web & REST (Spring MVC)

- DispatcherServlet > Front controller.
- Annotations:
    - @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping.
    - @PathVariable, @RequestParam, @RequestBody, @ResponseBody.
    - @ControllerAdvice + @ExceptionHandler for global error handling.
- Response:
    - ResponseEntity<T> for status + headers + body.
- CORS:
    - @CrossOrigin, or configure via WebMvcConfigurer.

4. Data Access

- Spring JDBC:
    - JdbcTemplate, NamedParameterJdbcTemplate.
- Spring Data JPA:
    - Repos: CrudRepository, JpaRepository, PagingAndSortingRepository.
    - Derived queries:findByName, findByAgeGreaterThan.
    - Custom queries: @Query, JPQL, native SQL.
- Entities:
    - @Entity, @Table, @Id, @GeneratedValue.
    - Relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany.
- Transactions:
    - @Transactional.
    - Propagation: REQUIRED, REQUIRES_NEW, MANDATORY, etc.
    - Isolation: READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE.
- Lazy vs Eager Loading
    - Default to FetchType.LAZY.
    - Avoid FetchType.EAGER (leads to N+1 problems and large queries).
- N+1 Query Problem
    - Use JOIN FETCH in JPQL or @EntityGraph to pre-fetch relationships when needed.
- 2nd Level Cache (L2 Cache)
    - Enable Hibernate L2 cache with Ehcache/Redis.

    - Cache static reference data.
- Query Cache
    - Enable query caching for repeated queries.
    - Use carefully.
- Batch Inserts/Updates
    - Group writes using Hibernate batching.
- Avoid Select
    - Explicitly fetch only needed columns.

5. Spring Boot Advanced Features

- Profiles: Environment-specific beans with @Profile("dev").
- Actuator:
    - Endpoints: /actuator/health, /actuator/metrics, /actuator/env.
    - Custom endpoints with @Endpoint / @ReadOperation
    - Integrates with Prometheus, Grafana.
- Scheduling & Async:
    - @EnableScheduling, @Scheduled(fixedRate=1000).
    - @EnableAsync, @Async for background tasks
    - Async exception handling with AsyncUncaughtExceptionHandler
- Caching:
    - @EnableCaching, @Cacheable, @CachePut, @CacheEvict.
    - Cache providers: Caffeine, Redis, Ehcache, Hazelcast

6. AOP (Aspect Oriented Programming)

- Concepts:
    - Aspect: cross-cutting concern.
    - Join point: method execution.
    - Advice: code run at join points.
    - Pointcut: expression matching join points.
- Annotations:
    - @Aspect, @Before, @After, @Around, @AfterReturning, @AfterThrowing.

7. Security

- **Spring Security Basics**
    - Filters: SecurityFilterChain (replaces WebSecurityConfigurerAdapter)
    - @EnableWebSecurity
    - DelegatingFilterProxy: Ties security filters into servlet filter chain
- **Authentication**
    - In-memory users (InMemoryUserDetailsManager)
    - JDBC-based authentication (with UserDetailsService)
    - LDAP-based authentication (common in enterprises)
    - JWT-based (stateless APIs)
    - OAuth2 / OpenID Connect (social login, SSO)

- **Authorization**
    - Role-based access: hasRole, hasAuthority
    - Method-level: @PreAuthorize, @Secured, @RolesAllowed
    - URL-based:.authorizeHttpRequests() in security config
- **Password Management**
    - PasswordEncoder (e.g., BCryptPasswordEncoder)
- **Filters & Interceptors in Security**
    - Security filters (auth, CSRF, CORS) run before controllers
    - Custom filters: extend OncePerRequestFilter
    - Custom interceptors for logging/auditing
- **CORS (Cross-Origin Resource Sharing)**
    - Configure via @CrossOrigin or global config in WebMvcConfigurer

8. Web Security

- CSRF Protection
    - Enabled by default in Spring Security (CsrfFilter)
    - Use CSRF tokens in forms (<input type="hidden" name="_csrf">)
    - Disable only for stateless APIs with JWT
- XSS Protection
    - Escape output in views (Thymeleaf auto-escapes by default)
    - Use Spring's HtmlUtils.htmlEscape() for manual escaping
    - Set response headers (X-XSS-Protection, Content-Security-Policy)
- SQL Injection Prevention
    - Always use prepared statements / parameterized queries
    - With Spring Data JPA : repository methods prevent injection
    - Avoid string concatenation in queries ("WHERE name='" + input + "'")

9. Testing

- Test Annotations:
    - @SpringBootTest: full context.
    - @WebMvcTest: controllers only.
    - @DataJpaTest: JPA only.
- Mocking:
    - @MockBean, Mockito.
    - MockMvc for REST testing.
- Database Tests:
    - @AutoConfigureTestDatabase, H2 in-memory DB.

10. Filters & Interceptors

- **Servlet Filters** (javax.servlet.Filter / OncePerRequestFilter)
    - Run before request reaches DispatcherServlet
    - Used for logging, authentication, CORS, request wrapping

- **Spring Interceptors** (HandlerInterceptor)
    - Run before and after controller execution
    - Good for auditing, request validation, modifying model attributes
- **Difference**:
    - Filters: generic to all requests (Servlet level)
    - Interceptors: Tied to Spring MVC request lifecycle, Tied to controllers

11. Reactive (Spring WebFlux)

- When to use: Non-blocking, async, high-throughput apps.
- Types:
    - Mono<T>: 0/1 element.
    - Flux<T>: 0…N elements.
- Programming Model:
    - Annotation style (@RestController).
    - Functional style (RouterFunction).

12. Spring Cloud & Microservices

- Components:
    - Eureka (service registry).
    - Config Server (centralized config).
    - Feign Client (REST calls).
    - Gateway (API routing).
    - Resilience4j (Circuit Breaker).
    - Sleuth + Zipkin (tracing).
- Distributed Config:
    - Git-backed configs, refresh with /actuator/refresh.

13. Validation

- Bean Validation (JSR 380):
    - @NotNull, @NotBlank, @Size, @Email, @Pattern, @Min, @Max.
    - Custom validator with ConstraintValidator.
- Usage:
    - @Valid on method params in REST controllers.

14. Events

- Publishing:
  applicationEventPublisher.publishEvent(new CustomEvent(...));
- Listening:
  @EventListener(CustomEvent.class).
- Built-in Events:
  ContextRefreshedEvent, ContextClosedEvent, ApplicationReadyEvent.

15. Legacy (XML & Old Features)

- XML-based config (applicationContext.xml).
- context:component-scan, bean definitions.

16. Deployment & Observability

- Zero Downtime:
  - Graceful shutdown (server.shutdown=graceful).
  - Readiness + liveness probes.
- Monitoring:
  - Spring Boot Actuator + Micrometer.
  - Logs: SLF4J, Logback.
- Packaging:
  - Executable JAR/WAR.
  - Deploy on Docker, Kubernetes, or Cloud.

17. Performance

Startup Optimization

- Lazy Init: spring.main.lazy-initialization=true
- Limit @ComponentScan scope
- Remove unused starters/dependencies
- Analyze startup with Spring Boot Startup

Actuator Runtime Performance Optimization

- Switch to WebClient (reactive, non-blocking) instead of RestTemplate
- Connection pooling: HikariCP (tune max pool size)
- Cache: Spring Cache (Redis, Caffeine)
- Asynchronous processing: @Async, Kafka, queues
- Batch inserts/updates: JPA batching, bulk

APIs Database Performance

- Use proper indexes (single, composite, covering indexes)
- Optimize queries with JPQL/Criteria instead of fetching everything
- Fetch strategies: prefer lazy loading but balance with JOIN FETCH where needed
- Pagination with Pageable to avoid loading huge datasets
- Avoid N+1 problem: use @EntityGraph, JOIN FETCH, or DTO projections
- Statement batching in JPA (hibernate.jdbc.batch_size)
- Connection pool tuning (HikariCP settings)
- Use database-specific features (partitions, materialized views, query hints)

API Performance Optimization

- **Compression**: Enable GZIP (server.compression.enabled=true in Spring Boot)
- **Response Size** : Use pagination (Pageable, Slice), projections, and DTOs to avoid over-fetching
- **Caching Strategies**
  - Client-side: Cache-Control, ETag, Last-Modified headers

- Server-side: Spring Cache (Redis, Caffeine), HTTP response caching
- **Connection Keep-Alive**: Reduce handshake overhead
- **Rate Limiting / Throttling**: Bucket4j, Resilience4j
- **Asynchronous APIs**: WebFlux, @Async, message queues (Kafka/RabbitMQ)

473 November

9. Spring Scenario 57

How can we migrate an existing Spring Boot application to a new database schema without any downtime?

When an application is running in production, stopping it even for a few minutes can cause problems for users. So, when we move to a new database schema, the goal is to make the change while the application is still live.

To achieve this safely, we need a clear plan.

1. **Plan the schema changes**
   Start by listing what needs to change in the database. For example, adding a new column, renaming a table, or changing a data type.
2. **Use a migration tool**
   Tools like **Flyway** or **Liquibase** help manage database versions. These tools run scripts in a
   proper order so that every environment (development, test, production) has the same structure.
3. **Make the changes backward compatible**
   Before updating the application code, make sure the new schema works with the old version too. For example, if we are adding a new column, keep the old column until the new version is fully deployed.
4. **Deploy in phases**
   First, update the database schema. Then, deploy the new application code that uses it. This avoids errors during the transition.
5. **Test carefully**
   Run tests in a staging environment to check that both the old and new code can run with the new schema.
6. **Monitor after release**
   After the deployment, monitor logs and database activity to ensure everything works smoothly.

Example:

Suppose we are changing the column name user_name to full_name.

1. First, add a new column full_name without removing user_name.
2. Update the application code to start using full_name.
3. After confirming everything works fine, remove user_name later in another release.

10. How do you fix the timeout errors in Spring boot application?

Timeout errors in Spring applications can arise from various sources, including long-running operations, network issues, or misconfigured timeout settings. Addressing these errors involves identifying the source and implementing appropriate solutions.

1. Configure Request Timeouts:

Spring MVC Async Request Timeout: For asynchronous requests in Spring MVC, configure the spring.mvc.async.request-timeout property in application.properties or application.yml. This sets the maximum time a request can take before being terminated.

Code: `spring.mvc.async.request-timeout=5000 # 5 seconds`

Database Transaction Timeout: For database operations, use the @Transactional annotation's timeout property to set a timeout for the transaction.

```
 @Transactional(timeout = 30) // 30 seconds
public void performDatabaseOperation() {
        // ...
}
```

External Service Calls (RestTemplate/WebClient): When making calls to external services, configure the connection and read timeouts for your RestTemplate or WebClient instance.

```
 @Bean
public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
                        .setConnectTimeout(Duration.ofSeconds(5)) // Connection timeout
                        .setReadTimeout(Duration.ofSeconds(10))  // Read timeout
                        .build();
}
```

2. Optimize Long-

Running Operations:

Asynchronous

Processing:

For tasks that are inherently long-running, consider using asynchronous processing with CompletableFuture, @Async, or message queues to offload the work and prevent blocking the main request thread.

Caching:

Implement caching strategies to store frequently accessed data and reduce the need for repeated expensive operations.

Database Query Optimization:

Optimize slow database queries by adding indexes, rewriting queries, or adjusting database configurations.